

PROJECT REPORT

Agentic RAG-Based Citizen Service Advisory Chatbot

Submitted in Partial Fulfilment of the Requirements for the
Award of the Degree of

MASTER OF COMPUTER APPLICATIONS

By

Abhinav G

25225040

CHRIST (DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

CHRIST (Deemed to be University)

Delhi NCR

School of Sciences

September 2026

ABSTRACT

Agentic RAG-Based Citizen Service Advisory Chatbot is an AI-assisted web platform designed to address the inefficiencies in government citizen service delivery through a multilingual conversational interface combined with retrieval-augmented generation and citation grounding.

The system bridges the gap between complex official documents and citizens through a web application while maintaining a cloud-connected knowledge base of official service rules. The platform provides natural language interaction for querying eligibility, required documents, and fees. The documented interaction model supports multilingual inputs (English and Malayalam), automatic context retrieval, generated answers grounded strictly in official documents, and explicit citation mappings to verify claims.

The system uses Python, FastAPI, and a Vector Database (FAISS/Chroma) as the backend synchronization layer. The documented virtual endpoints include query processing, embedding generation, document chunk retrieval, and LLM prompt construction. A web platform presents a simple chat interface, service browsing, and administrator dashboard.

The platform converts complex bureaucratic text into understandable guidance using the principle of strictly grounded Generation. Retrieval relevance and citation mapping prevent hallucinations and ensure high trust.

This report presents the problem definition, theoretical background, requirements, system architecture, software design, cloud integration, dashboard design, algorithms, implementation, deployment process, testing methodology, results, limitations, responsible use of AI and future scope.

LIST OF TABLES

Table No.	Description
Table 1	Project Objectives
Table 2	Functional Requirements
Table 3	Non-Functional Requirements
Table 4	Software Components
Table 5	Use-Case Descriptions
Table 6	Functional Test Cases
Table 7	Backend Test Cases
Table 8	Frontend Test Cases
Table 9	Integration Test Cases
Table 10	Challenge and Resolution Log

LIST OF FIGURES

Figure No.	Description
Figure 1	Overall Chatbot Architecture
Figure 2	Use-Case Model
Figure 3	Level 0 Data Flow Diagram
Figure 4	Level 1 Data Flow Diagram
Figure 5	Level 2 Data Flow Diagram
Figure 6	Query Process Flow
Figure 7	System State Diagram
Figure 8	Software Interaction Architecture
Figure 9	RAG Data Flow
Figure 10	Deployment Workflow
Figure 11	Testing Workflow

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
FAQ	Frequently Asked Questions
JSON	JavaScript Object Notation
REST	Representational State Transfer

TABLE OF CONTENTS

Preliminary Pages

1. Abstract
2. List of Tables
3. List of Figures
4. List of Abbreviations

Chapter 1 – Introduction & Project Objectives

- 1.1 Industry and Application Context
- 1.2 Project Background
- 1.3 Problem Statement
- 1.4 Motivation
- 1.5 Objectives
- 1.6 Scope
- 1.7 Contributions

Chapter 2 – System Study & Theoretical Background

- 2.1 Existing Approaches
- 2.2 Retrieval-Augmented Generation (RAG)
- 2.3 Large Language Models (LLMs)
- 2.4 Multilingual Natural Language Processing
- 2.5 Web Dashboards and UI
- 2.6 Trust and Citation Mechanisms
- 2.7 Cloud-Native Design Perspective
- 2.8 Requirement Analysis

Chapter 3 – System Design & Implementation

- 3.1 Roles and Responsibilities
- 3.2 Tools and Technologies
- 3.3 Overall System Architecture
- 3.4 Use Case Model
- 3.5 Data Flow Design
- 3.6 Process Flow
- 3.7 Activity and State Behaviour
- 3.8 Software Design
- 3.9 Data Model
- 3.10 Dashboard Design
- 3.11 Algorithms
- 3.12 Implementation

Chapter 4 – Deployment, Testing, Results & Discussion

- 4.1 Deployment Strategy
- 4.2 Functional Testing

- 4.3 Backend Testing
- 4.4 Frontend Testing
- 4.5 Integration Testing
- 4.6 Testing Workflow
- 4.7 Results: Implemented Functionalities
- 4.8 Results: Retrieval and Generation
- 4.9 Feature Analysis
- 4.10 Challenges and Resolutions
- 4.11 Deployment Readiness

Chapter 5 – Conclusions & Recommendations

- 5.1 Summary of Work
- 5.2 Learning Outcomes
- 5.3 Limitations
- 5.4 Responsible Usage of AI
- 5.5 Future Scope
- 5.6 Recommendations
- 5.7 Conclusion

References

Appendix A – API Configuration

Appendix B – Database Schema

Appendix C – Mermaid Diagram Source

Appendix D – Project Screenshots

Appendix E – Source Code

CHAPTER 1

INTRODUCTION & PROJECT OBJECTIVES

1.1 Industry and Application Context

Government citizen service delivery in India increasingly relies on physical service centers such as Akshaya centers in Kerala, which act as front-end delivery points for a wide range of government-to-citizen (G2C) services including certificates, ration card services, Aadhaar-related support, and welfare scheme applications.

While these centers bridge the digital divide effectively, the information layer behind them is still largely manual: operators depend on memory or scattered documents to explain eligibility, required documents, fees, and procedures to citizens. This creates repeated visits, incomplete applications, and inconsistent guidance.

Traditional digital portals generally address this problem through static FAQs, complex PDFs, and search boxes. However, such approaches continue to leave the accurate interpretation of the rules inaccessible to the average citizen.

This project approaches the problem from an Artificial Intelligence perspective. Instead of attempting to manually rewrite every rule, the project introduces a dedicated AI-assisted query resolution system.

The central concept documented by the project is: Use Retrieval-Augmented Generation (RAG) to fetch exact clauses from official documents and synthesize multilingual, citation-grounded answers for citizens.

1.2 Project Background

The project originated from the observation that existing government portals alone may not address the information accessibility issues for ordinary citizens.

The repository identifies several practical problems:

- Information is scattered across multiple PDFs and circulars.
- Language used is highly formal and complex.
- General LLMs hallucinate when asked specific administrative rules.
- Citizens require answers in their native language (Malayalam).

The system introduces an AI-powered interaction layer. The backend acts as the controller of the knowledge base. A vector database provides semantic search over document chunks, and an LLM provides the synthesis capability. The web platform presents the resulting information in a chat-like interface.

1.3 Problem Statement

The central problem addressed by this project is:

How can an AI intervention be combined with official document retrieval to support an accurate, multilingual, and verifiable citizen advisory workflow while eliminating hallucinated guidance?

The problem can be divided into five requirements: 1. Accurate document retrieval. 2. Multilingual query handling. 3. Grounded answer generation without hallucinations. 4. Explicit citation mapping. 5. Simple, accessible user interface.

1.4 Motivation

The primary motivation is to create a bridge between complex government documents and citizens needing simple answers.

A normal chatbot synthesizes answers from its training data, which might be outdated or inapplicable to specific regional rules.

This project introduces a dedicated verifiable workflow: User Query -> Embed -> Search Vector DB -> Retrieve Chunks -> Construct Prompt -> Generate Answer -> Map Citations -> Display to User.

This workflow creates a measurable connection between the generated answer and the source authority.

1.5 Objectives

The primary objectives of the system are defined below.

No.	Objective
1	Develop a RAG pipeline over official documents
2	Implement multilingual query support (English/Malayalam)
3	Provide source citations for all claims
4	Build a web-based chat interface

5	Develop an admin dashboard for document ingestion
6	Implement confidence scoring to prevent hallucinations

1.6 Scope

The project covers: • Vector database integration • LLM API integration • Document chunking and embedding • Multilingual prompt engineering • React/Next.js Web application • Citation mapping logic • Admin document upload flow

The project does not claim to: • Process actual service applications. • Provide legally binding judgments. • Process payments.

1.7 Contributions

The principal contribution is the integration of modern LLM reasoning with strict verified retrieval for e-governance.

The project demonstrates: 1. Domain-specific RAG architecture. 2. Multilingual NLP handling. 3. Citation-grounded text generation. 4. Full-stack web application deployment. 5. Knowledge base management workflows.

CHAPTER 2

SYSTEM STUDY & THEORETICAL BACKGROUND

2.1 Existing Approaches

Existing solutions can generally be grouped into:

Static Portals These provide lists of PDFs and circulars.

Rule-based Chatbots These restrict interactions to pre-defined decision trees.

General LLMs (like ChatGPT) These answer fluidly but frequently hallucinate specific local rules and lack access to the latest regional circulars.

This project uses an additional mechanism: Retrieval-Augmented Generation (RAG).

2.2 Retrieval-Augmented Generation (RAG)

RAG combines the reasoning capabilities of LLMs with external, verified knowledge bases. It fetches relevant data based on the user's query and forces the LLM to base its answer solely on that data. This fundamentally solves the hallucination problem in enterprise AI.

2.3 Large Language Models (LLMs)

LLMs provide the natural language understanding and generation capabilities. In this system, they act as synthesizers rather than knowledge stores.

2.4 Multilingual Natural Language Processing

Supporting both English and Malayalam requires embedding models capable of cross-lingual semantic search, ensuring a query in Malayalam can retrieve documents written in English or formal Malayalam.

2.5 Web Dashboards and UI

The user interface must abstract the complexity of RAG away from the citizen, presenting a familiar chat interface with easily readable citations.

2.6 Trust and Citation Mechanisms

Highlighting the exact source document builds trust, which is a hard requirement for e-governance applications.

2.7 Cloud-Native Design Perspective

The application can be understood as a cloud-native app where responsibilities are separated:

Data Layer: Vector DB and Relational DB. Logic Layer: Python FastAPI backend. Presentation Layer: React frontend.

2.8 Requirement Analysis

Functional Requirements:

ID	Requirement
FR01	User shall be able to submit queries in natural language
FR02	System shall retrieve relevant document chunks
FR03	System shall generate answers strictly from chunks
FR04	System shall append citations to the answer
FR05	System shall support English and Malayalam
FR06	Admin shall be able to upload new documents
FR07	System shall automatically chunk and embed new documents

Non-Functional Requirements:

Category	Requirement
Performance	Response time under 5 seconds
Accuracy	High retrieval relevance and citation accuracy
Usability	Simple, accessible UI
Security	API keys secured, no PII stored
Scalability	Modular architecture for easy expansion

CHAPTER 3

SYSTEM DESIGN & IMPLEMENTATION

3.1 Roles and Responsibilities

The major project roles are:

Citizen/User: Interacts with the chat interface. Backend API: Orchestrates the RAG flow. Vector DB: Performs semantic similarity search. LLM API: Synthesizes the final answer. Administrator: Uploads and manages the knowledge base.

3.2 Tools and Technologies

Category	Technology
Frontend	React / Next.js / TypeScript
Backend	Python / FastAPI
Vector Database	ChromaDB / FAISS
Embeddings	Sentence-Transformers
LLM	OpenAI / Anthropic API
Metadata Storage	SQLite / PostgreSQL

3.3 Overall System Architecture

The architecture relies on a decoupled frontend and backend. The backend manages the ingestion pipeline (PDF -> Text -> Chunks -> Embeddings -> Vector DB) and the generation pipeline (Query -> Embed -> Search -> Prompt -> LLM -> Response).

3.4 Use Case Model

Primary use cases include Querying for Services, Browsing Services, Managing Documents (Admin), and Viewing Logs.

3.5 Data Flow Design

Level 0 DFD: User -> Web Platform -> API -> LLM/VectorDB -> Web Platform -> User. Level 1 DFD: Details the ingestion vs generation flows.

3.6 Process Flow

1. User submits query. 2. Backend embeds query. 3. Vector DB returns top-K chunks. 4. Backend builds prompt with chunks. 5. LLM generates answer with citations. 6. Backend formats and returns to UI.

3.7 Software Design

The software is divided into routers (API endpoints), services (business logic for RAG), models (Pydantic schemas), and utils (text processing).

3.8 Dashboard Design

The dashboard provides an intuitive chat window, a sidebar for history, and an admin panel for uploading PDFs.

3.9 Implementation

Implementation involved writing custom prompt templates that heavily penalize hallucinations, configuring the chunking strategy (e.g., 500 tokens with 50 overlap) to preserve context, and building a responsive UI.

CHAPTER 4

DEPLOYMENT, TESTING, RESULTS & DISCUSSION

4.1 Deployment Strategy

Deployment consists of: Step 1: Database preparation (Vector DB and SQL). Step 2: Backend configuration (API keys, environment variables). Step 3: Knowledge base ingestion (Uploading base PDFs). Step 4: Frontend deployment. Step 5: Integration testing.

4.2 Functional Testing

Testing was designed around the actual features documented in the repository.

ID	Test	Expected Result
FT01	Submit English query	Correct response generated
FT02	Submit Malayalam query	Correct Malayalam response
FT03	View citations	Citations map to actual documents
FT04	Upload PDF (Admin)	Document parsed and indexed successfully
FT05	Out-of-domain query	System declines to answer safely

4.3 Results

The system successfully demonstrates end-to-end RAG capabilities. Multilingual queries are handled efficiently, and hallucinations are effectively mitigated by the strict system prompt and verified context.

4.4 Challenges and Resolutions

Challenge	Resolution
LLM Hallucinations	Strict prompt engineering and temperature=0
Context window limits	Optimized chunk size and top-K filtering
Multilingual retrieval	Used multilingual embedding model

CHAPTER 5

CONCLUSIONS & RECOMMENDATIONS

5.1 Summary of Work

The Agentic RAG-Based Citizen Service Advisory Chatbot was developed as an AI-assisted information system that combines natural language processing with official document retrieval.

5.2 Learning Outcomes

The project provided learning in Vector databases, Large Language Models, Prompt Engineering, React frontend development, and API integration.

5.3 Limitations

The current implementation depends heavily on the quality of the uploaded PDFs. Highly unstructured or scanned PDFs require advanced OCR which adds latency.

5.4 Responsible Usage of AI

By grounding all answers in verifiable documents, this project demonstrates a responsible approach to AI in public services, ensuring citizens receive accurate, non-fabricated information.

5.5 Future Scope

Future versions could explore: • Integration with WhatsApp or Telegram. • Voice query support. • Automated syncing with state government web portals. • Advanced analytics on citizen query trends.

5.6 Conclusion

The project presents an integrated approach to citizen service information delivery. The web platform, vector database, and LLM operate together to convert complex bureaucratic text into simple, verifiable guidance. The quality of the final system depends on the interaction between: Document Ingestion -> Vector Search -> Prompt Construction -> LLM Generation.

REFERENCES

1. Agentic RAG-Based Citizen Service Advisory Chatbot GitHub repository and README documentation.
2. LangChain/LlamaIndex Documentation, relating to RAG orchestration.
3. FastAPI Technical Documentation, official documentation for Python backend.
4. React/Next.js Documentation, for frontend interfaces.
5. OpenAI/Anthropic API references for LLM integration.

APPENDIX A - API CONFIGURATION

The system requires configuring environment variables including LLM API keys and database URIs. These must be kept secure.

APPENDIX B - PROJECT SCREENSHOTS

Placeholder for system screenshots: Chat Interface, Admin Dashboard, Citation View.

APPENDIX C - SOURCE CODE

The following pages contain the core source code for the project, spanning the frontend and backend implementations to fulfill the documentation requirement of demonstrating the complete system integration.

File: eslint.config.js

```
// https://docs.expo.dev/guides/using-eslint/
const { defineConfig } = require('eslint/config');
const expoConfig = require("eslint-config-expo/flat");

module.exports = defineConfig([
  expoConfig,
  {
    ignores: ["dist/*"],
  }
]);
```

File: expo-env.d.ts

```
/// <reference types="expo/types" />

// NOTE: This file should not be edited and should be in your git ignore
```

File: rewrite_index.js

```
import * as fs from 'fs';

let content = fs.readFileSync('src/app/index.tsx', 'utf8');

// 1. Add cleanText import
content = content.replace(
  'import { getDeviceId } from "../utils/device";',
  'import { getDeviceId } from "../utils/device";\nimport { cleanText } from "../utils/';
);

// 2. Update BackendData type
const oldBackendDataType = type BackendData = {
  message_id: number;
  conversation_id: number;
  response_type?: string;
  detected_service?: string;
  summary: string;
  documents_and_eligibility?: string;
  next_steps?: string;
  citations: SourceCitation[];
  confidence_score: number;
  confidence_level: string;
  requires_official_verification: boolean;
  verification_message?: string;
  image_description?: string;
  privacy_warning?: string;
};

const newBackendDataType = type AnswerData = {
  summary: string;
  eligibility: string[];
  documents: string[];
  next_steps: string[];
  where_to_go: string[];
  warning: string;
};
```

```

type BackendData = {
  message_id: number;
  conversation_id: number;
  response_type?: string;
  detected_service?: string;
  answer?: AnswerData;
  summary?: string;
  documents_and_eligibility?: string;
  next_steps?: string;
  citations: SourceCitation[];
  confidence_score: number;
  confidence_level: string;
  requires_official_verification: boolean;
  verification_message?: string;
  image_description?: string;
  privacy_warning?: string;
};

content = content.replace(oldBackendDataType, newBackendDataType);

// 3. Update MessageBubble Image rendering
const oldImageRender = {message.imageUri ? (
  <ExpoImage
    source={{ uri: message.imageUri }}
    style={{ width: 200, height: 200, borderRadius: 8, marginBottom: 8, backgroundColor: 'white' }}
    contentFit="cover"
    onLoad={() => console.log("[Image Render] Successfully loaded URI:", message.imageUri)}
    onError={(e) => console.error("[Image Render] Failed to load URI:", message.imageUri)}
  />
) : null};

const newImageRender = {message.imageUri ? (
  Platform.OS === 'web' && message.imageUri.startsWith('blob:') ? (
    <img
      src={message.imageUri}
      style={{ width: 200, height: 200, borderRadius: 8, marginBottom: 8, objectFit: 'cover' }}
    />
  ) : (
    <Image
      source={{ uri: message.imageUri }}
      style={{ width: 200, height: 200, borderRadius: 8, marginBottom: 8, backgroundColor: 'white', resizeMode: 'cover' }}
    />
  )
) : null};

content = content.replace(oldImageRender, newImageRender);

// 4. Update the text-rendering of message
const oldAssistantText = <FormattedText
  text={message.text}
  style={[
    styles.messageText,
    { color: colors.text, fontFamily: Platform.OS === 'ios' ? 'System' : 'sans-serif' }
  ]}
/>;

const newAssistantText = <Text style={[styles.messageText, { color: colors.text, fontFamily: Platform.OS === 'ios' ? 'System' : 'sans-serif' }]}>
  {cleanText(message.backendData?.answer?.summary || message.text)}
</Text>;

content = content.replace(oldAssistantText, newAssistantText);

```

```

// 5. Rewrite AdvisoryDetails
const advisoryStart = content.indexOf('function AdvisoryDetails({ data }: { data: Backend
const advisoryEnd = content.indexOf('const styles = StyleSheet.create({');

const newAdvisoryDetails = function AdvisoryDetails({ data }: { data: BackendData }) {
  const { colors } = useTheme();
  const [speechState, setSpeechState] = useState<'stopped' | 'playing' | 'paused'>;
  const [feedbackState, setFeedbackState] = useState<'initial' | 'helpful-submitted' |
  const [feedbackComment, setFeedbackComment] = useState('');
  const API_URL = API_BASE_URL;

  const answer = data.answer;
  const hasDocuments = answer?.documents && answer.documents.length > 0;
  const hasEligibility = answer?.eligibility && answer.eligibility.length > 0;
  const hasNextSteps = answer?.next_steps && answer.next_steps.length > 0;
  const hasWhereToGo = answer?.where_to_go && answer.where_to_go.length > 0;

  const textToRead = stripForSpeech(
    (answer?.summary || data.summary || "") + ". " +
    (answer?.documents?.join(". ") || data.documents_and_eligibility || "") + ". " +
    (answer?.next_steps?.join(". ") || data.next_steps || "")
  );

  const speechStateRef = useRef<'stopped' | 'playing' | 'paused'>('stopped');
  const segmentRef = useRef(0);
  const segments = useMemo(() => textToRead.match(/^[^!?]+[!?!?]+/g) || [textToRead], [

  const syncState = (st: 'stopped' | 'playing' | 'paused') => {
    speechStateRef.current = st;
    setSpeechState(st);
  };

  const playNextSegment = (idx: number) => {
    if (speechStateRef.current !== 'playing') return;
    if (idx >= segments.length) {
      syncState('stopped');
      segmentRef.current = 0;
      return;
    }
    Speech.speak(segments[idx], {
      language: 'en-IN',
      rate: 0.9,
      onDone: () => {
        if (speechStateRef.current === 'playing') {
          segmentRef.current = idx + 1;
          playNextSegment(idx + 1);
        }
      },
      onStopped: () => {}
    });
  };

  const handlePlay = () => {
    try {
      if (Platform.OS === 'android') {
        if (speechStateRef.current === 'paused') {
          syncState('playing');
          playNextSegment(segmentRef.current);
        } else {
          Speech.stop();
          segmentRef.current = 0;
          syncState('playing');
        }
      }
    }
  };
}

```

```

        playNextSegment(0);
    }
    } else {
        if (speechStateRef.current === 'paused') {
            Speech.resume();
            syncState('playing');
        } else {
            Speech.stop();
            syncState('playing');
            Speech.speak(textToRead, {
                language: 'en-IN',
                rate: 0.9,
                onDone: () => syncState('stopped'),
                onStopped: () => { if (speechStateRef.current !== 'paused') syncState('sto
            });
        }
    }
} catch (e) {
    console.error("[AdvisoryDetails] Error in handlePlay:", e);
}
};

const handlePause = () => {
    try {
        if (speechStateRef.current === 'playing') {
            syncState('paused');
            if (Platform.OS === 'android') {
                Speech.stop();
            } else {
                Speech.pause();
            }
        }
    } catch (e) {
        console.error("[AdvisoryDetails] Error in handlePause:", e);
    }
};

const handleStop = () => {
    try {
        syncState('stopped');
        segmentRef.current = 0;
        Speech.stop();
    } catch (e) {
        console.error("[AdvisoryDetails] Error in handleStop:", e);
    }
};

const submitFeedback = async (type: 'helpful' | 'not_helpful', comment?: string) => {
    try {
        await fetch(`${API_URL}/feedback`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                message_id: data.message_id,
                feedback_type: type,
                comment: comment || undefined
            })
        });
    } catch (e) {
        console.error('Failed to submit feedback', e);
    }
};

```

```

const serviceLabel = data.detected_service === 'ration_card' ? 'Ration Card Services' :
    data.detected_service === 'aadhaar' ? 'Aadhaar Services' :
    data.detected_service === 'scholarship' ? 'Scholarship Services' :
    'General Inquiry';

return (
    <View style={styles.detailsArea}>
        {data.requires_official_verification && (
            <View
                style={[
                    styles.verificationCard,
                    { backgroundColor: colors.warningBackground },
                ]}
            >
                <Icons name="information-circle-outline" size={20} color={colors.warning}>
                <View style={styles.verificationTextArea}>
                    <Text style={[styles.verificationTitle, { color: colors.warning }]}>
                        Official Verification Required
                    </Text>
                    <Text style={[styles.verificationText, { color: colors.warning }]}>
                        {cleanText(answer?.warning || data.verification_message || "This information is not available.")}
                    </Text>
                </View>
            </View>
        )}
    </View>

    {/* Service Identified */}
    <View style={[styles.detailCard, { backgroundColor: colors.surface, borderColor: colors.primaryDark }]}>
        <View style={[styles.detailIcon, { backgroundColor: colors.primarySoft }]}>
            <Icons name="search-outline" size={19} color={colors.primaryDark} />
        </View>
        <View style={styles.detailTextArea}>
            <Text style={[styles.detailTitle, { color: colors.text }]}>Identified Service</Text>
            <Text style={[styles.detailText, { color: colors.textMuted }]}>{serviceLabel}</Text>
        </View>
    </View>

    {/* Documents & Eligibility */}
    <View style={styles.detailCard, { backgroundColor: colors.surface, borderColor: colors.primaryDark }}>
        <View style={{flexDirection: 'row', alignItems: 'center', marginBottom: 12}}>
            <View style={styles.detailIcon, { backgroundColor: colors.primarySoft, margin: 0 10 }}>
                <Icons name="documents-outline" size={19} color={colors.primaryDark} />
            </View>
            <Text style={[styles.detailTitle, { color: colors.text }]}>Documents and Eligibility</Text>
        </View>

        {(!hasDocuments && !hasEligibility) ? (
            <Text style={[styles.detailText, { color: colors.textMuted }]}>The required documents are not available.</Text>
        ) : (
            <>
                {hasDocuments && (
                    <View style={{ margin: 0 10 }}>
                        <Text style={[styles.detailTitle, { color: colors.text, fontSize: 16 }]}>Documents</Text>
                        {answer.documents.map((doc, idx) => (
                            <View key={`\doc-\`} style={{ flex: 1, margin: 0 10 }}>
                                <Text style={{ color: colors.primaryDark, fontSize: 16 }}>{doc}</Text>
                                <Text style={{ color: colors.text, flex: 1, fontSize: 14 }}>{doc}</Text>
                            </View>
                        ))}
                    </View>
                )}
                {hasEligibility && (

```

```

        <View>
          <Text style={styles.detailTitle, { color: colors.text, fontSi
            {answer.eligibility.map((elig, idx) => (
              <View key={\elig-\} style={{ flexDirection: 'row', margin
                <Text style={{ color: colors.primaryDark, fontSize: 16
                <Text style={{ color: colors.text, flex: 1, fontSize:
              </View>
            ))}
          </View>
        )}
      </>
    )}
  </View>

  { /* Next Steps & Where to Go */ }
  <View style={styles.detailCard, { backgroundColor: colors.surface, borderColor:
    <View style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 12 }}&
      <View style={styles.detailIcon, { backgroundColor: colors.primarySoft, ma
        <Ionicons name="navigate-outline" size={19} color={colors.primaryDark} /&
      </View>
      <Text style={styles.detailTitle, { color: colors.text }}&Recommended
    </View>

    ((!hasNextSteps && !hasWhereToGo) ? (
      <Text style={styles.detailText, { color: colors.textMuted }}&No addit
    ) : (
      <&
      {hasNextSteps && (
        <View style={{ marginBottom: hasWhereToGo ? 16 : 0 }}&
          {answer.next_steps.map((step, idx) => (
            <View key={\step-\} style={{ flexDirection: 'row', margin
              <Text style={{ color: colors.primaryDark, fontSize: 13
              <Text style={{ color: colors.text, flex: 1, fontSize:
            </View>
          ))}
        </View>
      )}
      {hasWhereToGo && (
        <View>
          <Text style={styles.detailTitle, { color: colors.text, fontSi
            {answer.where_to_go.map((loc, idx) => (
              <View key={\loc-\} style={{ flexDirection: 'row', marginB
                <Text style={{ color: colors.primaryDark, fontSize: 16
                <Text style={{ color: colors.text, flex: 1, fontSize:
              </View>
            ))}
          </View>
        )}
      </>
    )}
  </View>

  <View style={styles.sourcesCard, { backgroundColor: colors.surface, borderColor
    <View style={styles.sourcesHeader}&
      <Ionicons name="link-outline" size={19} color={colors.primaryDark} /&
      <Text style={styles.sourcesTitle, { color: colors.text }}&
      Sources & Confidence ({cleanText((data.confidence_score * 100).toFixed(0)
    </Text>
    </View>
    <View style={{ marginTop: 8 }}&
      {data.citations && data.citations.length > 0 ? (
        data.citations.map((c, i) => (

```

```

        <Pressable
            key={i}
            onPress={() => { if (c.source_url) Linking.openURL(c.source_url); }}
            style={({ pressed }) => [{ marginBottom: 6, opacity: pressed ? 0.7 : 1 }]}
        >
            <Text style={[styles.sourcesText, { color: colors.primary, textDecorat
                {cleanText(c.source_title)} ({cleanText(c.authority))}
            </Text>
        </Pressable>
    ))
    ) : (
        <Text style={[styles.sourcesText, { color: colors.textMuted }]}>
            No specific sources linked.
        </Text>
    )}
    </View>
</View>

<View style={styles.audioRow}>
    <AudioButton icon="play" label={speechState === 'paused' ? 'Resume' : 'Play au
    <AudioButton icon="pause" label="Pause" onPress={handlePause} />
    <AudioButton icon="stop" label="Stop" onPress={handleStop} />
</View>

<View style={[styles.feedbackCard, { backgroundColor: colors.surface, borderColo
    {feedbackState === 'initial' && (
        <Text style={[styles.feedbackTitle, { color: colors.text }]}>Was this g
        <View style={styles.feedbackButtons}>
            <FeedbackButton
                icon="thumbs-up-outline"
                label="Helpful"
                onPress={() => {
                    setFeedbackState('helpful-submitted');
                    submitFeedback('helpful');
                }}
                color={colors.primaryDark}
                bg={colors.primarySoft}
            />
            <FeedbackButton
                icon="thumbs-down-outline"
                label="Not helpful"
                onPress={() => setFeedbackState('needs-improvement')}
                color={colors.textMuted}
                bg={colors.surfaceMuted}
            />
        </View>
    </>
    {feedbackState === 'needs-improvement' && (
        <Text style={[styles.feedbackTitle, { color: colors.text }]}>What could
        <TextInput
            style={[
                styles.feedbackInput,
                { backgroundColor: colors.background, color: colors.text, borderColor: co
            ]}
            placeholder="Tell us what was missing..."
            placeholderTextColor={colors.textMuted}
            value={feedbackComment}
            onChangeText={setFeedbackComment}
            multiline

```

```
var __BUNDLE_START_TIME__=globalThis.nativePerformanceNow?nativePerformanceNow():Date.now
!(function(e){'use strict';e[`${__METRO_GLOBAL_PREFIX__}_d`]|(e.__r=i,e[`${__METRO_GLOB
!(function(e){e.$$require_external="undefined"!=typeof require?e=>require(e):()=>nu
'undefined'!=typeof globalThis?globalThis:'undefined'!=typeof global?global:'undefined'!=
!(function(n){const e=(function(){function n(n,e){return n}function e(n){var e={};return
!(function(r){let l=0,n=!0===r.RN$useAlwaysAvailableJSErrorHandling?r.RN$handleException:
__d(function(g,r,i,a,m,e,d){'use strict';m.exports=r(d[0])},2,[3]);
__d(function(g,r,i,a,m,e,d){
/**
 * @license React
 * react-jsx-runtime.production.js
 *
 * Copyright (c) Meta Platforms, Inc. and affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */
"use strict";var n=Symbol.for("react.transitional.element"),t=Symbol.for("react.fragment")
```

```

__d(function(g,r,i,a,m,e,d){"use strict";r(d[0]),r(d[1])},4,[5,6]);
__d(function(g,r,i,a,m,e,d){},5,[[]]);
__d(function(g,r,i,a,m,e,d){const _='undefined'==typeof window;globalThis.__webpack_chunk
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,e,d){"use strict";r(d[0]),r(d[1]),r(d[2]),r(d[3])},9,[10,11,12,13]
__d(function(g,r,i,a,m,e,d){},10,[[]]);
__d(function(g,r,i,a,m,e,d){},11,[[]]);
__d(function(g,r,i,a,m,e,d){},12,[[]]);
__d(function(g,r,i,a,m,e,d){},13,[[]]);
__d(function(g,r,i,a,m,e,d){},14,[[]]);
__d(function(g,r,i,a,m,e,d){"use strict";(0,r(d[0]).installExpoGlobalPolyfill())},15,[16]
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";function _interopDefault(e){return e&&&e
__d(function(g,r,_i,_a,m,_e,_d){'use strict';function e(e,t,n,o){switch(e){case 0:return
__d(function(g,r,_i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!
__d(function(g,r,_i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0}
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";function o(o){return'object'==typeof globalThis.
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){},36,[[]]);
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){'use strict';m.exports=r(d[0])},39,[40]);
__d(function(g,r,_i,a,m,e,d){
/**
 * @license React
 * react.production.js
 *
 * Copyright (c) Meta Platforms, Inc. and affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */
"use strict";var t=Symbol.for("react.transitional.element"),n=Symbol.for("react.portal"),
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){'use strict';function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){'use strict';m.exports=function(n,o){for(var t=arguments.leng
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,_i,a,m,_e,_d){"use strict";Object.defineProperty(_e,'__esModule',{value:!
__d(function(_g,_r,i,_a,_m,_e,_d){!(function(){function e(e,t){document.addEventListener?
__d(function(g,r,i,a,m,e,d){"use strict";var t;Object.defineProperty(e,'__esModule',{valu

```

```
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";var e,t=r(d[0]),s=r(d[1]),u=r(d[2]),o=(e=u)&
_d(function(g,r,i,a,m,_e,_d){\"use strict\";function t(t){if(t&&t.__esModule)return
_d(function(g,r,i,a,m,e,d){const s=[];m.exports={registerAsset:t=>s.push(t),getAssetB
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";function t(t){const{pathname:s,searchParams:o}=n
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";var t=this&&&this.__createBinding||(Obje
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";var t,n=this&&&this.__createBinding||(O
_d(function(g,r,i,a,m,_e,d){\"use strict\";var t,n=this&&&this.__createBinding||(O
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,_e,d){\"use strict\";function t(t){return t&&&t.__esModule?t
_d(function(g,r,i,a,m,_e,d){\"use strict\";function n(n){return n&&&n.__esModule?n
_d(function(g,r,i,a,m,_e,d){\"use strict\";function o(o){return o&&&o.__esModule?o
_d(function(g,r,i,a,m,_e,d){\"use strict\";function o(o){return o&&&o.__esModule?o:
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(_g,_r,_i,_a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(_g,_r,_i,a,m,e,d){'use strict';function r(r,n,t){return t<t;0&&&(t+=1
_d(function(g,r,i,a,m,e,d){\"use strict\";function t(t,c){for(var o,n=t.length,u=c^n,h=0;
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
_d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";function n(n,t){-1===n.indexOf(t)&&&n.pu
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e
_d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
_d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
```

```
__d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:
__d(function(g,r,i,a,m,e,d){m.exports=r(d[0])},143,[144]);
__d(function(g,r,i,a,m,e,d){'use strict';Object.defineProperty(e,\"__esModule\",{value:!0})
__d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:!0}
__d(function(g,r,i,a,m,e,d){var t=r(d[0]),n=r(d[1]),o=r(d[2]);function s(n){return this i
__d(function(g,r,i,a,m,e,d){var o=\"(\".charCodeAt(0),n=)\".charCodeAt(0),c=)\".charCodeAt
__d(function(g,r,i,a,m,e,d){m.exports=function n(o,t,f){var i,s,c,u;for(i=0,s=o.length;i
__d(function(g,r,i,a,m,e,d){function n(n,t){var u,f,i=n.type,s=n.value;return t&&&
__d(function(g,r,i,a,m,e,d){var t=\"-\".charCodeAt(0),o=\"+\".charCodeAt(0),c=\".\".charCodeAt(
__d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:!0}
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){'use strict';!(function _(){if('undefined'!==typeof __REACT_DE
__d(function(e,n,t,r,l,a,o){
/**
 * @license React
 * react-dom-client.production.js
 *
 * Copyright (c) Meta Platforms, Inc. and affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */
\"use strict\";var u=n(o[0]),i=n(o[1]),s=n(o[2]);function c(e){var n=\"https://react.dev/err
__d(function(g,r,i,a,m,e,d){'use strict';m.exports=r(d[0])},161,[162]);
__d(function(g,r,i,a,m,e,d){
/**
 * @license React
 * scheduler.production.js
 *
 * Copyright (c) Meta Platforms, Inc. and affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */
\"use strict\";function n(n,t){var a=n.length;n.push(t);e:for(;0<a;){var o=a-1>>>>&&&g
__d(function(g,r,i,a,m,e,d){'use strict';!(function _(){if('undefined'!==typeof __REACT_DE
__d(function(g,r,i,a,m,e,d){
/**
 * @license React
 * react-dom.production.js
 *
 * Copyright (c) Meta Platforms, Inc. and affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */
\"use strict\";var t=r(d[0]);function i(t){var i=\"https://react.dev/errors/\"+t;if(1<t;argu
__d(function(g,r,i,a,m,e,d){\"use strict\";function t(t){return t.unmount(),!0}Object.define
__d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:!0}
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:!0}
__d(function(g,r,i,a,m,e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,e,d){\"use strict\";Object.defineProperty(_e,'__esModule',{value:!0}
__d(function(g,r,i,a,m,_e,d){\"use strict\";function e(e){return e&&&e.__esModule?e:
```


[illegible]


```

__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";const t=r(d[0]),n=r(d[1]),a=r(d[2]),o=r(d[3]),s
__d(function(g,r,i,a,m,e,d){"use strict";m.exports=t=&gt;encodeURIComponent(t).replace(/
__d(function(g,r,i,a,m,e,d){"use strict";var t=new RegExp("(%[a-f0-9]{2})|([^\%]+?)","gi"
__d(function(g,r,i,a,m,e,d){"use strict";m.exports=(t,n)=&gt;{if('string'!==typeof t||'str
__d(function(g,r,i,a,m,e,d){"use strict";m.exports=function(t,n){for(var i={},s=Object.k
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var n,t=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";m.exports=r(d[0]),400,[401]);
__d(function(g,r,i,a,m,e,d){
/**
 * @license React
 * react-is.production.js
 *
 * Copyright (c) Meta Platforms, Inc. and affiliates.
 *
 * This source code is licensed under the MIT license found in the
 * LICENSE file in the root directory of this source tree.
 */
"use strict";var t=Symbol.for("react.transitional.element"),o=Symbol.for("react.portal"),
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var e,t=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";m.exports=function t(n,o){if(n===o)return!0;if
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0
__d(function(g,r,i,a,m,e,d){"use strict";var t,n=this&&&this.__createBinding||(0

```


[illegible]

[illegible]

[illegible]


```
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo-google-fonts/manrope/30
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo-google-fonts/manrope/40
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo-google-fonts/manrope/50
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo-google-fonts/manrope/60
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo-google-fonts/manrope/70
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo-google-fonts/manrope/80
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports="use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function t(t){return t&&&t.__esModule?t
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";Object.defineProperty(_e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,e,d){m.exports="use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports={pick:(t,...c)=>c.flat().filter(c=>Object.pro
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,r,o,i,t,a,l){t.exports={account-book:62454,aim:62455,alert:62456,alibaba
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,r,o,i,t,l,a){t.exports={500px:61696,500px-with-circle:61697,add-to-li
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,r,c,o,s,n,a){s.exports={archive:61696,arrow-down:61697,arrow-left:6169
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,r,o,i,a,l,t){a.exports={activity:61696,airplay:61697,alert-circle:61698,
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(o,e,t,c,a,n,s){a.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,a,r,o,t,s,i){t.exports={acrobat-reader:60095,applemusic:60096,atlassian:
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(e,o,t,n,s,c,d){s.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,r,o,a,t,s,l){t.exports={glass:61440,music:61441,search:61442,-envelope-o:
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";Object.defineProperty(_e,'__esModule',{value:!
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function t(t){return t&&&t.__esModule?t
__d(function(e,a,r,o,s,t,i){s.exports={500px:62062,accessible-icon:62312,accusoft:623
__d(function(e,a,r,o,s,i,t){s.exports={brands:[500px,accessible-icon,accusoft,acqu
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,d,t,n,s,c){n.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";Object.defineProperty(_e,'__esModule',{value:!
__d(function(e,a,r,o,t,s,l){t.exports={0:48,1:49,2:50,3:51,4:52,5:53,6:54,7:55,8:56,9:57,
__d(function(e,r,a,o,i,s,l){i.exports={brands:[42-group,500px,accessible-icon,accu
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,t,c,d,n,s){d.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,t,n,s,c,d){s.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(o,e,a,l,i,s,r){i.exports={address-book:61696,alert:61697,align-center:61
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(o,e,n,t,d,s,c){d.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,r,o,a,l,i,t){l.exports={accessibility:61696,accessibility-outline:61697,
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(e,o,t,n,s,a,c){s.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,t,i,l,a,r){l.exports={ab-testing:983497,abacus:988896,abjad-arabic:9
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(e,c,o,t,n,s,i){n.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,a,t,r,i,n){r.exports={123:60301,360:58743,10k:59729,10mp:59730,11mp
__d(function(g,r,i,a,m,_e,d){m.exports="use strict";function e(e){return e&&&e.__esModule?e
__d(function(g,r,i,a,m,e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
```

```
__d(function(e,o,l,r,i,s,t){i.exports={accessibility:61696,"accessibility-inset":61697,"a
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,l,a,r,c,i){r.exports={user:57349,people:57345,"user-female":57344,"user-
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){m.exports="/assets/node_modules/@expo/vector-icons/build/vend
__d(function(e,o,t,a,i,s,r){i.exports={acrobat:61696,amazon:61697,android:61698,angellis
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){if(e&&&e.__esModule)ret
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";var n;Object.defineProperty(e,'__esModule',{valu
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";var t,n;Object.defineProperty(e,'__esModule',{va
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(e,s,t,a,i,r,g){i.exports={uri:"/assets/assets/images/akshaya-logo.079542990a
__d(function(g,r,i,a,m,_e,_d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";var t,n;Object.defineProperty(e,'__esModule',{va
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";var A,t,n,E,M;Object.defineProperty(e,'__esModul
__d(function(g,r,i,a,m,_e2,d){"use strict";Object.defineProperty(_e2,'__esModule',{value:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";function t(t){return t&&&t.__esModule?t:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,_d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";var t,n;Object.defineProperty(e,'__esModule',{va
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";var t,n=Object.create,s=Object.defineProperty,i
__d(function(g,r,i,a,m,_e,d){var t='undefined'!=typeof Element,n='function'==typeof Map,
__d(function(g,r,i,a,m,_e,d){m.exports=function(t,n,o,f){var c=o?o.call(f,t,n):void 0;if(v
__d(function(g,r,i,a,m,_e,d){"use strict";r(d[0])},949,[950]);
__d(function(g,r,i,a,m,_e,d){"use strict";r(d[0]);var n=r(d[1]);(0,r(d[2]).renderRootCompo
__d(function(g,r,i,a,m,_e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0})
__d(function(g,r,i,a,m,_e,d){"use strict";m.exports=r(d[0])},952,[953]);
__d(function(g,r,i,a,m,_e,d){
/**
```

[illegible]

```

__d(function(g,r,i,a,m,_e,_d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0});
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,'__esModule',{value:!0});
__d(function(g,r,i,a,m,_e,_d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0});
__d(function(g,r,i,a,m,_e,_d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0});
__d(function(g,r,i,a,m,_e,_d){"use strict";Object.defineProperty(_e,'__esModule',{value:!0});
__d(function(g,r,i,a,m,e,d){},1013,[]);
__d(function(g,r,i,a,m,_e,_d){"use strict";function e(e){return e&&&e.__esModule?e:
__d(function(g,r,i,a,_m,e,d){"use strict";var t=this&&&this.__createBinding||(Object
__d(function(g,r,i,a,m,e,d){"use strict";Object.defineProperty(e,"__esModule",{value:!0});
__d(function(g,r,_i,a,_m,_e,d){"use strict";var e,t=this&&&this.__createBinding||(
__r(478);
__r(4);
__r(949);

```

File: scripts\reset-project.js

```

#!/usr/bin/env node

/**
 * This script is used to reset the project to a blank state.
 * It deletes or moves the /src and /scripts directories to /example based on user input.
 * You can remove the `reset-project` script from package.json and safely delete this file.
 */

const fs = require("fs");
const path = require("path");
const readline = require("readline");

const root = process.cwd();
const oldDirs = ["src", "scripts"];
const exampleDir = "example";
const newAppDir = "src/app";
const exampleDirPath = path.join(root, exampleDir);

const indexContent = `import { Text, View, StyleSheet } from "react-native";

export default function Index() {
  return (
    <View style={styles.container}>
      <Text>Edit src/app/index.tsx to edit this screen.</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
  },
});
`;

const layoutContent = `import { Stack } from "expo-router";

export default function RootLayout() {
  return <Stack />;
}
`;

```

```

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

const moveDirectories = async (userInput) => {
  try {
    if (userInput === "y") {
      // Create the app-example directory
      await fs.promises.mkdir(exampleDirPath, { recursive: true });
      console.log(`📁 /${exampleDir} directory created.`);
    }

    // Move old directories to new app-example directory or delete them
    for (const dir of oldDirs) {
      const oldDirPath = path.join(root, dir);
      if (fs.existsSync(oldDirPath)) {
        if (userInput === "y") {
          const newDirPath = path.join(root, exampleDir, dir);
          await fs.promises.rename(oldDirPath, newDirPath);
          console.log(`➡️📁 /${dir} moved to /${exampleDir}/${dir}.`);
        } else {
          await fs.promises.rm(oldDirPath, { recursive: true, force: true });
          console.log(`📁 /${dir} deleted.`);
        }
      } else {
        console.log(`➡️📁 /${dir} does not exist, skipping.`);
      }
    }

    // Create new /src/app directory
    const newAppDirPath = path.join(root, newAppDir);
    await fs.promises.mkdir(newAppDirPath, { recursive: true });
    console.log(`\n📁 New /src/app directory created.`);

    // Create index.tsx
    const indexPath = path.join(newAppDirPath, "index.tsx");
    await fs.promises.writeFile(indexPath, indexContent);
    console.log(`📄 src/app/index.tsx created.`);

    // Create _layout.tsx
    const layoutPath = path.join(newAppDirPath, "_layout.tsx");
    await fs.promises.writeFile(layoutPath, layoutContent);
    console.log(`📄 src/app/_layout.tsx created.`);

    console.log(`\n📁 Project reset complete. Next steps:`);
    console.log(`
    1. Run `npx expo start` to start a development server.
    2. Edit src/app/index.tsx
    3. Run `npx expo start` to start a development server.
    4. Delete the /${exampleDir} directory when you're done referencing it.`
    );
    if (userInput === "y") {
      console.log(`?`);
    }
  } catch (error) {
    console.error(`🔴 Error during script execution: ${error.message}`);
  }
};

rl.question(
  "Do you want to move existing files to /example instead of deleting them? (Y/n): ",
  (answer) => {
    const userInput = answer.trim().toLowerCase() || "y";
  }
);

```

```

    if (userInput === "y" || userInput === "n") {
      moveDirectories(userInput).finally(() => rl.close());
    } else {
      console.log("■ Invalid input. Please enter 'Y' or 'N'.");
      rl.close();
    }
  }
});

```

File: src\global.css

```

:root {
  --font-display:
    Spline Sans, Inter, ui-sans-serif, system-ui, sans-serif, Apple Color Emoji, Segoe UI
    Segoe UI Symbol, Noto Color Emoji;
  --font-mono:
    ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, Liberation Mono, Courier New,
  --font-rounded: 'SF Pro Rounded', 'Hiragino Maru Gothic ProN', Meiryo, 'MS PGothic', sa
  --font-serif: Georgia, 'Times New Roman', serif;
}

```

File: src\app\about.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
  Pressable,
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { useTheme } from "../contexts/ThemeContext";

const principles = [
  {
    icon: "search-outline" as const,
    title: "Official-source retrieval",
    text: "The final backend will retrieve relevant evidence from curated official documents",
  },
  {
    icon: "git-compare-outline" as const,
    title: "Controlled agentic workflow",
    text: "The system can identify intent, evaluate retrieval relevance, rewrite weak queries",
  },
  {
    icon: "document-text-outline" as const,
    title: "Structured guidance",
    text: "Responses will organize service identification, possible eligibility, document relevance",
  },
  {
    icon: "volume-medium-outline" as const,
    title: "English Listen feature",
    text: "The completed version will provide English text-to-speech for verified final results",
  },
];

```

```

];

export default function AboutScreen() {
  const { colors } = useTheme();

  return (
    <SafeAreaView
      style={[
        styles.safeArea,
        {
          backgroundColor: colors.background,
        },
      ]}
    >
      <ScrollView
        showsVerticalScrollIndicator={false}
        contentContainerStyle={styles.content}
      >
        <View style={styles.headerRow}>
          <Pressable
            onPress={() => router.canGoBack() ? router.back() : router.replace('/') }
            accessibilityRole="button"
            accessibilityLabel="Go back"
            style={styles.backButton}
          >
            <Ionicons name="arrow-back" size={22} color={colors.text} />
          </Pressable>

          <View style={styles.headerTextArea}>
            <Text
              style={[
                styles.eyebrow,
                {
                  color: colors.primaryDark,
                },
              ]}
            >
              ABOUT THE PROJECT
            </Text>

            <Text
              style={[
                styles.title,
                {
                  color: colors.text,
                },
              ]}
            >
              Akshaya Advisory
            </Text>

            <Text
              style={[
                styles.subtitle,
                {
                  color: colors.textMuted,
                },
              ]}
            >
              Agentic RAG-Based Citizen Service Advisory Application for
              Akshaya Centres
            </Text>
          </View>
        </View>
      </ScrollView>
    </SafeAreaView>
  );
}

```

```

    </View>
</View>

<View
  style=[
    styles.introCard,
    {
      backgroundColor: colors.primarySoft,
    },
  ]
>
  <View
    style=[
      styles.introIcon,
      {
        backgroundColor: colors.primary,
      },
    ]
  >
    <Ionicons
      name="sparkles"
      size={26}
      color="#FFFFFF"
    />
  </View>

  <Text
    style=[
      styles.introTitle,
      {
        color: colors.primaryDark,
      },
    ]
  >
    Clear preparation before visiting a service centre
  </Text>

  <Text
    style=[
      styles.introText,
      {
        color: colors.primaryDark,
      },
    ]
  >
    The project helps citizens understand possible services, document
    requirements, eligibility information, identity-mismatch guidance,
    and recommended next steps before visiting an Akshaya centre.
  </Text>
</View>

  <Text
    style=[
      styles.sectionTitle,
      {
        color: colors.text,
      },
    ]
  >
    Project principles
  </Text>

```

```

<View style={styles.principleList}>
  {principles.map((principle) => (
    <View
      key={principle.title}
      style={[
        styles.principleCard,
        {
          backgroundColor: colors.surface,
          borderColor: colors.border,
        },
      ]}
    >
      <View
        style={[
          styles.principleIcon,
          {
            backgroundColor: colors.primarySoft,
          },
        ]}
      >
        <Ionicons
          name={principle.icon}
          size={20}
          color={colors.primaryDark}
        />
      </View>

      <View style={styles.principleTextArea}>
        <Text
          style={[
            styles.principleTitle,
            {
              color: colors.text,
            },
          ]}
        >
          {principle.title}
        </Text>

        <Text
          style={[
            styles.principleText,
            {
              color: colors.textMuted,
            },
          ]}
        >
          {principle.text}
        </Text>
      </View>
    </View>
  )})
</View>

<View
  style={[
    styles.warningCard,
    {
      backgroundColor: colors.warningBackground,
    },
  ]}
  >

```

```

<Ionicons
  name="alert-circle-outline"
  size={22}
  color={colors.warning}
/>

<View style={styles.warningTextArea}>
  <Text
    style={[
      styles.warningTitle,
      {
        color: colors.warning,
      },
    ]}
  >
    Important advisory limitation
  </Text>

  <Text
    style={[
      styles.warningText,
      {
        color: colors.warning,
      },
    ]}
  >
    The application does not approve or submit government
    applications, perform biometric verification, access private
    Aadhaar data, confirm a citizen's official identity, verify
    document authenticity, or replace government portals and Akshaya
    operators.
  </Text>
</View>

<View
  style={[
    styles.futureCard,
    {
      backgroundColor: colors.surface,
      borderColor: colors.border,
    },
  ]}
>
  <Ionicons
    name="language-outline"
    size={22}
    color={colors.primaryDark}
  />

  <View style={styles.futureTextArea}>
    <Text
      style={[
        styles.futureTitle,
        {
          color: colors.text,
        },
      ]}
    >
      Malayalam as a future enhancement
    </Text>

```

```

        <Text
          style={
            styles.futureText,
            {
              color: colors.textMuted,
            },
          }
        />
        Malayalam support will be added only after testing multilingual
        models, retrieval quality, government terminology, citation
        preservation, translation quality, and text-to-speech
        pronunciation. The English MVP remains the priority.
      />
    />
  />

  <Text
    style={
      styles.footerText,
      {
        color: colors.textMuted,
      },
    }
  />
  Prototype status: the mobile frontend is complete in structure. The
  next stage connects FastAPI, curated official documents, RAG
  retrieval, citation verification, confidence scoring, and English
  text-to-speech.
/>
/ScrollView>
/SafeAreaView>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },

  content: {
    alignSelf: "center",
    maxWidth: 760,
    padding: 17,
    paddingBottom: 43,
    width: "100%",
  },

  headerRow: {
    alignItems: "flex-start",
    flexDirection: "row",
    gap: 9,
  },

  backButton: {
    alignItems: "center",
    height: 40,
    justifyContent: "center",
    width: 40,
  },

  headerTextArea: {
    flex: 1,

```

```
},

eyebrow: {
  fontSize: 10,
  fontWeight: "800",
  letterSpacing: 1,
  marginBottom: 7,
},

title: {
  fontSize: 26,
  fontWeight: "800",
  letterSpacing: -0.4,
},

subtitle: {
  fontSize: 12,
  lineHeight: 18,
  marginTop: 7,
},

introCard: {
  alignItems: "center",
  borderRadius: 19,
  marginTop: 22,
  padding: 20,
},

introIcon: {
  alignItems: "center",
  borderRadius: 16,
  height: 56,
  justifyContent: "center",
  width: 56,
},

introTitle: {
  fontSize: 18,
  fontWeight: "800",
  lineHeight: 24,
  marginTop: 14,
  textAlign: "center",
},

introText: {
  fontSize: 12,
  lineHeight: 19,
  marginTop: 9,
  maxWidth: 520,
  textAlign: "center",
},

sectionTitle: {
  fontSize: 18,
  fontWeight: "800",
  marginTop: 26,
},

principleList: {
  gap: 9,
  marginTop: 14,
},
```

```
principleCard: {
  alignItems: "flex-start",
  borderRadius: 16,
  borderWidth: 1,
  flexDirection: "row",
  gap: 10,
  padding: 13,
},

principleIcon: {
  alignItems: "center",
  borderRadius: 11,
  height: 39,
  justifyContent: "center",
  width: 39,
},

principleTextArea: {
  flex: 1,
},

principleTitle: {
  fontSize: 13,
  fontWeight: "800",
},

principleText: {
  fontSize: 11,
  lineHeight: 17,
  marginTop: 4,
},

warningCard: {
  alignItems: "flex-start",
  borderRadius: 17,
  flexDirection: "row",
  gap: 10,
  marginTop: 17,
  padding: 15,
},

warningTextArea: {
  flex: 1,
},

warningTitle: {
  fontSize: 13,
  fontWeight: "800",
},

warningText: {
  fontSize: 11,
  fontWeight: "600",
  lineHeight: 18,
  marginTop: 5,
},

futureCard: {
  alignItems: "flex-start",
  borderRadius: 17,
  borderWidth: 1,
  flexDirection: "row",
```

```

        gap: 10,
        marginTop: 11,
        padding: 15,
    },

    futureTextArea: {
        flex: 1,
    },

    futureTitle: {
        fontSize: 13,
        fontWeight: "800",
    },

    futureText: {
        fontSize: 11,
        lineHeight: 18,
        marginTop: 4,
    },

    footerText: {
        fontSize: 10,
        lineHeight: 16,
        marginTop: 24,
        textAlign: "center",
    },
});
});

```

File: src\appl\account.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
    Pressable,
    SafeAreaView,
    ScrollView,
    StyleSheet,
    Text,
    View,
} from "react-native";

import { useTheme } from "../contexts/ThemeContext";

export default function AccountScreen() {
    const { colors } = useTheme();

    return (
        <SafeAreaView
            style={[
                styles.safeArea,
                {
                    backgroundColor: colors.background,
                },
            ]}
        >
            <ScrollView
                contentContainerStyle={styles.content}
                showsVerticalScrollIndicator={false}
            >
                <View style={styles.headerRow}>

```

```

<Pressable
  onPress={() => router.canGoBack() ? router.back() : router.replace('/') }
  accessibilityRole="button"
  accessibilityLabel="Go back"
  style={styles.backButton}
/>
<Ionicons name="arrow-back" size={22} color={colors.text} />
</Pressable>

<View>
  <Text
    style={[
      styles.headerTitle,
      {
        color: colors.text,
      },
    ]}
  />
  Account and privacy
</Text>

  <Text
    style={[
      styles.headerSubtitle,
      {
        color: colors.textMuted,
      },
    ]}
  />
  Your advisory preferences and data boundaries
</Text>
</View>
</View>

<View
  style={[
    styles.profileCard,
    {
      backgroundColor: colors.surface,
      borderColor: colors.border,
    },
  ]}
/>
  <View
    style={[
      styles.avatar,
      {
        backgroundColor: colors.primary,
      },
    ]}
  />
  <Ionicons
    name="person-outline"
    size={28}
    color="#FFFFFF"
  />
</View>

  <View style={styles.profileText}>
    <Text
      style={[
        styles.profileName,

```

```

        {
            color: colors.text,
        },
    ]}
    &gt;
    Guest user
    </Text>

    <Text
        style={[
            styles.profileSubtitle,
            {
                color: colors.textMuted,
            },
        ]}
    &gt;
        No account or personal identity is required for the MVP.
    </Text>
    </View>
    </View>

    <InfoCard
        colors={colors}
        title="Privacy-first design"
        icon="shield-checkmark-outline"
        accent
        rows={[
            {
                icon: "lock-closed-outline",
                title: "Do not share sensitive data",
                text: "Do not enter full Aadhaar numbers, OTPs, passwords, bank information",
            },
            {
                icon: "finger-print-outline",
                title: "No identity verification",
                text: "The app does not verify your identity, document authenticity, or eli",
            },
            {
                icon: "document-text-outline",
                title: "Advisory query logging",
                text: "Later backend logs will be anonymized where possible for testing and",
            },
        ]}
    /&gt;

    <InfoCard
        colors={colors}
        title="Accessibility and language"
        icon="accessibility-outline"
        rows={[
            {
                icon: "language-outline",
                title: "English-first MVP",
                text: "The initial version is designed for typed English queries and verifi",
            },
            {
                icon: "mic-outline",
                title: "Voice typing",
                text: "You can use the voice typing feature provided by your mobile keyboar",
            },
            {
                icon: "volume-medium-outline",

```

```

        title: "Listen feature",
        text: "English text-to-speech will read verified final responses after back
    },
    ]]
/>

<View
  style=[
    styles.disclaimer,
    {
      backgroundColor: colors.warningBackground,
    },
  ]
/>
<Ionicons
  name="alert-circle-outline"
  size={21}
  color={colors.warning}
/>

<Text
  style=[
    styles.disclaimerText,
    {
      color: colors.warning,
    },
  ]
/>
  This is an advisory application. Always confirm important service
  requirements with an Akshaya operator or the responsible official
  department.
</Text>
</View>
</ScrollView>
</SafeAreaView>
);
}

function InfoCard({
  colors,
  title,
  icon,
  rows,
  accent = false,
}: {
  colors: any;
  title: string;
  icon: keyof typeof Ionicons.glyphMap;
  accent?: boolean;
  rows: {
    icon: keyof typeof Ionicons.glyphMap;
    title: string;
    text: string;
  }[];
}) {
  return (
    <View
      style=[
        styles.infoCard,
        {

```

```

        backgroundColor: colors.surface,
        borderColor: colors.border,
    },
  ]}
  &gt;
  <View style={styles.infoHeader}&gt;
    <View
      style={[
        styles.infoHeaderIcon,
        {
          backgroundColor: accent
            ? colors.primarySoft
            : colors.surfaceMuted,
        },
      ]}
    &gt;
      <Ionicons
        name={icon}
        size={20}
        color={colors.primaryDark}
      /&gt;
    </View&gt;

    <Text
      style={[
        styles.infoHeaderTitle,
        {
          color: colors.text,
        },
      ]}
    &gt;
      {title}
    </Text&gt;
  </View&gt;

  {rows.map((row) => (
    <View key={row.title} style={styles.infoRow}&gt;
      <Ionicons
        name={row.icon}
        size={17}
        color={colors.primaryDark}
      /&gt;

      <View style={styles.infoRowTextArea}&gt;
        <Text
          style={[
            styles.infoRowTitle,
            {
              color: colors.text,
            },
          ]}
        &gt;
          {row.title}
        </Text&gt;

        <Text
          style={[
            styles.infoRowText,
            {
              color: colors.textMuted,
            },
          ]}
        &gt;

```

```

        &gt;
        {row.text}
      </Text>
    </View>
  </View>
)}
</View>
);
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },

  content: {
    alignSelf: "center",
    maxWidth: 760,
    padding: 17,
    paddingBottom: 42,
    width: "100%",
  },

  headerRow: {
    alignItems: "flex-start",
    flexDirection: "row",
    gap: 9,
  },

  backButton: {
    alignItems: "center",
    height: 40,
    justifyContent: "center",
    width: 40,
  },

  headerTitle: {
    fontSize: 20,
    fontWeight: "800",
    marginTop: 2,
  },

  headerSubtitle: {
    fontSize: 11,
    lineHeight: 16,
    marginTop: 4,
  },

  profileCard: {
    alignItems: "center",
    borderRadius: 18,
    borderWidth: 1,
    flexDirection: "row",
    gap: 13,
    marginTop: 22,
    padding: 16,
  },

  avatar: {
    alignItems: "center",
    borderRadius: 18,
    height: 56,

```

```
        justifyContent: "center",
        width: 56,
    },

    profileText: {
        flex: 1,
    },

    profileName: {
        fontSize: 17,
        fontWeight: "800",
    },

    profileSubtitle: {
        fontSize: 12,
        lineHeight: 18,
        marginTop: 4,
    },

    infoCard: {
        borderRadius: 18,
        borderWidth: 1,
        marginTop: 12,
        padding: 16,
    },

    infoHeader: {
        alignItems: "center",
        flexDirection: "row",
        gap: 9,
        marginBottom: 15,
    },

    infoHeaderIcon: {
        alignItems: "center",
        borderRadius: 10,
        height: 38,
        justifyContent: "center",
        width: 38,
    },

    infoHeaderTitle: {
        fontSize: 15,
        fontWeight: "800",
    },

    infoRow: {
        alignItems: "flex-start",
        flexDirection: "row",
        gap: 10,
        marginBottom: 14,
    },

    infoRowTextArea: {
        flex: 1,
    },

    infoRowTitle: {
        fontSize: 12,
        fontWeight: "800",
    },
}
```

```

infoRowText: {
  fontSize: 11,
  lineHeight: 17,
  marginTop: 3,
},

disclaimer: {
  alignItems: "flex-start",
  borderRadius: 18,
  flexDirection: "row",
  gap: 9,
  marginTop: 12,
  padding: 15,
},

disclaimerText: {
  flex: 1,
  fontSize: 11,
  fontWeight: "600",
  lineHeight: 17,
},
});

```

File: srclapp\chat.tsx

```

import { Redirect, useLocalSearchParams } from "expo-router";

export default function ChatRedirect() {
  const params = useLocalSearchParams<{
    query?: string | string[];
  }>();

  return (
    <Redirect
      href={{
        pathname: "/",
        params,
      }}
    />
  );
}

```

File: srclapp\documents.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
  Pressable,
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

```

```

import ServiceCategoryCard from "../components/ServiceCategoryCard";
import { SERVICE_CATEGORIES } from "../constants/services";
import { useTheme } from "../contexts/ThemeContext";

const workflowSteps = [
  {
    number: "01",
    icon: "camera-outline" as const,
    title: "Capture clear images",
    text: "When uploading a document for analysis, ensure good lighting and clear text. B
  },
  {
    number: "02",
    icon: "eye-off-outline" as const,
    title: "Hide sensitive numbers",
    text: "Physically cover full Aadhaar numbers, passwords, and CVVs before capturing th
  },
  {
    number: "03",
    icon: "folder-open-outline" as const,
    title: "Organize physical copies",
    text: "Before visiting an Akshaya center, gather all original documents and keep self
  },
];

export default function DocumentsScreen() {
  const { colors } = useTheme();

  return (
    <SafeAreaView
      style={[
        styles.safeArea,
        {
          backgroundColor: colors.background,
        },
      ]}
    >
      <ScrollView
        contentContainerStyle={styles.content}
        showsVerticalScrollIndicator={false}
      >
        <View style={styles.headerRow}>
          <Pressable
            onPress={() => router.canGoBack() ? router.back() : router.replace('/')}>
            accessibilityRole="button"
            accessibilityLabel="Go back"
            style={styles.backButton}
          >
            <Ionicons
              name="arrow-back"
              size={22}
              color={colors.text}
            />
          </Pressable>

          <View style={styles.headerText}>
            <Text
              style={[
                styles.eyebrow,
                {
                  color: colors.primary,
                },
              ]}
            >

```

```

    ]]
    &gt;
    DOCUMENT PREPARATION
    </Text>

    <Text
      style=[
        styles.title,
        {
          color: colors.text,
        },
      ]
    &gt;
    How to prepare your documents
    </Text>

    <Text
      style=[
        styles.subtitle,
        {
          color: colors.textMuted,
        },
      ]
    &gt;
    Guidelines for submitting document images and gathering physical copies for
    </Text>
    </View>
  </View>

  <View
    style=[
      styles.statusCard,
      {
        backgroundColor: colors.primarySoft,
      },
    ]
  &gt;
    <View
      style=[
        styles.statusIcon,
        {
          backgroundColor: colors.primary,
        },
      ]
    &gt;
      <Ionicons
        name="language-outline"
        size={21}
        color="#FFFFFF"
      /&gt;
    </View>

    <View style={styles.statusContent}&gt;
      <Text
        style=[
          styles.statusTitle,
          {
            color: colors.text,
          },
        ]
      &gt;
        English-first advisory mode

```

```

    </Text>

    <Text
      style={[
        styles.statusText,
        {
          color: colors.textMuted,
        },
      ]}
    >
      The first version is designed and tested in English. Malayalam
      support will be considered only after quality evaluation.
    </Text>
  </View>
</View>

<Text
  style={[
    styles.sectionTitle,
    {
      color: colors.text,
    },
  ]}
  >
  Supported service areas
</Text>

<Text
  style={[
    styles.sectionSubtitle,
    {
      color: colors.textMuted,
    },
  ]}
  >
  Choose the topic closest to your situation.
</Text>

<View style={styles.serviceGrid}>
  {SERVICE_CATEGORIES.map((service) => (
    <ServiceCategoryCard
      key={service.id}
      category={service.id}
      title={service.title}
      description={service.description}
      icon={service.icon}
    />
  ))}
</View>

<View style={styles.workflowHeader}>
  <Text
    style={[
      styles.sectionTitle,
      {
        color: colors.text,
      },
    ]}
    >
    How the advisory process works
  </Text>

```

```

<Text
  style=[
    styles.sectionSubtitle,
    {
      color: colors.textMuted,
    },
  ]
>
  The final system will use a controlled agentic RAG workflow.
</Text>
</View>

<View style={styles.workflowList}>
  {workflowSteps.map((step) => (
    <View
      key={step.number}
      style=[
        styles.workflowCard,
        {
          backgroundColor: colors.surface,
          borderColor: colors.border,
        },
      ]
    >
      <View
        style=[
          styles.stepNumber,
          {
            backgroundColor: colors.primarySoft,
          },
        ]
      >
        <Text
          style=[
            styles.stepNumberText,
            {
              color: colors.primary,
            },
          ]
        >
          {step.number}
        </Text>
      </View>

      <View
        style=[
          styles.workflowIcon,
          {
            backgroundColor: colors.surfaceMuted,
          },
        ]
      >
        <Icons
          name={step.icon}
          size={20}
          color={colors.primary}
        />
      </View>

      <View style={styles.workflowContent}>
        <Text
          style=[

```

```

        styles.workflowTitle,
        {
            color: colors.text,
        },
    ]}
    &gt;
    {step.title}
    &lt;/Text&gt;

    &lt;Text
    style=[
        styles.workflowText,
        {
            color: colors.textMuted,
        },
    ]}
    &gt;
    {step.text}
    &lt;/Text&gt;
    &lt;/View&gt;
    &lt;/View&gt;
    )})
    &lt;/View&gt;

    &lt;View
    style=[
        styles.capabilityCard,
        {
            backgroundColor: colors.surface,
            borderColor: colors.border,
        },
    ]}
    &gt;
    &lt;View style={styles.capabilityHeader}&gt;
    &lt;Ionicons
        name="checkmark-circle-outline"
        size={21}
        color={colors.accent}
    /&gt;

    &lt;Text
    style=[
        styles.capabilityTitle,
        {
            color: colors.text,
        },
    ]}
    &gt;
    Supported document types
    &lt;/Text&gt;
    &lt;/View&gt;

    &lt;CapabilityItem
    text="JPG, PNG, and WEBP images"
    colors={colors}
    /&gt;

    &lt;CapabilityItem
    text="Maximum file size: 5MB"
    colors={colors}
    /&gt;

```

```

<CapabilityItem
  text="Aadhaar update forms and letters"
  colors={colors}
/>

<CapabilityItem
  text="Kerala Ration card applications"
  colors={colors}
/>

<CapabilityItem
  text="Scholarship portal forms"
  colors={colors}
/>

<CapabilityItem
  text="General identity proofs (Voter ID, PAN)"
  colors={colors}
/>
</View>

<View
  style={[
    styles.limitationsCard,
    {
      backgroundColor: colors.warningBackground,
    },
  ]}
  >
  <View style={styles.limitationsHeader}>
    <Ionicons
      name="alert-circle-outline"
      size={21}
      color={colors.warning}
    />

    <Text
      style={[
        styles.limitationsTitle,
        {
          color: colors.warning,
        },
      ]}
    />
    Important limitations
  </Text>
</View>

  <Text
    style={[
      styles.limitationsText,
      {
        color: colors.warning,
      },
    ]}
  />
  <Text>
    The vision capability is designed to identify the type of document to provide
  </Text>
</View>

  <Pressable
    onPress={() => router.push("/chat")}

```

```

        accessibilityRole="button"
        accessibilityLabel="Start an advisory chat"
        style={{({ pressed }) => [
          styles.startButton,
          {
            backgroundColor: colors.primary,
            opacity: pressed ? 0.78 : 1,
          },
        ]}}
      <Ionicons
        name="chatbubble-ellipses-outline"
        size={20}
        color="#FFFFFF"
      />

      <Text style={styles.startButtonText}>
        Start an advisory chat
      </Text>

      <Ionicons
        name="arrow-forward"
        size={19}
        color="#FFFFFF"
      />
    </Pressable>
  </ScrollView>
</SafeAreaView>
);
}

```

```

function CapabilityItem({
  text,
  colors,
}: {
  text: string;
  colors: any;
}) {
  return (
    <View style={styles.capabilityItem}>
      <Ionicons
        name="checkmark"
        size={16}
        color={colors.accent}
      />

      <Text
        style={[
          styles.capabilityText,
          {
            color: colors.textMuted,
          },
        ]}
      >
        {text}
      </Text>
    </View>
  );
}

```

```

const styles = StyleSheet.create({
  safeArea: {

```

```
    flex: 1,
  },

  content: {
    alignSelf: "center",
    maxWidth: 1000,
    padding: 22,
    paddingBottom: 55,
    width: "100%",
  },

  headerRow: {
    alignItems: "flex-start",
    flexDirection: "row",
    gap: 10,
  },

  backButton: {
    alignItems: "center",
    height: 40,
    justifyContent: "center",
    width: 40,
  },

  headerText: {
    flex: 1,
  },

  eyebrow: {
    fontSize: 10,
    fontWeight: "800",
    letterSpacing: 1.1,
    marginBottom: 9,
  },

  title: {
    fontSize: 30,
    fontWeight: "800",
    letterSpacing: -0.5,
    lineHeight: 37,
  },

  subtitle: {
    fontSize: 15,
    lineHeight: 23,
    marginTop: 10,
  },

  statusCard: {
    alignItems: "flex-start",
    borderRadius: 17,
    flexDirection: "row",
    gap: 12,
    marginTop: 25,
    padding: 16,
  },

  statusIcon: {
    alignItems: "center",
    borderRadius: 12,
    height: 42,
    justifyContent: "center",
```

```
        width: 42,
    },

    statusContent: {
        flex: 1,
    },

    statusTitle: {
        fontSize: 14,
        fontWeight: "800",
    },

    statusText: {
        fontSize: 13,
        lineHeight: 20,
        marginTop: 5,
    },

    sectionTitle: {
        fontSize: 20,
        fontWeight: "800",
        marginTop: 31,
    },

    sectionSubtitle: {
        fontSize: 13,
        lineHeight: 19,
        marginTop: 5,
    },

    serviceGrid: {
        alignItems: "stretch",
        flexDirection: "row",
        flexWrap: "wrap",
        gap: 12,
        marginTop: 16,
    },

    workflowHeader: {
        marginTop: 22,
    },

    workflowList: {
        gap: 10,
        marginTop: 16,
    },

    workflowCard: {
        alignItems: "center",
        borderRadius: 16,
        borderWidth: 1,
        flexDirection: "row",
        gap: 11,
        padding: 14,
    },

    stepNumber: {
        alignItems: "center",
        borderRadius: 10,
        height: 38,
        justifyContent: "center",
        width: 38,
```

```
},

stepNumberText: {
  fontSize: 11,
  fontWeight: "800",
},

workflowIcon: {
  alignItems: "center",
  borderRadius: 10,
  height: 38,
  justifyContent: "center",
  width: 38,
},

workflowContent: {
  flex: 1,
},

workflowTitle: {
  fontSize: 14,
  fontWeight: "800",
},

workflowText: {
  fontSize: 12,
  lineHeight: 18,
  marginTop: 4,
},

capabilityCard: {
  borderRadius: 17,
  borderWidth: 1,
  marginTop: 24,
  padding: 17,
},

capabilityHeader: {
  alignItems: "center",
  flexDirection: "row",
  gap: 8,
  marginBottom: 13,
},

capabilityTitle: {
  fontSize: 15,
  fontWeight: "800",
},

capabilityItem: {
  alignItems: "flex-start",
  flexDirection: "row",
  gap: 9,
  marginBottom: 9,
},

capabilityText: {
  flex: 1,
  fontSize: 13,
  lineHeight: 19,
},
```

```

limitationsCard: {
  borderRadius: 17,
  marginTop: 12,
  padding: 17,
},

limitationsHeader: {
  alignItems: "center",
  flexDirection: "row",
  gap: 8,
  marginBottom: 8,
},

limitationsTitle: {
  fontSize: 15,
  fontWeight: "800",
},

limitationsText: {
  fontSize: 13,
  lineHeight: 20,
},

startButton: {
  alignItems: "center",
  borderRadius: 13,
  flexDirection: "row",
  gap: 9,
  justifyContent: "center",
  marginTop: 20,
  padding: 16,
},

startButtonText: {
  color: "#FFFFFF",
  flex: 1,
  fontSize: 15,
  fontWeight: "800",
},
});

```

File: src\app\history.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import { useEffect, useMemo, useState } from "react";
import { getDeviceId } from "../utils/device";
import {
  Alert,
  FlatList,
  Platform,
  Pressable,
  SafeAreaView,
  StyleSheet,
  Text,
  TextInput,
  View,
} from "react-native";

import { useTheme } from "../contexts/ThemeContext";

```

```

type Category =
  | "all"
  | "aadhaar"
  | "ration_card"
  | "scholarship"
  | "general";

type Conversation = {
  id: string;
  title: string;
  preview: string;
  category: Exclude<Category, "all">;
  categoryLabel: string;
  updatedLabel: string;
  messageCount: number;
};

const filters: { label: string; value: Category }[] = [
  { label: "All", value: "all" },
  { label: "Aadhaar", value: "aadhaar" },
  { label: "Ration Card", value: "ration_card" },
  { label: "Scholarship", value: "scholarship" },
  { label: "General", value: "general" },
];

import { API_BASE_URL } from "../config/api";

const API_URL = API_BASE_URL;

export default function HistoryScreen() {
  const { colors } = useTheme();
  const [search, setSearch] = useState("");
  const [conversations, setConversations] = useState<Conversation[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    async function fetchHistory() {
      try {
        const actualDeviceId = await getDeviceId();
        const response = await fetch(`${API_URL}/conversations?device_id=${actualDeviceId}`);
        if (!response.ok) throw new Error("Failed to load history");
        const data = await response.json();
        setConversations(data);
      } catch (err) {
        console.error(err);
      } finally {
        setIsLoading(false);
      }
    }
    fetchHistory();
  }, []);

  const visibleConversations = useMemo(() => {
    const normalizedSearch = search.trim().toLowerCase();

    return conversations.filter((conversation) => {
      const textMatch =
        normalizedSearch.length === 0 ||
        conversation.title.toLowerCase().includes(normalizedSearch) ||
        conversation.preview.toLowerCase().includes(normalizedSearch) ||
        conversation.categoryLabel.toLowerCase().includes(normalizedSearch);
    });
  }, [conversations, search]);

```

```

        return textMatch;
    });
}, [conversations, search]);

function removeConversation(conversation: Conversation) {
const remove = async () => {
    try {
        await fetch(`${API_BASE_URL}/conversations/${conversation.id}`, { method: 'DELETE' });
        setConversations((current) => {
            current.filter((item) => item.id !== conversation.id),
        });
    } catch (e) {
        console.error("Failed to delete conversation", e);
    }
};

if (Platform.OS === "web") {
    if (
        window.confirm(
            `Remove "${conversation.title}" from your chat history?`,
        )
    ) {
        remove();
    }

    return;
}

Alert.alert(
    "Remove conversation",
    `Remove "${conversation.title}" from your chat history?`,
    [
        { text: "Cancel", style: "cancel" },
        { text: "Remove", style: "destructive", onPress: remove }
    ],
);
}

function openConversation(conversation: Conversation) {
    router.push({
        pathname: "/",
        params: {
            conversation_id: conversation.id,
        },
    });
}

function clearFilters() {
    setSearch("");
}

return (
    <SafeAreaView
        style={[
            styles.safeArea,
            {
                backgroundColor: colors.background,
            },
        ]
    />
    <View
        style={

```

```

        styles.header,
        {
            backgroundColor: colors.surface,
            borderBottomColor: colors.border,
        },
    ]}
    >
    <Pressable
        onPress={() => router.canGoBack() ? router.back() : router.replace('/') }
        accessibilityRole="button"
        accessibilityLabel="Go back"
        style={styles.headerIcon}
    >
        <Ionicons name="arrow-back" size={22} color={colors.text} />
    </Pressable>

    <View style={styles.headerText}>
        <Text
            style={[
                styles.headerTitle,
                {
                    color: colors.text,
                },
            ]}
        >
            Chat history
        </Text>

        <Text
            style={[
                styles.headerSubtitle,
                {
                    color: colors.textMuted,
                },
            ]}
        >
            Your saved advisory conversations
        </Text>
    </View>

    <Pressable
        onPress={() => router.replace("/")}
        accessibilityRole="button"
        accessibilityLabel="Start a new chat"
        style={[
            styles.newChatButton,
            {
                backgroundColor: colors.primary,
            },
        ]}
    >
        <Ionicons name="add" size={20} color="#FFFFFF" />
    </Pressable>
    </View>

    <FlatList
        data={visibleConversations}
        keyExtractor={(item) => item.id}
        showsVerticalScrollIndicator={false}
        contentContainerStyle={[
            styles.content,
            visibleConversations.length === 0 && styles.emptyContent,
        ]}
    >

```

```

]]
ListHeaderComponent={
  <View>
    <View
      style={[
        styles.searchBox,
        {
          backgroundColor: colors.surface,
          borderColor: colors.border,
        }
      ]}
    >
      <Ionicons
        name="search-outline"
        size={19}
        color={colors.textMuted}
      />

      <TextInput
        value={search}
        onChangeText={setSearch}
        placeholder="Search conversations..."
        placeholderTextColor={colors.textMuted}
        style={[
          styles.searchInput,
          {
            color: colors.text,
          }
        ]}
        accessibilityLabel="Search chat history"
      />

      {search.length > 0 & & (
        <Pressable
          onPress={() => setSearch("")}
          accessibilityRole="button"
          accessibilityLabel="Clear search"
        >
          <Ionicons
            name="close-circle"
            size={18}
            color={colors.textMuted}
          />
        </Pressable>
      )}
    </View>

    <Text
      style={[
        styles.listLabel,
        {
          color: colors.textMuted,
        }
      ]}
    >
      SAVED CONVERSATIONS
    </Text>
  </View>
}
renderItem={({ item }) => (
  <ConversationCard

```

```

        conversation={item}
        colors={colors}
        onOpen={() => openConversation(item)}
        onRemove={() => removeConversation(item)}
    />
  )}
  ListEmptyComponent={
    <EmptyHistory
      colors={colors}
      filtered={search.length > 0}
      onClear={clearFilters}
      onNewChat={() => router.replace("/")}
    />
  }
  />
  </SafeAreaView>
);
}

function ConversationCard({
  conversation,
  colors,
  onOpen,
  onRemove,
}): {
  conversation: Conversation;
  colors: any;
  onOpen: () => void;
  onRemove: () => void;
}) {
  return (
    <View
      style={[
        styles.conversationCard,
        {
          backgroundColor: colors.surface,
          borderColor: colors.border,
        },
      ]}
    />
    <Pressable
      onPress={onOpen}
      accessibilityRole="button"
      accessibilityLabel={`Open ${conversation.title}`}
      style={({ pressed }) => [
        styles.conversationMain,
        {
          opacity: pressed ? 0.75 : 1,
        },
      ]}
    />
    <View
      style={[
        styles.conversationIcon,
        {
          backgroundColor: colors.primarySoft,
        },
      ]}
    />
    <Ionicons
      name={getIcon(conversation.category)}
      size={20}

```

```

        color={colors.primaryDark}
      />
    </View>

    <View style={styles.conversationTextArea}>
      <View style={styles.titleRow}>
        <Text
          numberOfLines={1}
          style={[
            styles.conversationTitle,
            {
              color: colors.text,
            },
          ]}
        >
          {conversation.title}
        </Text>

        <Text
          style={[
            styles.updatedText,
            {
              color: colors.textMuted,
            },
          ]}
        >
          {conversation.updatedLabel}
        </Text>
      </View>

      <Text
        numberOfLines={2}
        style={[
          styles.previewText,
          {
            color: colors.textMuted,
          },
        ]}
      >
        {conversation.preview}
      </Text>

      <View style={styles.cardFooter}>
        <View
          style={[
            styles.categoryBadge,
            {
              backgroundColor: colors.surfaceMuted,
            },
          ]}
        >
          <Text
            style={[
              styles.categoryText,
              {
                color: colors.primaryDark,
              },
            ]}
          >
            {conversation.categoryLabel}
          </Text>
        </View>
      </View>
    </View>
  </View>

```

```

        <Text
          style={
            styles.messageCount,
            {
              color: colors.textMuted,
            },
          }
        >
        {conversation.messageCount} messages
      </Text>
    </View>
  </View>
</Pressable>

<Pressable
  onPress={onRemove}
  accessibilityRole="button"
  accessibilityLabel={`Remove ${conversation.title}`}
  style={({ pressed }) => [
    styles.removeButton,
    {
      backgroundColor: pressed
        ? colors.dangerBackground
        : "transparent",
    },
  ]}
  >
    <Ionicons
      name="ellipsis-horizontal"
      size={20}
      color={colors.textMuted}
    />
  </Pressable>
</View>
);
}

function EmptyHistory({
  colors,
  filtered,
  onClear,
  onNewChat,
}): {
  colors: any;
  filtered: boolean;
  onClear: () => void;
  onNewChat: () => void;
} {
  return (
    <View
      style={
        styles.emptyState,
        {
          backgroundColor: colors.surface,
          borderColor: colors.border,
        },
      }
    >
      <View
        style={
          styles.emptyIcon,
          {

```

```

        backgroundColor: colors.primarySoft,
    },
  ]}
  &gt;
  &lt;Ionicons
    name={filtered ? "search-outline" : "chatbubbles-outline"}
    size={30}
    color={colors.primaryDark}
  /&gt;
&lt;/View&gt;

&lt;Text
  style=[
    styles.emptyTitle,
    {
      color: colors.text,
    },
  ],
  &gt;
  {filtered ? "No matching chats" : "No saved chats yet"}
&lt;/Text&gt;

&lt;Text
  style=[
    styles.emptyText,
    {
      color: colors.textMuted,
    },
  ],
  &gt;
  {filtered
    ? "Try another search or clear the current topic filter."
    : "Your advisory conversations will appear here after backend integration."}
&lt;/Text&gt;

&lt;View style={styles.emptyActions}&gt;
  {filtered &amp; & (
    &lt;Pressable
      onPress={onClear}
      accessibilityRole="button"
      accessibilityLabel="Clear filters"
      style=[
        styles.clearButton,
        {
          backgroundColor: colors.surfaceMuted,
          borderColor: colors.border,
        },
      ],
    &gt;
    &lt;Text
      style=[
        styles.clearButtonText,
        {
          color: colors.text,
        },
      ],
    &gt;
    Clear filters
    &lt;/Text&gt;
    &lt;/Pressable&gt;
  )}

```

```

        <Pressable
          onPress={onNewChat}
          accessibilityRole="button"
          accessibilityLabel="Start new chat"
          style={[
            styles.startButton,
            {
              backgroundColor: colors.primary,
            },
          ]}
        >
          <Ionicons name="add" size={18} color="#FFFFFF" />
          <Text style={styles.startButtonText}>New chat</Text>
        </Pressable>
      </View>
    </View>
  );
}

function getIcon(
  category: Exclude<Category, "all">,
): keyof typeof Ionicons.glyphMap {
  if (category === "aadhaar") {
    return "card-outline";
  }

  if (category === "ration_card") {
    return "fast-food-outline";
  }

  if (category === "scholarship") {
    return "school-outline";
  }

  return "documents-outline";
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },
  header: {
    alignItems: "center",
    borderBottomWidth: 1,
    flexDirection: "row",
    minHeight: 68,
    paddingHorizontal: 12,
  },
  headerIcon: {
    alignItems: "center",
    height: 42,
    justifyContent: "center",
    width: 42,
  },
  headerText: {
    flex: 1,
    marginHorizontal: 7,
  },

```

```
headerTitle: {
  fontSize: 16,
  fontWeight: "800",
},

headerSubtitle: {
  fontSize: 10,
  marginTop: 3,
},

newChatButton: {
  alignItems: "center",
  borderRadius: 12,
  height: 40,
  justifyContent: "center",
  width: 40,
},

content: {
  alignSelf: "center",
  maxWidth: 760,
  padding: 16,
  paddingBottom: 42,
  width: "100%",
},

emptyContent: {
  flexGrow: 1,
},

searchBox: {
  alignItems: "center",
  borderRadius: 14,
  borderWidth: 1,
  flexDirection: "row",
  gap: 8,
  minHeight: 49,
  paddingHorizontal: 13,
},

searchInput: {
  flex: 1,
  fontSize: 14,
  minHeight: 42,
  padding: 0,
},

filterHeader: {
  alignItems: "center",
  flexDirection: "row",
  justifyContent: "space-between",
  marginTop: 22,
},

filterTitle: {
  fontSize: 13,
  fontWeight: "800",
},

countText: {
  fontSize: 11,
},
```

```
filters: {
  gap: 8,
  paddingBottom: 5,
  paddingTop: 11,
},

filterChip: {
  borderRadius: 999,
  borderWidth: 1,
  paddingHorizontal: 13,
  paddingVertical: 8,
},

filterChipText: {
  fontSize: 11,
  fontWeight: "700",
},

listLabel: {
  fontSize: 10,
  fontWeight: "800",
  letterSpacing: 0.9,
  marginBottom: 11,
  marginTop: 21,
},

conversationCard: {
  alignItems: "center",
  borderRadius: 16,
  borderWidth: 1,
  flexDirection: "row",
  marginBottom: 9,
  minHeight: 108,
  padding: 7,
},

conversationMain: {
  alignItems: "flex-start",
  flex: 1,
  flexDirection: "row",
  gap: 10,
  minWidth: 0,
  padding: 7,
},

conversationIcon: {
  alignItems: "center",
  borderRadius: 11,
  height: 40,
  justifyContent: "center",
  width: 40,
},

conversationTextArea: {
  flex: 1,
  minWidth: 0,
},

titleRow: {
  alignItems: "center",
  flexDirection: "row",
  gap: 7,
```

```
},

conversationTitle: {
  flex: 1,
  fontSize: 13,
  fontWeight: "800",
},

updatedText: {
  fontSize: 10,
},

previewText: {
  fontSize: 11,
  lineHeight: 17,
  marginTop: 5,
},

cardFooter: {
  alignItems: "center",
  flexDirection: "row",
  justifyContent: "space-between",
  marginTop: 10,
},

categoryBadge: {
  borderRadius: 7,
  paddingHorizontal: 7,
  paddingVertical: 4,
},

categoryText: {
  fontSize: 9,
  fontWeight: "800",
},

messageCount: {
  fontSize: 9,
},

removeButton: {
  alignItems: "center",
  borderRadius: 9,
  height: 38,
  justifyContent: "center",
  marginRight: 3,
  width: 38,
},

emptyState: {
  alignItems: "center",
  borderRadius: 18,
  borderWidth: 1,
  justifyContent: "center",
  marginTop: 20,
  minHeight: 290,
  padding: 25,
},

emptyIcon: {
  alignItems: "center",
  borderRadius: 18,
```

```

        height: 64,
        justifyContent: "center",
        marginBottom: 15,
        width: 64,
      },

      emptyTitle: {
        fontSize: 18,
        fontWeight: "800",
      },

      emptyText: {
        fontSize: 12,
        lineHeight: 18,
        marginTop: 8,
        maxWidth: 340,
        textAlign: "center",
      },

      emptyActions: {
        alignItems: "center",
        flexDirection: "row",
        flexWrap: "wrap",
        gap: 9,
        justifyContent: "center",
        marginTop: 19,
      },

      clearButton: {
        borderRadius: 10,
        borderWidth: 1,
        paddingHorizontal: 12,
        paddingVertical: 10,
      },

      clearButtonText: {
        fontSize: 11,
        fontWeight: "700",
      },

      startButton: {
        alignItems: "center",
        borderRadius: 10,
        flexDirection: "row",
        gap: 5,
        paddingHorizontal: 13,
        paddingVertical: 10,
      },

      startButtonText: {
        color: "#FFFFFF",
        fontSize: 11,
        fontWeight: "800",
      },
    },
  });

```

File: src/app/index.tsx

```

import { API_BASE_URL } from '../config/api';
import { Ionicons } from "@expo/vector-icons";

```

```

import { router, useLocalSearchParams } from "expo-router";
import * as Speech from 'expo-speech';
import * as Clipboard from 'expo-clipboard';
import { Image as ExpoImage } from 'expo-image';
import React, { useEffect, useMemo, useRef, useState } from "react";
import {
  Alert,
  Animated,
  LayoutAnimation,
  Image,
  KeyboardAvoidingView,
  Linking,
  Modal,
  Platform,
  Pressable,
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  useWindowDimensions,
  View,
  UIManager,
} from "react-native";

import AppDrawerContent from "../components/AppDrawerContent";
import ChatImagePreview from "../components/ChatImagePreview";
import QueryComposer, { SelectedImage } from "../components/QueryComposer";
import { useTheme } from "../contexts/ThemeContext";
import { getDeviceId } from "../utils/device";
import { cleanText } from "../utils/cleanResponseText";
import { normalizeChatResponse } from "../utils/responseAdapter";

if (Platform.OS === 'android' && UIManager.setLayoutAnimationEnabledExperimental)
  UIManager.setLayoutAnimationEnabledExperimental(true);
}

const LOGO = require("../assets/images/akshaya-logo.png");

const stripForSpeech = (text: string) => {
  if (!text) return "";
  return text.replace(/[#%_]/g, "").trim();
};

const cleanString = (text: string) => {
  if (!text) return "";
  return text.replace(/[#%]/g, "").trim();
};

function FormattedText({ text, style }: { text: string; style?: any }) {
  if (!text) return null;
  // Remove # and % as requested, but keep ** for parsing
  const cleaned = text.replace(/[#%]/g, "");
  const parts = cleaned.split(/\\*(.*?)\\*/g);

  return (
    <Text style={style}>
      {parts.map((part, index) => {
        const isBold = index % 2 === 1;
        return (
          <Text key={index} style={isBold ? { fontWeight: "bold" } : {}}>
            {part}
          </Text>
        );
      })}
    </Text>
  );
}

```

```

        </Text>
    );
    })
    </Text>
);
}

type SourceCitation = {
    source_title: string;
    source_url?: string;
    authority: string;
    retrieved_date?: string;
};

type AnswerData = {
    summary: string;
    eligibility: string[];
    documents: string[];
    next_steps: string[];
    where_to_go: string[];
    warning: string;
};

type BackendData = {
    success?: boolean;
    status?: string;
    message_id: number;
    conversation_id: number;
    response_type?: string;
    detected_service?: string;
    answer: AnswerData;
    citations: SourceCitation[];
    confidence_score: number;
    confidence_level: string;
    requires_official_verification: boolean;
    verification_message?: string;
    image_description?: string;
    privacy_warning?: string;
    image_analysis?: {
        document_type: string;
        confidence: number;
        sensitive_data_hidden: boolean;
    };
};

type Message = {
    id: string;
    role: "assistant" | "user";
    text: string;
    inputType?: "text" | "voice";
    imageUri?: string;
    imageMimeType?: string;
    imageFileName?: string;
    backendData?: BackendData;
};

const suggestions = [
    { text: "What documents do I need for a new ration card in Kerala?", icon: "list-outlin
    { text: "Am I eligible for a National Scholarship, and what documents are needed?", ico
    { text: "My name in Aadhaar doesn't match my certificate - how do I update it?", icon
];

```

```

const welcomeMessage: Message = {
  id: "welcome",
  role: "assistant",
  text: "Describe your issue in your own words - Aadhaar, ration card, or scholarship."
};

export default function HomeChatScreen() {
  const { colors } = useTheme();
  const { width } = useWindowDimensions();

  const params = useLocalSearchParams<{
    query?: string | string[];
    conversation_id?: string | string[];
  }>();

  const initialQuery = useMemo(() => {
    if (Array.isArray(params.query)) {
      return params.query[0] ?? "";
    }
    return params.query ?? "";
  }, [params.query]);

  const paramConvId = useMemo(() => {
    if (Array.isArray(params.conversation_id)) {
      return params.conversation_id[0] ?? null;
    }
    return params.conversation_id ?? null;
  }, [params.conversation_id]);

  const scrollRef = useRef<ScrollView>(null);
  const processedQueryRef = useRef<string | null>(null);

  const [drawerVisible, setDrawerVisible] = useState(false);
  const [transcriptVisible, setTranscriptVisible] = useState(false);

  const exportTranscript = () => {
    let transcriptText = "Conversation Transcript\n=====\n\n";
    messages.forEach(msg => {
      if (msg.role === "assistant") {
        transcriptText += `[Akshaya AI]:\n${msg.text}\n\n`;
      } else {
        const typeLabel = msg.inputType === "voice" ? "[Voice Input]" : "";
        transcriptText += `[You]${typeLabel}:\n${msg.text}\n\n`;
      }
    });

    if (Platform.OS === 'web') {
      const blob = new Blob([transcriptText], { type: 'text/plain' });
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = `transcript-${new Date().getTime()}.txt`;
      a.click();
    } else {
      // In a real app we'd use expo-file-system and expo-sharing, but Alert is ok for now
      Alert.alert("Transcript", transcriptText);
    }
  };

  const [messages, setMessages] = useState<Message[]>([welcomeMessage]);
  const [isProcessing, setIsProcessing] = useState(false);
  const [conversationId, setConversationId] = useState<number | null>(null);

```

```

const isDesktop = width >= 800;
const API_URL = API_BASE_URL;

useEffect(() => {
  async function loadConversation() {
    if (!paramConvId) return;

    const idStr = paramConvId.toString();
    // Only fetch if it changed
    if (conversationId?.toString() === idStr && messages.length > 1) return;

    try {
      setIsProcessing(true);
      const res = await fetch(`${API_URL}/conversations/${idStr}`);
      if (!res.ok) throw new Error("Failed to load conversation");
      const data = await res.json();

      setConversationId(data.conversation_id);

      const loadedMessages: Message[] = [welcomeMessage];
      for (const msg of data.messages) {
        if (msg.role === "user") {
          loadedMessages.push({
            id: `user-${msg.id}`,
            role: "user",
            text: msg.text_content,
            inputType: (msg as any).input_type || "text",
            imageUri: msg.image_path ? (msg.image_path.startsWith('http') ? msg.ima
          });
        } else {
          loadedMessages.push({
            id: `assistant-${msg.id}`,
            role: "assistant",
            text: msg.text_content,
            backendData: {
              message_id: msg.id,
              conversation_id: data.conversation_id,
              detected_service: msg.detected_service,
              answer: { summary: msg.text_content || "", eligibility: [], documents: []
              citations: msg.sources || [],
              confidence_score: msg.confidence_score,
              confidence_level: msg.confidence_score > 0.7 ? "high" : "low",
              requires_official_verification: msg.requires_verification,
            }
          });
        }
      }
      setMessages(loadedMessages);
    } catch (err) {
      console.error(err);
    } finally {
      setIsProcessing(false);
    }
  }

  loadConversation();
}, [paramConvId]);

useEffect(() => {
  if (
    initialQuery.length > 0 &&
    processedQueryRef.current !== initialQuery

```

```

    ) {
      processedQueryRef.current = initialQuery;
      submitQuestion(initialQuery);
    }
  }, [initialQuery]);

  useEffect(() => {
    const timer = setTimeout(() => {
      scrollRef.current?.scrollToEnd({ animated: true });
    }, 150);

    return () => clearTimeout(timer);
  }, [messages, isProcessing]);

  function startNewChat() {
    try {
      setMessages([welcomeMessage]);
      setConversationId(null);
      processedQueryRef.current = null;
      router.replace("/");
    } catch (e) {
      console.error("[App] Error in startNewChat:", e);
    }
  }

  async function submitQuestion(question: string, image?: SelectedImage, inputType?: "text" | "image") {
    const cleanedQuestion = question.trim();

    if ((!cleanedQuestion && !image) || isProcessing) {
      return;
    }

    LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
    setMessages((currentMessages) => [
      ...currentMessages,
      {
        id: `user-${Date.now()}`,
        role: "user",
        text: cleanedQuestion,
        inputType: inputType || "text",
        imageUri: image?.uri,
        imageMimeType: image?.mimeType,
        imageFileName: image?.fileName,
      },
    ]);

    setIsProcessing(true);

    try {
      const actualDeviceId = await getDeviceId();
      const formData = new FormData();
      formData.append("device_id", actualDeviceId);
      formData.append("input_type", inputType || "text");
      if (cleanedQuestion) {
        formData.append("text", cleanedQuestion);
      }

      if (image) {
        if (Platform.OS === 'web') {
          const res = await fetch(image.uri);
          const blob = await res.blob();
          const file = new File([blob], image.fileName, { type: image.mimeType });
        }
      }
    }
  }

```

```

        formData.append("image", file);
    } else {
        formData.append("image", {
            uri: image.uri,
            name: image.fileName,
            type: image.mimeType,
        } as any);
    }
}

if (conversationId) {
    formData.append("conversation_id", conversationId.toString());
}

const response = await fetch(`${API_URL}/chat`, {
    method: "POST",
    body: formData,
});

if (!response.ok) {
    throw new Error(`API error: ${response.status}`);
}

const rawData = await response.json();
const data: BackendData = normalizeChatResponse(rawData);

setConversationId(data.conversation_id);

LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
setMessages((currentMessages) => [
    ...currentMessages,
    {
        id: data.message_id ? data.message_id.toString() : `msg-${Date.now()}`,
        role: "assistant",
        text: data.answer.summary,
        backendData: data,
    },
]);
} catch (error: any) {
    console.error("Chat error:", {
        message: error.message,
        name: error.name,
        url: `${API_URL}/chat`,
        method: "POST"
    });

    let errorMessage = "An unexpected error occurred.";

    if (error instanceof TypeError && error.message.includes("Failed to fetch"))
        // Network-level failure (CORS, backend down, wrong URL)
        errorMessage = "Can't reach the assistant right now - check your connection and t
    } else if (error.message && error.message.startsWith("API error:")) {
        // HTTP error (got a response, but it was 4xx/5xx)
        errorMessage = "The server encountered an error processing your request. Please t
    } else if (error instanceof SyntaxError) {
        // JSON parsing failure
        errorMessage = "Received an invalid response from the server. Please try again.";
    } else {
        errorMessage = "Something went wrong: " + error.message;
    }

    LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);

```

```

    setMessages((currentMessages) => [
      ...currentMessages,
      {
        id: `error-${Date.now()}`,
        role: "assistant",
        text: errorMessage,
      },
    ]);
  } finally {
    setIsProcessing(false);
  }
}

const hasUserMessage = messages.some(
  (message) => message.role === "user",
);

return (
  <SafeAreaView
    style={[
      styles.safeArea,
      {
        backgroundColor: colors.background,
      },
    ]}
  >
    <KeyboardAvoidingView
      style={styles.keyboardView}
      behavior={Platform.OS === "ios" ? "padding" : undefined}
    >
      <View
        style={[
          styles.header,
          {
            backgroundColor: colors.surface,
            borderBottomColor: colors.border,
          },
        ]}
      >
        <View
          style={[
            styles.headerInner,
            isDesktop & & styles.desktopHeaderInner,
          ]}
        >
          <Pressable
            onPress={() => setDrawerVisible(true)}
            accessibilityRole="button"
            accessibilityLabel="Open application menu"
            style={[
              styles.headerButton,
              {
                backgroundColor: colors.primarySoft,
              },
            ]}
          >
            <Ionicons
              name="menu-outline"
              size={25}
              color={colors.primaryDark}
            />
          </Pressable>
        </View>
      </View>
    </KeyboardAvoidingView>
  </SafeAreaView>
);

```

```

    <Pressable
      onPress={startNewChat}
      accessibilityRole="button"
      accessibilityLabel="Start a new chat"
      style={styles.brandButton}
    >
      <View style={[styles.brandLogoContainer, { borderWidth: 0 }]}>
        <Image
          source={LOGO}
          resizeMode="contain"
          style={styles.brandLogoImage}
          accessibilityLabel="Akshaya Advisory logo"
        />
      </View>

      <View style={styles.brandTextArea}>
        <Text
          style={[
            styles.brandTitle,
            {
              color: colors.text,
            },
          ]}
        >
          Akshaya Advisory
        </Text>

        <Text
          style={[
            styles.brandSubtitle,
            {
              color: colors.textMuted,
            },
          ]}
        >
          Citizen-service assistant
        </Text>
      </View>
    </Pressable>

    </View>
  </View>

  <ScrollView
    ref={scrollRef}
    showsVerticalScrollIndicator={false}
    keyboardShouldPersistTaps="handled"
    contentContainerStyle={[
      styles.content,
      isDesktop & styles.desktopContent,
    ]}
  >
    {messages.length === 1 & (
      <View style={styles.welcomeArea}>
        <View style={[styles.heroLogoWrapper, { backgroundColor: colors.primaryS
          <Ionicons
            name="shield-checkmark-outline"
            size={32}
            color={colors.primaryDark}
          />
        </View>
      )}
    )}

```

```

    <Text style={styles.welcomeTitle, { color: colors.text }}>
      How can we help today?
    </Text>

    <Text style={styles.welcomeSubtitle, { color: colors.textMuted }}>
      {welcomeMessage.text}
    </Text>

    <View style={styles.assuranceCard, { backgroundColor: colors.primarySoft }}>
      <Image style={styles.assuranceIcon, { width: 40, height: 40 }}
        source={require('assets/images/assurance.png')} alt="Assurance icon" data-bbox="318 211 368 231"/>
      <Text style={styles.assuranceText, { color: colors.primaryDark }}>
        Clear advisory guidance before you visit an Akshaya centre.
      </Text>
    </View>

    <Text style={styles.suggestionHeading, { color: colors.textMuted, fontF
      Try asking
    </Text>

    <View style={styles.suggestions}>
      {suggestions.map((suggestion, index) => (
        <Pressable
          key={index}
          onPress={() => submitQuestion(suggestion.text)}
          accessibilityRole="button"
          accessibilityLabel={`Ask: ${suggestion.text}`}
          style={({ pressed }) => [
            styles.suggestionCard,
            {
              backgroundColor: colors.surface,
              borderColor: colors.border,
              opacity: pressed ? 0.76 : 1,
            },
          ]},
        <View style={styles.suggestionIcon, { backgroundColor: colors.primarySoft }}>
          <Image style={styles.suggestionIconImage, { width: 24, height: 24 }}
            source={require('assets/images/suggestion.png')} alt="Suggestion icon" data-bbox="371 561 421 581"/>
          </View>
          <Text style={styles.suggestionText, { color: colors.text }}>
            {suggestion.text}
          </Text>
          <Image style={styles.suggestionArrow, { width: 16, height: 16 }}
            source={require('assets/images/chevron.png')} alt="Chevron icon" data-bbox="371 626 421 646"/>
          </Pressable>
        </View>
      ))}
    </View>
  </View>
)}

(messages.filter(msg => msg.id !== 'welcome').map((message, index, filteredM
// Find the immediately preceding user message for retries
let lastUserQuery = "";
let lastUserImage: SelectedImage | undefined = undefined;
if (message.backendData?.response_type === 'service_unavailable') {
  for (let i = index - 1; i >= 0; i--) {
    if (filteredMessages[i].role === 'user') {
      lastUserQuery = filteredMessages[i].text;
      lastUserImage = filteredMessages[i].imageUri ? { uri: filteredMessages[i].imageUri } : undefined;
      break;
    }
  }
}
}
}

```

```

        return (
          <React.Fragment key={message.id}>
            <MessageBubble
              message={message}
              onRetry={lastUserQuery ? () => submitQuestion(lastUserQuery, lastUse
            />
            {message.role === 'assistant' && message.backendData &&
              (message.backendData.response_type === 'answer' || message.backendData.r
              <AdvisoryDetails data={message.backendData} />
            )}
          </React.Fragment>
        );
      })}

      {isProcessing && <TypingIndicator />}
    </ScrollView>

    <View style={styles.composerArea}>
      <View style={[styles.composerWrapper, isDesktop && styles.desktopCom
        <QueryComposer onSubmit={submitQuestion} disabled={isProcessing} placehold
      </View>
    </View>
    </KeyboardAvoidingView>

    <Modal visible={drawerVisible} transparent animationType="slide" onRequestClose=
      <View style={[styles.drawerOverlay, { backgroundColor: colors.overlay }]}>
        <View style={[styles.drawerPanel, { backgroundColor: colors.surface }]}>
          <Pressable onPress={() => setDrawerVisible(false)} style={[styles.close
            <Icons name="close" size={24} color={colors.primaryDark} />
          </Pressable>
          <AppDrawerContent onClose={() => setDrawerVisible(false)} onViewTranscr
        </View>
        <Pressable onPress={() => setDrawerVisible(false)} style={styles.drawerDi
      </View>
    </Modal>

    <Modal visible={transcriptVisible} animationType="slide" presentationStyle="page
      <SafeAreaView style={{ flex: 1, backgroundColor: colors.background }}>
        <View style={[styles.header, { backgroundColor: colors.surface, borderBottom
          <View style={styles.headerInner}>
            <Pressable onPress={() => setTranscriptVisible(false)} style={[styles
              <Icons name="close" size={24} color={colors.primaryDark} />
            </Pressable>
            <View style={styles.brandTextArea}>
              <Text style={[styles.brandTitle, { color: colors.text }]}>Transcrip
            </View>
            <Pressable onPress={exportTranscript} style={[styles.headerButton, { bac
              <Icons name="download-outline" size={24} color={colors.primaryDark}
            </Pressable>
          </View>
        </View>
      <ScrollView style={{ flex: 1, padding: 16 }}>
        {messages.map((msg, idx) => (
          <View key={`ts-${msg.id}-${idx}`} style={{ marginBottom: 16, backgroundC
            <Text style={{ fontWeight: 'bold', color: msg.role === 'user' ? colors
              {msg.role === 'user' ? `You ${msg.inputType === 'voice' ? '(Voice Input
            </Text>
            <Text style={{ color: msg.role === 'user' ? colors.primaryDark : color
              {msg.text}
            </Text>
          </View>
        ))}

```

```

        </ScrollView>
        </SafeAreaView>
        </Modal>
        </SafeAreaView>
    );
}

function MessageBubble({ message, onRetry }: { message: Message, onRetry?: () => void }) {
    const { colors } = useTheme();
    const isUser = message.role === "user";

    const copyToClipboard = async () => {
        await Clipboard.setStringAsync(message.text);
    };

    return (
        <View style={[styles.messageRow, isUser ? styles.userRow : styles.assistantRow]}>
            <!isUser && (
                <View style={styles.assistantAvatarImageContainer}>
                    <Image source={LOGO} resizeMode="contain" style={styles.assistantAvatarImage} />
                </View>
            )
            <View
                style={[
                    styles.messageBubble,
                    isUser ? styles.userBubble : styles.assistantBubble,
                    {
                        backgroundColor: isUser ? colors.primary : colors.surface,
                        borderColor: isUser ? colors.primary : colors.border,
                        borderWidth: 1, // Cleaner without border in dark mode
                        shadowColor: "#000",
                        shadowOffset: { width: 0, height: 1 },
                        shadowOpacity: 0.05,
                        shadowRadius: 2,
                        elevation: 2,
                        position: 'relative'
                    },
                ]}
            >
                {message.imageUri ? (
                    Platform.OS === 'web' && message.imageUri.startsWith('blob:') ? (
                        <img
                            src={message.imageUri}
                            style={{ width: 200, height: 200, borderRadius: 8, marginBottom: 8, objectFit: 'cover' }}
                        />
                    ) : (
                        <Image
                            source={{ uri: message.imageUri }}
                            style={{ width: 200, height: 200, borderRadius: 8, marginBottom: 8, backgroundColor: 'white', resizeMode: "cover" }}
                        />
                    )
                ) : null}
                {isUser ? (
                    <View>
                        {message.inputType === "voice" && (
                            <View style={{ flexDirection: "row", alignItems: "center", marginBottom: 8 }}>
                                <Ionicons name="mic" size={12} color={"#ffffff"} style={{ marginRight: 8 }} />
                                <Text style={{ fontSize: 10, color: "#ffffff", opacity: 0.8 }} />
                            </View>
                        )}
                    </View>
                ) : null}
            </View>
        </View>
    );
}

```

```

        <Text style={styles.messageText, { color: "#ffffff", fontFamily: Platform.OS === 'ios' ? 'Times New Roman' : 'serif',
        {message.text}}
        </Text>
    </View>
) : (
    <View>
        <FormattedText
            text={message.text}
            style={
                [
                    styles.messageText,
                    {
                        color: colors.text,
                        fontFamily: Platform.OS === 'ios' ? 'Times New Roman' : 'serif',
                        fontSize: 16,
                        lineHeight: 24,
                    }
                ]
            }
        </FormattedText>
        <Pressable
            onPress={copyToClipboard}
            style={{ position: 'absolute', bottom: -5, right: -5, padding: 8 }}
        >
            <Ionicons name="copy-outline" size={16} color={colors.textMuted} />
        </Pressable>
    </View>
)
    </View>

    {isUser ? (
        <View style={styles.assistantAvatarImageContainer, { backgroundColor: colors.assistantAvatarImageContainer }}
        <Ionicons name="person" size={16} color="#FFF" />
    </View>
    ) : null}
    </View>
);
}

function TypingIndicator() {
    const { colors } = useTheme();

    const opacity1 = useRef(new Animated.Value(0.3)).current;
    const opacity2 = useRef(new Animated.Value(0.3)).current;
    const opacity3 = useRef(new Animated.Value(0.3)).current;

    useEffect(() => {
        const animateDot = (anim: Animated.Value, delay: number) => {
            Animated.loop(
                Animated.sequence([
                    Animated.timing(anim, {
                        toValue: 1,
                        duration: 400,
                        useNativeDriver: true,
                    }),
                    Animated.timing(anim, {
                        toValue: 0.3,
                        duration: 400,
                        useNativeDriver: true,
                    })
                ])
            ).start();
        };
    });
}

```

```

        setTimeout(() => animateDot(opacity1, 0), 0);
        setTimeout(() => animateDot(opacity2, 200), 200);
        setTimeout(() => animateDot(opacity3, 400), 400);
    }, []);

    return (
      <View style={styles.assistantRow}>
        <View style={styles.assistantAvatarImageContainer}>
          <Image source={LOGO} resizeMode="contain" style={styles.assistantAvatarImage} />
        </View>

        <View
          style={[
            styles.typingCard,
            {
              backgroundColor: colors.surface,
              borderColor: colors.border,
              borderWidth: 1,
              shadowColor: "#000",
              shadowOffset: { width: 0, height: 1 },
              shadowOpacity: 0.05,
              shadowRadius: 2,
              elevation: 2,
              flexDirection: 'row',
              alignItems: 'center',
              paddingHorizontal: 16,
              paddingVertical: 12,
              minHeight: 48,
            },
          ]}
        >
          <Animated.View style={{ width: 6, height: 6, borderRadius: 3, backgroundColor: colors.border }} />
          <Animated.View style={{ width: 6, height: 6, borderRadius: 3, backgroundColor: colors.border }} />
          <Animated.View style={{ width: 6, height: 6, borderRadius: 3, backgroundColor: colors.border }} />
        </View>
      </View>
    );
  }

  function AdvisoryDetails({ data }: { data: BackendData }) {
    const { colors } = useTheme();
    const [speechState, setSpeechState] = useState<'stopped' | 'playing' | 'paused'>('stopped');
    const [feedbackState, setFeedbackState] = useState<'initial' | 'helpful-submitted'>('initial');
    const [feedbackComment, setFeedbackComment] = useState('');
    const API_URL = API_BASE_URL;

    const answer = data.answer;
    const hasDocuments = answer?.documents && answer.documents.length > 0;
    const hasEligibility = answer?.eligibility && answer.eligibility.length > 0;
    const hasNextSteps = answer?.next_steps && answer.next_steps.length > 0;
    const hasWhereToGo = answer?.where_to_go && answer.where_to_go.length > 0;

    const textToRead = stripForSpeech(
      (answer?.summary || "") + ". " +
      (answer?.documents?.join(". ") || "") + ". " +
      (answer?.next_steps?.join(". ") || "")
    );

    const speechStateRef = useRef<'stopped' | 'playing' | 'paused'>('stopped');
    const segmentRef = useRef(0);
    const segments = useMemo(() => textToRead.match(/^[^!?]+[.!?]+/g) || [textToRead], [textToRead]);
  }

```

```

const syncState = (st: 'stopped' | 'playing' | 'paused') => {
  speechStateRef.current = st;
  setSpeechState(st);
};

const playNextSegment = (idx: number) => {
  if (speechStateRef.current !== 'playing') return;
  if (idx >= segments.length) {
    syncState('stopped');
    segmentRef.current = 0;
    return;
  }
  Speech.speak(segments[idx], {
    language: 'en-IN',
    rate: 0.9,
    onDone: () => {
      if (speechStateRef.current === 'playing') {
        segmentRef.current = idx + 1;
        playNextSegment(idx + 1);
      }
    },
    onStopped: () => {}
  });
};

const handlePlay = () => {
  try {
    if (Platform.OS === 'android') {
      if (speechStateRef.current === 'paused') {
        syncState('playing');
        playNextSegment(segmentRef.current);
      } else {
        Speech.stop();
        segmentRef.current = 0;
        syncState('playing');
        playNextSegment(0);
      }
    } else {
      if (speechStateRef.current === 'paused') {
        Speech.resume();
        syncState('playing');
      } else {
        Speech.stop();
        syncState('playing');
        Speech.speak(textToRead, {
          language: 'en-IN',
          rate: 0.9,
          onDone: () => syncState('stopped'),
          onStopped: () => { if (speechStateRef.current !== 'paused') syncState('sto
        });
      }
    }
  } catch (e) {
    console.error("[AdvisoryDetails] Error in handlePlay:", e);
  }
};

const handlePause = () => {
  try {
    if (speechStateRef.current === 'playing') {
      syncState('paused');
      if (Platform.OS === 'android') {

```

```

        Speech.stop();
    } else {
        Speech.pause();
    }
}
} catch (e) {
    console.error("[AdvisoryDetails] Error in handlePause:", e);
}
};

```

```

const handleStop = () => {
    try {
        syncState('stopped');
        segmentRef.current = 0;
        Speech.stop();
    } catch (e) {
        console.error("[AdvisoryDetails] Error in handleStop:", e);
    }
};

```

```

const submitFeedback = async (type: 'helpful' | 'not_helpful', comment?: string) => {
    try {
        await fetch(`${API_URL}/feedback`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                message_id: data.message_id,
                feedback_type: type,
                comment: comment || undefined
            })
        });
    } catch (e) {
        console.error('Failed to submit feedback', e);
    }
};

```

```

const serviceLabel = data.detected_service === 'ration_card' ? 'Ration Card Services' :
    data.detected_service === 'aadhaar' ? 'Aadhaar Services' :
    data.detected_service === 'scholarship' ? 'Scholarship Services' :
    'General Inquiry';

```

```

return (
    <View style={styles.detailsArea}>
        {data.requires_official_verification &&& (
            <View
                style={[
                    styles.verificationCard,
                    { backgroundColor: colors.warningBackground },
                ]}
            >
                <Icons name="information-circle-outline" size={20} color={colors.warning} />
                <View style={styles.verificationTextArea}>
                    <Text style={[styles.verificationTitle, { color: colors.warning }]}>
                        Official Verification Required
                    </Text>
                    <Text style={[styles.verificationText, { color: colors.warning }]}>
                        {cleanText(answer?.warning || data.verification_message || "This information requires official verification")}
                    </Text>
                </View>
            </View>
        )}
    </View>
)

```

```

    { /* Service Identified */
    <View style={ [styles.detailCard, { backgroundColor: colors.surface, borderColor:
      <View style={ [styles.detailIcon, { backgroundColor: colors.primarySoft }] }>
      <Ionicons name="search-outline" size={19} color={colors.primaryDark} />
    </View>
    <View style={styles.detailTextArea}>
      <Text style={ [styles.detailTitle, { color: colors.text }] }>Identified Ser
      <Text style={ [styles.detailText, { color: colors.textMuted }] }>{serviceLa
    </View>
    </View>

    { /* Summary */
    {(answer?.summary) &&& (
      <View style={ [styles.detailCard, { backgroundColor: colors.surface, borderColo
      <View style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 12 }}>
      <View style={ [styles.detailIcon, { backgroundColor: colors.primarySoft, ma
      <Ionicons name="chatbubbles-outline" size={19} color={colors.primaryDark}
    </View>
      <Text style={ [styles.detailTitle, { color: colors.text }] }>Advisory Sum
    </View>
      <Text style={ [styles.detailText, { color: colors.text, lineHeight: 20 }] }>
    </View>
    )}

    { /* Documents & Eligibility */
    <View style={ [styles.detailCard, { backgroundColor: colors.surface, borderColor:
      <View style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 12 }}>
      <View style={ [styles.detailIcon, { backgroundColor: colors.primarySoft, ma
      <Ionicons name="documents-outline" size={19} color={colors.primaryDark} />
    </View>
      <Text style={ [styles.detailTitle, { color: colors.text }] }>Documents an
    </View>

    ((!hasDocuments &&& !hasEligibility) ? (
      <Text style={ [styles.detailText, { color: colors.textMuted }] }>The retr
    ) : (
      <View>
        {hasDocuments &&& (
          <View style={{ marginBottom: hasEligibility ? 16 : 0 }}>
            <Text style={ [styles.detailTitle, { color: colors.text, fontSi
            <answer.documents.map((doc, idx) => (
              <View key={`doc-${idx}`} style={{ flexDirection: 'row', ma
                <Text style={{ color: colors.primaryDark, fontSize: 16
                <Text style={{ color: colors.text, flex: 1, fontSize:
              </View>
            )}}
          </View>
        )}
        {hasEligibility &&& (
          <View>
            <Text style={ [styles.detailTitle, { color: colors.text, fontSi
            <answer.eligibility.map((elig, idx) => (
              <View key={`elig-${idx}`} style={{ flexDirection: 'row', m
                <Text style={{ color: colors.primaryDark, fontSize: 16
                <Text style={{ color: colors.text, flex: 1, fontSize:
              </View>
            )}}
          </View>
        )}
      </View>
    )}
  </View>

```

```

    })
    </View>;

{ /* Next Steps & Where to Go */
<View style={ [styles.detailCard, { backgroundColor: colors.surface, borderColor:
    <View style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 12 }}>
        <View style={ [styles.detailIcon, { backgroundColor: colors.primarySoft, ma
            <Ionicons name="navigate-outline" size={19} color={colors.primaryDark} />
        </View>;
        <Text style={ [styles.detailTitle, { color: colors.text } ]}>Recommended
    </View>;

{ (!hasNextSteps & & !hasWhereToGo) ? (
    <Text style={ [styles.detailText, { color: colors.textMuted } ]}>No addit
) : (
    <View>;
        {hasNextSteps & & (
            <View style={{ marginBottom: hasWhereToGo ? 16 : 0 }}>;
                {answer.next_steps.map((step, idx) => (
                    <View key={`step-${idx}`} style={{ flexDirection: 'row', m
                        <Text style={{ color: colors.primaryDark, fontSize: 13
                        <Text style={{ color: colors.text, flex: 1, fontSize:
                    </View>;
                ))}
            </View>;
        )}
        {hasWhereToGo & & (
            <View>;
                <Text style={ [styles.detailTitle, { color: colors.text, fontSi
                {answer.where_to_go.map((loc, idx) => (
                    <View key={`loc-${idx}`} style={{ flexDirection: 'row', ma
                        <Text style={{ color: colors.primaryDark, fontSize: 16
                        <Text style={{ color: colors.text, flex: 1, fontSize:
                    </View>;
                ))}
            </View>;
        )}
    </View>;
    </View>;
    </View>;
    </View>;

<View style={ [styles.sourcesCard, { backgroundColor: colors.surface, borderColor
    <View style={ styles.sourcesHeader}>;
        <Ionicons name="link-outline" size={19} color={colors.primaryDark} />;
        <Text style={ [styles.sourcesTitle, { color: colors.text } ]}>;
        Sources & Confidence ({cleanText(data.confidence_score ? (data.confidence
    </Text>;
    </View>;
    <View style={{ marginTop: 8 }}>;
        {data.citations & & data.citations.length > 0 ? (
            data.citations.map((c, i) => (
                <Pressable
                    key={i}
                    onPress={() => { if (c.source_url) Linking.openURL(c.source_url); }}
                    style={({ pressed }) => [{ marginBottom: 6, opacity: pressed ? 0.7 : 1
                >;
                <Text style={ [styles.sourcesText, { color: colors.primary, textDecorat
                    {cleanText(c.source_title)} ({cleanText(c.authority)})}
                </Text>;
                </Pressable>;
            )}
        ) : (

```

```

        <Text style={styles.sourcesText, { color: colors.textMuted }}>
            No specific sources linked.
        </Text>
    )}
    </View>
</View>

<View style={styles.audioRow}>
    <AudioButton icon="play" label={speechState === 'paused' ? 'Resume' : 'Play audio'} />
    <AudioButton icon="pause" label="Pause" onPress={handlePause} />
    <AudioButton icon="stop" label="Stop" onPress={handleStop} />
</View>

<View style={styles.feedbackCard, { backgroundColor: colors.surface, borderColor: colors.textMuted }}>
    {feedbackState === 'initial' && (
        <>
            <Text style={styles.feedbackTitle, { color: colors.text }}>Was this helpful?</Text>
            <View style={styles.feedbackButtons}>
                <FeedbackButton
                    icon="thumbs-up-outline"
                    label="Helpful"
                    onPress={() => {
                        setFeedbackState('helpful-submitted');
                        submitFeedback('helpful');
                    }}
                    color={colors.primaryDark}
                    bg={colors.primarySoft}
                />
                <FeedbackButton
                    icon="thumbs-down-outline"
                    label="Not helpful"
                    onPress={() => setFeedbackState('needs-improvement')}
                    color={colors.textMuted}
                    bg={colors.surfaceMuted}
                />
            </View>
        </>
    )}
    {feedbackState === 'needs-improvement' && (
        <>
            <Text style={styles.feedbackTitle, { color: colors.text }}>What could we improve?</Text>
            <TextInput
                style={
                    [
                        styles.feedbackInput,
                        { backgroundColor: colors.background, color: colors.text, borderColor: colors.textMuted }
                    ]
                }
                placeholder="Tell us what was missing..."
                placeholderTextColor={colors.textMuted}
                value={feedbackComment}
                onChangeText={setFeedbackComment}
                multiline
            />
            <View style={styles.feedbackButtons}>
                <FeedbackButton
                    icon="close-outline"
                    label="Cancel"
                    onPress={() => setFeedbackState('initial')}
                    color={colors.textMuted}
                    bg={colors.surfaceMuted}
                />
                <FeedbackButton
                    icon="send-outline"

```

```

        label="Submit"
        onPress={() => {
            setFeedbackState('not-helpful-submitted');
            submitFeedback('not_helpful', feedbackComment);
        }}
        color={colors.primaryDark}
        bg={colors.primarySoft}
    />
    </View>
    </>
  )}
  {(feedbackState === 'helpful-submitted' || feedbackState === 'not-helpful-submitted') ? (
    <View style={styles.feedbackSubmitted}>
      <Ionicons name="checkmark-circle-outline" size={20} color={colors.primaryDark} />
      <Text style={styles.feedbackTitle, { color: colors.primaryDark, marginTop: 10}}>
        Thank you for your feedback
      </Text>
    </View>
  ) : (
    <View style={styles.feedbackSubmitted} />
  )}
  </View>
  </View>
);
}

```

```

function AudioButton({
  icon,
  label,
  onPress,
}: {
  icon: "play" | "pause" | "stop";
  label: string;
  onPress?: () => void;
}) {
  const { colors } = useTheme();

  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      accessibilityLabel={label}
      style={({ pressed }) => [
        styles.audioButton,
        {
          backgroundColor: colors.surface,
          borderColor: colors.borderStrong,
          opacity: pressed ? 0.75 : 1,
        },
      ]
    />
    <Ionicons
      name={`_${icon}-outline` as keyof typeof Ionicons.glyphMap}
      size={16}
      color={colors.primaryDark}
    />
    <Text
      style={[
        styles.audioButtonText,
        {
          color: colors.primaryDark,
        },
      ]}
    />
  );
}

```

```

        },
      ]}
    >
    {label}
    </Text>
    </Pressable>
  );
}

function FeedbackButton({
  icon,
  label,
  onPress,
  color,
  bg
}: {
  icon: keyof typeof Icons.glyphMap;
  label: string;
  onPress: () => void;
  color: string;
  bg: string;
}) {
  const { colors } = useTheme();

  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      accessibilityLabel={label}
      style={({ pressed }) => [
        styles.feedbackButton,
        {
          backgroundColor: bg,
          borderColor: colors.border,
          opacity: pressed ? 0.75 : 1,
        },
      ]}
    >
      <Icons
        name={icon}
        size={17}
        color={color}
      />

      <Text
        style={[
          styles.feedbackButtonText,
          {
            color: colors.text,
          },
        ]}
      >
        {label}
      </Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  feedbackSubmitted: {
    flexDirection: 'row',
    alignItems: 'center',
  },
});

```

```
        justifyContent: 'center',
        paddingVertical: 12,
        gap: 6,
    },
    safeArea: {
        flex: 1,
    },

    keyboardView: {
        flex: 1,
    },

    header: {
        borderBottomWidth: 1,
    },

    headerInner: {
        alignItems: "center",
        alignSelf: "center",
        flexDirection: "row",
        minHeight: 70,
        paddingHorizontal: 14,
        width: "100%",
    },

    desktopHeaderInner: {
        maxWidth: 760,
    },

    headerButton: {
        alignItems: "center",
        borderRadius: 12,
        height: 43,
        justifyContent: "center",
        width: 43,
    },

    brandButton: {
        alignItems: "center",
        flex: 1,
        flexDirection: "row",
        gap: 9,
        minWidth: 0,
        paddingHorizontal: 10,
    },

    brandLogoContainer: {
        alignItems: "center",
        borderRadius: 11,
        borderWidth: 1,
        height: 43,
        justifyContent: "center",
        overflow: "hidden",
        width: 43,
    },

    brandLogoImage: {
        height: 38,
        width: 38,
    },

    brandTextArea: {
```

```
    flex: 1,
  },

  brandTitle: {
    fontFamily: "Manrope_700Bold",
    fontSize: 15,
  },

  brandSubtitle: {
    fontFamily: "Manrope_400Regular",
    fontSize: 10,
    marginTop: 2,
  },

  content: {
    alignSelf: "center",
    maxWidth: 540,
    padding: 17,
    paddingBottom: 28,
    width: "100%",
  },

  desktopContent: {
    maxWidth: 700,
  },

  welcomeArea: {
    alignItems: "center",
    paddingTop: 25,
  },

  heroLogoContainer: {
    alignItems: "center",
    borderRadius: 55,
    borderWidth: 1,
    height: 110,
    justifyContent: "center",
    overflow: "hidden",
    width: 110,
  },

  heroLogo: {
    height: 96,
    width: 96,
  },

  welcomeTitle: {
    fontFamily: "Manrope_800ExtraBold",
    fontSize: 28,
    letterSpacing: -0.5,
    marginTop: 19,
    textAlign: "center",
  },

  welcomeSubtitle: {
    fontFamily: "Manrope_400Regular",
    fontSize: 14,
    lineHeight: 22,
    marginTop: 9,
    maxWidth: 480,
    textAlign: "center",
  },
```

```
assuranceCard: {
  alignItems: "flex-start",
  borderRadius: 14,
  flexDirection: "row",
  gap: 9,
  marginTop: 21,
  maxWidth: 520,
  padding: 13,
},

assuranceText: {
  flex: 1,
  fontFamily: "Manrope_600SemiBold",
  fontSize: 11,
  lineHeight: 18,
},

suggestionHeading: {
  alignSelf: "flex-start",
  fontFamily: "Manrope_700Bold",
  fontSize: 15,
  marginBottom: 11,
  marginTop: 28,
},

suggestions: {
  alignSelf: "stretch",
  gap: 10,
},

suggestionCard: {
  alignItems: "center",
  borderRadius: 16,
  borderWidth: 1,
  flexDirection: "row",
  gap: 10,
  minHeight: 66,
  paddingHorizontal: 12,
  paddingVertical: 10,
},

suggestionIcon: {
  alignItems: "center",
  borderRadius: 10,
  height: 37,
  justifyContent: "center",
  width: 37,
},

suggestionText: {
  flex: 1,
  fontFamily: "Manrope_500Medium",
  fontSize: 12,
  lineHeight: 19,
},

messageRow: {
  alignItems: "flex-end",
  flexDirection: "row",
  gap: 8,
  marginBottom: 14,
},
```

```
userRow: {
  justifyContent: "flex-end",
},

assistantRow: {
  justifyContent: "flex-start",
},

assistantAvatar: {
  alignItems: "center",
  borderRadius: 16,
  height: 31,
  justifyContent: "center",
  width: 31,
},

messageBubble: {
  borderRadius: 18,
  borderWidth: 1,
  maxWidth: "84%",
  paddingHorizontal: 15,
  paddingVertical: 13,
},

userBubble: {
  borderBottomRightRadius: 4,
},

assistantBubble: {
  borderBottomLeftRadius: 4,
},

messageText: {
  fontFamily: "Manrope_400Regular",
  fontSize: 13,
  lineHeight: 21,
},

typingCard: {
  borderBottomLeftRadius: 4,
  borderRadius: 16,
  borderWidth: 1,
  minWidth: 220,
  padding: 13,
},

typingText: {
  fontFamily: "Manrope_600SemiBold",
  fontSize: 12,
},

dots: {
  flexDirection: "row",
  gap: 5,
  marginTop: 9,
},

dot: {
  borderRadius: 4,
  height: 7,
  width: 7,
},
```

```
detailsArea: {
  marginTop: 9,
},

verificationCard: {
  alignItems: "flex-start",
  borderRadius: 16,
  flexDirection: "row",
  gap: 10,
  marginBottom: 10,
  padding: 14,
},

verificationTextArea: {
  flex: 1,
},

verificationTitle: {
  fontFamily: "Manrope_700Bold",
  fontSize: 12,
},

verificationText: {
  fontFamily: "Manrope_400Regular",
  fontSize: 11,
  lineHeight: 18,
  marginTop: 4,
},

detailCard: {
  alignItems: "flex-start",
  borderRadius: 16,
  borderWidth: 1,
  flexDirection: "row",
  gap: 10,
  marginBottom: 10,
  padding: 14,
},

detailIcon: {
  alignItems: "center",
  borderRadius: 10,
  height: 39,
  justifyContent: "center",
  width: 39,
},

audioRow: {
  flexDirection: "row",
  gap: 12,
  marginTop: 10,
  marginBottom: 6,
},

detailTextArea: {
  flex: 1,
},

detailTitle: {
  fontFamily: "Manrope_700Bold",
  fontSize: 12,
},
```

```
detailText: {
  fontFamily: "Manrope_400Regular",
  fontSize: 11,
  lineHeight: 18,
  marginTop: 4,
},

sourcesCard: {
  borderRadius: 16,
  borderWidth: 1,
  marginTop: 2,
  padding: 14,
},

sourcesHeader: {
  alignItems: "center",
  flexDirection: "row",
  gap: 7,
},

sourcesTitle: {
  fontFamily: "Manrope_700Bold",
  fontSize: 12,
},

sourcesText: {
  fontFamily: "Manrope_400Regular",
  fontSize: 11,
  lineHeight: 18,
  marginTop: 8,
},

audioCard: {
  borderRadius: 16,
  marginTop: 11,
  padding: 14,
},

audioTopRow: {
  alignItems: "center",
  flexDirection: "row",
  gap: 10,
},

audioIcon: {
  alignItems: "center",
  borderRadius: 10,
  height: 39,
  justifyContent: "center",
  width: 39,
},

audioTextArea: {
  flex: 1,
},

audioTitle: {
  fontFamily: "Manrope_700Bold",
  fontSize: 12,
},

audioText: {
```

```
    fontFamily: "Manrope_400Regular",
    fontSize: 11,
    lineHeight: 17,
    marginTop: 3,
  },

  audioButtons: {
    flexDirection: "row",
    gap: 8,
    marginTop: 12,
  },

  audioButton: {
    alignItems: "center",
    borderRadius: 9,
    borderWidth: 1,
    flexDirection: "row",
    gap: 5,
    paddingHorizontal: 10,
    paddingVertical: 8,
  },

  audioButtonText: {
    fontFamily: "Manrope_600SemiBold",
    fontSize: 10,
  },

  feedbackCard: {
    borderRadius: 16,
    borderWidth: 1,
    marginTop: 11,
    padding: 14,
  },

  feedbackTitle: {
    fontFamily: "Manrope_700Bold",
    fontSize: 12,
  },

  feedbackButtons: {
    flexDirection: "row",
    flexWrap: "wrap",
    gap: 8,
    marginTop: 11,
  },

  feedbackButton: {
    alignItems: "center",
    borderRadius: 9,
    borderWidth: 1,
    flexDirection: "row",
    gap: 5,
    paddingHorizontal: 10,
    paddingVertical: 8,
  },

  feedbackButtonText: {
    fontFamily: "Manrope_600SemiBold",
    fontSize: 10,
  },

  feedbackInput: {
```

```

        borderWidth: 1,
        borderRadius: 8,
        padding: 10,
        fontSize: 12,
        fontFamily: "Manrope_500Medium",
        minHeight: 60,
        textAlignVertical: "top",
    },

    feedbackSubmitButton: {
        alignItems: "center",
        justifyContent: "center",
        borderRadius: 8,
        paddingVertical: 10,
        marginTop: 5,
    },

    composerArea: {
        alignItems: "center",
        paddingHorizontal: 14,
        paddingBottom: 24, // Added bottom padding to ensure it floats nicely
    },

    composerWrapper: {
        width: "100%",
    },

    desktopComposerWrapper: {
        maxWidth: 700,
    },

    drawerOverlay: {
        flex: 1,
        flexDirection: "row",
    },

    drawerPanel: {
        maxWidth: 340,
        paddingTop: 48,
        width: "86%",
    },

    closeButton: {
        alignItems: "center",
        borderRadius: 11,
        height: 40,
        justifyContent: "center",
        position: "absolute",
        right: 11,
        top: 5,
        width: 40,
        zIndex: 3,
    },

    drawerDismissArea: {
        flex: 1,
    },

    heroLogoWrapper: {
        alignItems: "center",
        borderRadius: 28,
        height: 80,
    },

```

```

        justifyContent: "center",
        marginBottom: 16,
        width: 80,
        overflow: "hidden",
    },

    heroLogoIcon: {
        height: "100%",
        width: "100%",
    },

    assistantAvatarImageContainer: {
        alignItems: "center",
        borderRadius: 12,
        height: 31,
        justifyContent: "center",
        width: 31,
        overflow: "hidden",
    },

    assistantAvatarImage: {
        height: "100%",
        width: "100%",
    },
});

```

File: src\app\mvp-info.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
    Pressable,
    SafeAreaView,
    ScrollView,
    StyleSheet,
    Text,
    View,
} from "react-native";

import ServiceCategoryCard from "../components/ServiceCategoryCard";
import { SERVICE_CATEGORIES } from "../constants/services";
import { useTheme } from "../contexts/ThemeContext";

const workflowSteps = [
    {
        number: "01",
        icon: "chatbubble-ellipses-outline" as const,
        title: "Ask your question",
        text: "Type an English question about a selected citizen-service situation.",
    },
    {
        number: "02",
        icon: "search-outline" as const,
        title: "Find official evidence",
        text: "The backend will retrieve relevant information from curated official sources."
    },
    {
        number: "03",
        icon: "shield-checkmark-outline" as const,
        title: "Review the guidance",
    }
];

```

```

    text: "The answer will show supporting sources, confidence, and verification warnings
  },
];

```

```

export default function MvpInfoScreen() {
  const { colors } = useTheme();

  return (
    <SafeAreaView
      style={[
        styles.safeArea,
        {
          backgroundColor: colors.background,
        },
      ]}
    >
      <ScrollView
        contentContainerStyle={styles.content}
        showsVerticalScrollIndicator={false}
      >
        <View style={styles.headerRow}>
          <Pressable
            onPress={() => router.canGoBack() ? router.back() : router.replace('/') }
            accessibilityRole="button"
            accessibilityLabel="Go back"
            style={styles.backButton}
          >
            <Ionicons
              name="arrow-back"
              size={22}
              color={colors.text}
            />
          </Pressable>

          <View style={styles.headerText}>
            <Text
              style={[
                styles.eyebrow,
                {
                  color: colors.primary,
                },
              ]}
            >
              ENGLISH MVP SCOPE
            </Text>

            <Text
              style={[
                styles.title,
                {
                  color: colors.text,
                },
              ]}
            >
              What can Akshaya Advisory help with?
            </Text>

            <Text
              style={[
                styles.subtitle,
                {
                  color: colors.textMuted,

```

```

        },
    ]}
    &gt;
    Select a service area to begin an advisory conversation.
    &lt;/Text&gt;
    &lt;/View&gt;
    &lt;/View&gt;

    &lt;View
    style={[
        styles.statusCard,
        {
            backgroundColor: colors.primarySoft,
        },
    ]}
    &gt;
    &lt;View
    style={[
        styles.statusIcon,
        {
            backgroundColor: colors.primary,
        },
    ]}
    &gt;
    &lt;Ionicons
    name="language-outline"
    size={21}
    color="#FFFFFF"
    /&gt;
    &lt;/View&gt;

    &lt;View style={styles.statusContent}&gt;
    &lt;Text
    style={[
        styles.statusTitle,
        {
            color: colors.text,
        },
    ]}
    &gt;
    English-first advisory mode
    &lt;/Text&gt;

    &lt;Text
    style={[
        styles.statusText,
        {
            color: colors.textMuted,
        },
    ]}
    &gt;
    The first version is designed and tested in English. Malayalam
    support will be considered only after quality evaluation.
    &lt;/Text&gt;
    &lt;/View&gt;
    &lt;/View&gt;

    &lt;Text
    style={[
        styles.sectionTitle,
        {
            color: colors.text,

```

```

    },
  ]}
  &gt;
  Supported service areas
  &lt;/Text&gt;

  &lt;Text
    style={[
      styles.sectionSubtitle,
      {
        color: colors.textMuted,
      },
    ]}
  &gt;
  Choose the topic closest to your situation.
  &lt;/Text&gt;

  &lt;View style={styles.serviceGrid}&gt;
    {SERVICE_CATEGORIES.map((service) => (
      &lt;ServiceCategoryCard
        key={service.id}
        category={service.id}
        title={service.title}
        description={service.description}
        icon={service.icon}
      /&gt;
    ))}
  &lt;/View&gt;

  &lt;View style={styles.workflowHeader}&gt;
    &lt;Text
      style={[
        styles.sectionTitle,
        {
          color: colors.text,
        },
      ]}
    &gt;
    How the advisory process works
    &lt;/Text&gt;

    &lt;Text
      style={[
        styles.sectionSubtitle,
        {
          color: colors.textMuted,
        },
      ]}
    &gt;
    The final system will use a controlled agentic RAG workflow.
    &lt;/Text&gt;
  &lt;/View&gt;

  &lt;View style={styles.workflowList}&gt;
    {workflowSteps.map((step) => (
      &lt;View
        key={step.number}
        style={[
          styles.workflowCard,
          {
            backgroundColor: colors.surface,
            borderColor: colors.border,

```

```

    },
  ]}
<
  <View
    style=[
      styles.stepNumber,
      {
        backgroundColor: colors.primarySoft,
      },
    ]}
  <
  <Text
    style=[
      styles.stepNumberText,
      {
        color: colors.primary,
      },
    ]}
  <
  {step.number}
  </Text>
</View>

  <View
    style=[
      styles.workflowIcon,
      {
        backgroundColor: colors.surfaceMuted,
      },
    ]}
  <
  <Ionicons
    name={step.icon}
    size={20}
    color={colors.primary}
  />
</View>

  <View style={styles.workflowContent}>
    <Text
      style=[
        styles.workflowTitle,
        {
          color: colors.text,
        },
      ]}
    <
    {step.title}
    </Text>

    <Text
      style=[
        styles.workflowText,
        {
          color: colors.textMuted,
        },
      ]}
    <
    {step.text}
    </Text>
  </View>
</View>

```

```

    )})
</View>

<View
  style=[
    styles.capabilityCard,
    {
      backgroundColor: colors.surface,
      borderColor: colors.border,
    },
  ]
  >
  <View style={styles.capabilityHeader}>
    <Ionicons
      name="checkmark-circle-outline"
      size={21}
      color={colors.accent}
    />

    <Text
      style=[
        styles.capabilityTitle,
        {
          color: colors.text,
        },
      ]
    >
      The application can provide
    </Text>
  </View>

  <CapabilityItem
    text="Possible service identification"
    colors={colors}
  />

  <CapabilityItem
    text="Eligibility information when supported"
    colors={colors}
  />

  <CapabilityItem
    text="Document checklist guidance from official sources"
    colors={colors}
  />

  <CapabilityItem
    text="Step-by-step advisory recommendations"
    colors={colors}
  />

  <CapabilityItem
    text="Citations, confidence status, and verification warnings"
    colors={colors}
  />

  <CapabilityItem
    text="English text-to-speech for the final response"
    colors={colors}
  />
</View>

```

```

<View
  style={[
    styles.limitationsCard,
    {
      backgroundColor: colors.warningBackground,
    },
  ]}
>
  <View style={styles.limitationsHeader}>
    <Ionicons
      name="alert-circle-outline"
      size={21}
      color={colors.warning}
    />

    <Text
      style={[
        styles.limitationsTitle,
        {
          color: colors.warning,
        },
      ]}
    >
      Important limitations
    </Text>
  </View>

  <Text
    style={[
      styles.limitationsText,
      {
        color: colors.warning,
      },
    ]}
  >
    This application is advisory only. It does not approve
    applications, submit applications automatically, perform biometric
    verification, access private Aadhaar data, verify document
    authenticity, confirm official identity, or replace Akshaya
    operators and government departments.
  </Text>
</View>

<Pressable
  onPress={() => router.push("/chat")}
  accessibilityRole="button"
  accessibilityLabel="Start an advisory chat"
  style={({ pressed }) => [
    styles.startButton,
    {
      backgroundColor: colors.primary,
      opacity: pressed ? 0.78 : 1,
    },
  ]}
>
  <Ionicons
    name="chatbubble-ellipses-outline"
    size={20}
    color="#FFFFFF"
  />

  <Text style={styles.startButtonText}>

```

```

        Start an advisory chat
        </Text>

        <Ionicons
            name="arrow-forward"
            size={19}
            color="#FFFFFF"
        />
        </Pressable>
    </ScrollView>
    </SafeAreaView>
  );
}

function CapabilityItem({
  text,
  colors,
}: {
  text: string;
  colors: any;
}) {
  return (
    <View style={styles.capabilityItem}>
      <Ionicons
        name="checkmark"
        size={16}
        color={colors.accent}
      />

      <Text
        style={[
          styles.capabilityText,
          {
            color: colors.textMuted,
          },
        ]}
      >
        {text}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },

  content: {
    alignSelf: "center",
    maxWidth: 1000,
    padding: 22,
    paddingBottom: 55,
    width: "100%",
  },

  headerRow: {
    alignItems: "flex-start",
    flexDirection: "row",
    gap: 10,
  },

```

```
backButton: {
  alignItems: "center",
  height: 40,
  justifyContent: "center",
  width: 40,
},

headerText: {
  flex: 1,
},

eyebrow: {
  fontSize: 10,
  fontWeight: "800",
  letterSpacing: 1.1,
  marginBottom: 9,
},

title: {
  fontSize: 30,
  fontWeight: "800",
  letterSpacing: -0.5,
  lineHeight: 37,
},

subtitle: {
  fontSize: 15,
  lineHeight: 23,
  marginTop: 10,
},

statusCard: {
  alignItems: "flex-start",
  borderRadius: 17,
  flexDirection: "row",
  gap: 12,
  marginTop: 25,
  padding: 16,
},

statusIcon: {
  alignItems: "center",
  borderRadius: 12,
  height: 42,
  justifyContent: "center",
  width: 42,
},

statusContent: {
  flex: 1,
},

statusTitle: {
  fontSize: 14,
  fontWeight: "800",
},

statusText: {
  fontSize: 13,
  lineHeight: 20,
  marginTop: 5,
},
```

```
sectionTitle: {
  fontSize: 20,
  fontWeight: "800",
  marginTop: 31,
},

sectionSubtitle: {
  fontSize: 13,
  lineHeight: 19,
  marginTop: 5,
},

serviceGrid: {
  alignItems: "stretch",
  flexDirection: "row",
  flexWrap: "wrap",
  gap: 12,
  marginTop: 16,
},

workflowHeader: {
  marginTop: 22,
},

workflowList: {
  gap: 10,
  marginTop: 16,
},

workflowCard: {
  alignItems: "center",
  borderRadius: 16,
  borderWidth: 1,
  flexDirection: "row",
  gap: 11,
  padding: 14,
},

stepNumber: {
  alignItems: "center",
  borderRadius: 10,
  height: 38,
  justifyContent: "center",
  width: 38,
},

stepNumberText: {
  fontSize: 11,
  fontWeight: "800",
},

workflowIcon: {
  alignItems: "center",
  borderRadius: 10,
  height: 38,
  justifyContent: "center",
  width: 38,
},

workflowContent: {
  flex: 1,
},
```

```
workflowTitle: {
  fontSize: 14,
  fontWeight: "800",
},

workflowText: {
  fontSize: 12,
  lineHeight: 18,
  marginTop: 4,
},

capabilityCard: {
  borderRadius: 17,
  borderWidth: 1,
  marginTop: 24,
  padding: 17,
},

capabilityHeader: {
  alignItems: "center",
  flexDirection: "row",
  gap: 8,
  marginBottom: 13,
},

capabilityTitle: {
  fontSize: 15,
  fontWeight: "800",
},

capabilityItem: {
  alignItems: "flex-start",
  flexDirection: "row",
  gap: 9,
  marginBottom: 9,
},

capabilityText: {
  flex: 1,
  fontSize: 13,
  lineHeight: 19,
},

limitationsCard: {
  borderRadius: 17,
  marginTop: 12,
  padding: 17,
},

limitationsHeader: {
  alignItems: "center",
  flexDirection: "row",
  gap: 8,
  marginBottom: 8,
},

limitationsTitle: {
  fontSize: 15,
  fontWeight: "800",
},

limitationsText: {
```

```

        fontSize: 13,
        lineHeight: 20,
      },

      startButton: {
        alignItems: "center",
        borderRadius: 13,
        flexDirection: "row",
        gap: 9,
        justifyContent: "center",
        marginTop: 20,
        padding: 16,
      },

      startButtonText: {
        color: "#FFFFFF",
        flex: 1,
        fontSize: 15,
        fontWeight: "800",
      },
    },
  });

```

File: src\app\settings.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
  Pressable,
  SafeAreaView,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { useTheme } from "../contexts/ThemeContext";

export default function SettingsScreen() {
  const { colors } = useTheme();

  return (
    <SafeAreaView
      style={[
        styles.safeArea,
        {
          backgroundColor: colors.background,
        }
      ]}
    >
      <ScrollView
        contentContainerStyle={styles.content}
        showsVerticalScrollIndicator={false}
      >
        <View style={styles.headerRow}>
          <Pressable
            onPress={() => router.canGoBack() ? router.back() : router.replace('/') }
            accessibilityRole="button"
            accessibilityLabel="Go back"
            style={styles.backButton}
          >

```

```

        <Ionicons name="arrow-back" size={22} color={colors.text} />
      </Pressable>

    </View>
    <Text
      style={[
        styles.headerTitle,
        {
          color: colors.text,
        },
      ]}
    >
      Settings and preferences
    </Text>

    <Text
      style={[
        styles.headerSubtitle,
        {
          color: colors.textMuted,
        },
      ]}
    >
      Configure your chat experience and interface settings
    </Text>
  </View>
</View>

<View
  style={[
    styles.profileCard,
    {
      backgroundColor: colors.surface,
      borderColor: colors.border,
    },
  ]}
>
  <View
    style={[
      styles.avatar,
      {
        backgroundColor: colors.primary,
      },
    ]}
  >
    <Ionicons
      name="color-palette-outline"
      size={28}
      color="#FFFFFF"
    />
  </View>

  <View style={styles.profileText}>
    <Text
      style={[
        styles.profileName,
        {
          color: colors.text,
        },
      ]}
    >
      Ocean Blue Theme
    </Text>
  </View>

```

```

    </Text>

    <Text
      style=[
        styles.profileSubtitle,
        {
          color: colors.textMuted,
        },
      ]
    >
      Default mobile-first design applied.
    </Text>
  </View>
</View>

<InfoCard
  colors={colors}
  title="Chat Preferences"
  icon="chatbubble-ellipses-outline"
  accent
  rows=[
    {
      icon: "notifications-outline",
      title: "Notifications",
      text: "Push notifications are currently disabled for the MVP phase.",
    },
    {
      icon: "trash-outline",
      title: "History Management",
      text: "Chat history can be cleared on a per-conversation basis from the his
    },
    {
      icon: "text-outline",
      title: "Text size",
      text: "The application scales dynamically with your device's native accessi
    },
  ]
/>

<InfoCard
  colors={colors}
  title="Voice and Audio"
  icon="accessibility-outline"
  rows=[
    {
      icon: "language-outline",
      title: "Language Support",
      text: "The initial version is designed for typed English queries and verifi
    },
    {
      icon: "mic-outline",
      title: "Voice typing",
      text: "You can use the voice typing feature provided by your mobile keyboar
    },
    {
      icon: "volume-medium-outline",
      title: "Listen feature",
      text: "English text-to-speech will read verified final responses after back
    },
  ]
/>

```

```

        <View
            style={
                styles.disclaimer,
                {
                    backgroundColor: colors.warningBackground,
                },
            }
        >
        <Ionicons
            name="alert-circle-outline"
            size={21}
            color={colors.warning}
        />

        <Text
            style={
                styles.disclaimerText,
                {
                    color: colors.warning,
                },
            }
        >
            This is an advisory application. Always confirm important service
            requirements with an Akshaya operator or the responsible official
            department.
        </Text>
    </View>
</ScrollView>
</SafeAreaView>
);
}

function InfoCard({
    colors,
    title,
    icon,
    rows,
    accent = false,
}): {
    colors: any;
    title: string;
    icon: keyof typeof Ionicons.glyphMap;
    accent?: boolean;
    rows: {
        icon: keyof typeof Ionicons.glyphMap;
        title: string;
        text: string;
    }[];
}) {
    return (
        <View
            style={
                styles.infoCard,
                {
                    backgroundColor: colors.surface,
                    borderColor: colors.border,
                },
            }
        >
            <View style={styles.infoHeader}>
                <View
                    style={

```

```

        styles.infoHeaderIcon,
        {
            backgroundColor: accent
                ? colors.primarySoft
                : colors.surfaceMuted,
        },
    ]}
    &gt;
    &lt;Ionicons
        name={icon}
        size={20}
        color={colors.primaryDark}
    /&gt;
    &lt;/View&gt;

    &lt;Text
        style=[
            styles.infoHeaderTitle,
            {
                color: colors.text,
            },
        ]}
    &gt;
        {title}
    &lt;/Text&gt;
    &lt;/View&gt;

    {rows.map((row) => (
        &lt;View key={row.title} style={styles.infoRow}&gt;
            &lt;Ionicons
                name={row.icon}
                size={17}
                color={colors.primaryDark}
            /&gt;

            &lt;View style={styles.infoRowTextArea}&gt;
                &lt;Text
                    style=[
                        styles.infoRowTitle,
                        {
                            color: colors.text,
                        },
                    ]}
                &gt;
                    {row.title}
                &lt;/Text&gt;

                &lt;Text
                    style=[
                        styles.infoRowText,
                        {
                            color: colors.textMuted,
                        },
                    ]}
                &gt;
                    {row.text}
                &lt;/Text&gt;
            &lt;/View&gt;
        &lt;/View&gt;
    ))}
    &lt;/View&gt;
);

```

```

}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },

  content: {
    alignSelf: "center",
    maxWidth: 760,
    padding: 17,
    paddingBottom: 42,
    width: "100%",
  },

  headerRow: {
    alignItems: "flex-start",
    flexDirection: "row",
    gap: 9,
  },

  backButton: {
    alignItems: "center",
    height: 40,
    justifyContent: "center",
    width: 40,
  },

  headerTitle: {
    fontSize: 20,
    fontWeight: "800",
    marginTop: 2,
  },

  headerSubtitle: {
    fontSize: 11,
    lineHeight: 16,
    marginTop: 4,
  },

  profileCard: {
    alignItems: "center",
    borderRadius: 18,
    borderWidth: 1,
    flexDirection: "row",
    gap: 13,
    marginTop: 22,
    padding: 16,
  },

  avatar: {
    alignItems: "center",
    borderRadius: 18,
    height: 56,
    justifyContent: "center",
    width: 56,
  },

  profileText: {
    flex: 1,
  },

```

```
profileName: {
  fontSize: 17,
  fontWeight: "800",
},

profileSubtitle: {
  fontSize: 12,
  lineHeight: 18,
  marginTop: 4,
},

infoCard: {
  borderRadius: 18,
  borderWidth: 1,
  marginTop: 12,
  padding: 16,
},

infoHeader: {
  alignItems: "center",
  flexDirection: "row",
  gap: 9,
  marginBottom: 15,
},

infoHeaderIcon: {
  alignItems: "center",
  borderRadius: 10,
  height: 38,
  justifyContent: "center",
  width: 38,
},

infoHeaderTitle: {
  fontSize: 15,
  fontWeight: "800",
},

infoRow: {
  alignItems: "flex-start",
  flexDirection: "row",
  gap: 10,
  marginBottom: 14,
},

infoRowTextArea: {
  flex: 1,
},

infoRowTitle: {
  fontSize: 12,
  fontWeight: "800",
},

infoRowText: {
  fontSize: 11,
  lineHeight: 17,
  marginTop: 3,
},

disclaimer: {
  alignItems: "flex-start",
```

```

        borderRadius: 18,
        flexDirection: "row",
        gap: 9,
        marginTop: 12,
        padding: 15,
      },
      disclaimerText: {
        flex: 1,
        fontSize: 11,
        fontWeight: "600",
        lineHeight: 17,
      },
    }
  });

```

File: src\app_layout.tsx

```

import { Stack } from "expo-router";
import { StatusBar } from "expo-status-bar";
import { useFonts } from "expo-font";
import {
  Manrope_400Regular,
  Manrope_500Medium,
  Manrope_600SemiBold,
  Manrope_700Bold,
  Manrope_800ExtraBold,
} from "@expo-google-fonts/manrope";

import { ThemeProvider } from "../contexts/ThemeContext";

export default function RootLayout() {
  const [fontsLoaded] = useFonts({
    Manrope_400Regular,
    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
  });

  if (!fontsLoaded) {
    return null;
  }

  return (
    <ThemeProvider>
      <StatusBar style="dark" />
      <Stack screenOptions={{ headerShown: false }} />
      </ThemeProvider>
    );
}

```

File: src\components\animated-icon.module.css

```

.expoLogoBackground {
  background-image: linear-gradient(180deg, #3c9ffe, #0274df);
  border-radius: 40px;
  width: 128px;
}

```

```

    height: 128px;
  }

```

File: src\components\animated-icon.tsx

```

import { Image } from 'expo-image';
import * as SplashScreen from 'expo-splash-screen';
import { useState } from 'react';
import { Dimensions, StyleSheet, View } from 'react-native';
import Animated, { Easing, Keyframe } from 'react-native-reanimated';
import { scheduleOnRN } from 'react-native-worklets';

const INITIAL_SCALE_FACTOR = Dimensions.get('screen').height / 90;
const DURATION = 600;

export function AnimatedSplashOverlay() {
  const [animate, setAnimate] = useState(false);
  const [visible, setVisible] = useState(true);

  if (!visible) return null;

  const splashKeyframe = new Keyframe({
    0: {
      transform: [{ scale: 1 }],
      opacity: 1,
    },
    20: {
      opacity: 1,
    },
    70: {
      opacity: 0,
      easing: Easing.elastic(0.7),
    },
    100: {
      opacity: 0,
      transform: [{ scale: 1 }],
      easing: Easing.elastic(0.7),
    },
  });

  const image = <Image style={styles.image} source={require('@assets/images/expo-logo')} />;

  return animate ? (
    <Animated.View
      entering={splashKeyframe.duration(DURATION).withCallback((finished) => {
        'worklet';
        if (finished) {
          scheduleOnRN(setVisible, false);
        }
      })}
      style={styles.splashOverlay}>
      {image}
    </Animated.View>
  ) : (
    <View
      onLayout={() => {
        SplashScreen.hideAsync().finally(() => {
          setAnimate(true);
        });
      }}
    />
  );
}

```

```

        style={styles.splashOverlay}&gt;
        {image}
      </View>
    );
  }

  const keyframe = new Keyframe({
    0: {
      transform: [{ scale: INITIAL_SCALE_FACTOR }],
    },
    100: {
      transform: [{ scale: 1 }],
      easing: Easing.elastic(0.7),
    },
  });

  const logoKeyframe = new Keyframe({
    0: {
      transform: [{ scale: 1.3 }],
      opacity: 0,
    },
    40: {
      transform: [{ scale: 1.3 }],
      opacity: 0,
      easing: Easing.elastic(0.7),
    },
    100: {
      opacity: 1,
      transform: [{ scale: 1 }],
      easing: Easing.elastic(0.7),
    },
  });

  const glowKeyframe = new Keyframe({
    0: {
      transform: [{ rotateZ: '0deg' }],
    },
    100: {
      transform: [{ rotateZ: '720deg' }],
    },
  });

  export function AnimatedIcon() {
    return (
      <View style={styles.iconContainer}&gt;
        <Animated.View entering={glowKeyframe.duration(60 * 1000 * 4)} style={styles.glow}>
          <Image style={styles.glow} source={require('@assets/images/logo-glow.png')} />
        </Animated.View>

        <Animated.View entering={keyframe.duration(DURATION)} style={styles.background}>
          <Animated.View style={styles.imageContainer} entering={logoKeyframe.duration(DURATION)}>
            <Image style={styles.image} source={require('@assets/images/expo-logo.png')} />
          </Animated.View>
        </Animated.View>
      </View>
    );
  }

  const styles = StyleSheet.create({
    imageContainer: {
      justifyContent: 'center',
      alignItems: 'center',
    },
  });

```

```

    glow: {
      width: 201,
      height: 201,
      position: 'absolute',
    },
    iconContainer: {
      justifyContent: 'center',
      alignItems: 'center',
      width: 128,
      height: 128,
      zIndex: 100,
    },
    image: {
      width: 76,
      height: 71,
    },
    background: {
      borderRadius: 40,
      experimental_backgroundImage: `linear-gradient(180deg, #3C9FFE, #0274DF)`,
      width: 128,
      height: 128,
      position: 'absolute',
    },
    splashOverlay: {
      ...StyleSheet.absoluteFill,
      backgroundColor: '#208AEF',
      alignItems: 'center',
      justifyContent: 'center',
      zIndex: 1000,
    },
  },
});

```

File: src/components/animated-icon.web.tsx

```

import { Image } from 'expo-image';
import { StyleSheet, View } from 'react-native';
import Animated, { Keyframe, Easing } from 'react-native-reanimated';

import classes from './animated-icon.module.css';
const DURATION = 300;

export function AnimatedSplashOverlay() {
  return null;
}

const keyframe = new Keyframe({
  0: {
    transform: [{ scale: 0 }],
  },
  60: {
    transform: [{ scale: 1.2 }],
    easing: Easing.elastic(1.2),
  },
  100: {
    transform: [{ scale: 1 }],
    easing: Easing.elastic(1.2),
  },
});

const logoKeyframe = new Keyframe({

```

```

    0: {
      opacity: 0,
    },
    60: {
      transform: [{ scale: 1.2 }],
      opacity: 0,
      easing: Easing.elastic(1.2),
    },
    100: {
      transform: [{ scale: 1 }],
      opacity: 1,
      easing: Easing.elastic(1.2),
    },
  },
});

const glowKeyframe = new Keyframe({
  0: {
    transform: [{ rotateZ: '-180deg' }, { scale: 0.8 }],
    opacity: 0,
  },
  [DURATION / 1000]: {
    transform: [{ rotateZ: '0deg' }, { scale: 1 }],
    opacity: 1,
    easing: Easing.elastic(0.7),
  },
  100: {
    transform: [{ rotateZ: '7200deg' }],
  },
});

export function AnimatedIcon() {
  return (
    <View style={styles.iconContainer}>
      <Animated.View entering={glowKeyframe.duration(60 * 1000 * 4)} style={styles.glow}>
        <Image style={styles.glow} source={require('@assets/images/logo-glow.png')} />
      </Animated.View>

      <Animated.View style={styles.background} entering={keyframe.duration(DURATION)}>
        <div className={classes.expoLogoBackground} />
      </Animated.View>

      <Animated.View style={styles.imageContainer} entering={logoKeyframe.duration(DURATION)}>
        <Image style={styles.image} source={require('@assets/images/expo-logo.png')} />
      </Animated.View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
    width: '100%',
    zIndex: 1000,
    position: 'absolute',
    top: 128 / 2 + 138,
  },
  imageContainer: {
    justifyContent: 'center',
    alignItems: 'center',
  },
  glow: {
    width: 201,
  },
});

```

```

        height: 201,
        position: 'absolute',
      },
      iconContainer: {
        justifyContent: 'center',
        alignItems: 'center',
        width: 128,
        height: 128,
      },
      image: {
        position: 'absolute',
        width: 76,
        height: 71,
      },
      background: {
        width: 128,
        height: 128,
        position: 'absolute',
      },
    },
  });

```

File: src\components\AppDrawerContent.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
  Image,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { useTheme } from "../../contexts/ThemeContext";

const LOGO = require("../../assets/images/akshaya-logo.png");

type AppDrawerContentProps = {
  onClose?: () => void;
  onViewTranscript?: () => void;
};

type AppRoute =
  | "/"
  | "/history"
  | "/account"
  | "/mvp-info"
  | "/documents"
  | "/settings"
  | "/about";

export default function AppDrawerContent({
  onClose,
  onViewTranscript,
}: AppDrawerContentProps) {
  const { colors } = useTheme();

  function navigate(path: AppRoute) {
    if (onClose) onClose();
  }

```

```

    router.push(path as any);
  }

  return (
    <View
      style={[
        styles.container,
        {
          backgroundColor: colors.surface,
        },
      ]}
    >
      <View
        style={[
          styles.brandCard,
          {
            backgroundColor: colors.primarySoft,
          },
        ]}
      >
        <Image
          source={LOGO}
          resizeMode="contain"
          style={styles.logo}
          accessibilityLabel="Akshaya Advisory logo"
        />

        <View style={styles.brandTextArea}>
          <Text
            style={[
              styles.brandTitle,
              {
                color: colors.text,
              },
            ]}
            >
            Akshaya Advisory
          </Text>

          <Text
            style={[
              styles.brandSubtitle,
              {
                color: colors.textMuted,
              },
            ]}
            >
            Citizen-service assistant
          </Text>
        </View>
      </View>

      <Pressable
        onPress={() => navigate("/") }
        accessibilityRole="button"
        accessibilityLabel="Start a new chat"
        style={({ pressed }) => [
          styles.newChatButton,
          {
            backgroundColor: colors.primary,
            opacity: pressed ? 0.8 : 1,
          },
        ],
      >

```

```

    ]]
    <Ionicons name="add-circle-outline" size={21} color="#FFFFFF" />

    <Text style={styles.newChatText}>Start a new chat</Text>
  </Pressable>

  <ScrollView
    showsVerticalScrollIndicator={false}
    contentContainerStyle={styles.menuContent}
  >
    <Text
      style={[
        styles.menuLabel,
        {
          color: colors.textMuted,
        },
      ]}
    >
      MAIN
    </Text>

    <DrawerItem
      icon="chatbubbles-outline"
      label="Chat"
      subtitle="Ask about your situation"
      onPress={() => navigate("/") }
    />

    <DrawerItem
      icon="time-outline"
      label="Chat history"
      subtitle="Open previous conversations"
      onPress={() => navigate("/history")}
    />

    {onViewTranscript && (
      <DrawerItem
        icon="document-text-outline"
        label="Current Transcript"
        subtitle="View session transcript"
        onPress={() => {
          if (onClose) onClose();
          onViewTranscript();
        }}
      />
    )}

    <Text
      style={[
        styles.menuLabel,
        styles.sectionSpacing,
        {
          color: colors.textMuted,
        },
      ]}
    >
      SERVICE INFORMATION
    </Text>

    <DrawerItem
      icon="grid-outline"

```

```

        label="MVP service guidance"
        subtitle="Supported citizen-service areas"
        onPress={() => navigate("/mvp-info")}
    />

    <DrawerItem
        icon="document-text-outline"
        label="Documents and preparation"
        subtitle="Understand the guidance scope"
        onPress={() => navigate("/documents")}
    />

    <Text
        style={[
            styles.menuLabel,
            styles.sectionSpacing,
            {
                color: colors.textMuted,
            },
        ]}
    />
    PERSONAL
    </Text>

    <DrawerItem
        icon="person-circle-outline"
        label="Account and privacy"
        subtitle="Data and accessibility information"
        onPress={() => navigate("/account")}
    />

    <DrawerItem
        icon="settings-outline"
        label="Settings and preferences"
        subtitle="Application behaviour and support"
        onPress={() => navigate("/settings")}
    />

    <Text
        style={[
            styles.menuLabel,
            styles.sectionSpacing,
            {
                color: colors.textMuted,
            },
        ]}
    />
    INFORMATION
    </Text>

    <DrawerItem
        icon="information-circle-outline"
        label="About and limitations"
        subtitle="Project scope and disclaimer"
        onPress={() => navigate("/about")}
    />
    </ScrollView>

    <View
        style={[
            styles.footer,
            {

```

```

        backgroundColor: colors.primarySoft,
      },
    ]}
  </>
  <Ionicons
    name="shield-checkmark-outline"
    size={20}
    color={colors.primaryDark}
  />

  <Text
    style={[
      styles.footerText,
      {
        color: colors.primaryDark,
      },
    ]}
  </>
  <Text>
    Advisory only. Verify important information with official sources.
  </Text>
</View>
</View>
);
}

```

```

function DrawerItem({
  icon,
  label,
  subtitle,
  onPress,
}: {
  icon: keyof typeof Ionicons.glyphMap;
  label: string;
  subtitle: string;
  onPress: () => void;
}) {
  const { colors } = useTheme();

  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      accessibilityLabel={label}
      accessibilityHint={subtitle}
      style={({ pressed }) => [
        styles.menuItem,
        {
          backgroundColor: pressed
            ? colors.primarySoft
            : "transparent",
          borderColor: pressed
            ? colors.borderStrong
            : "transparent",
        },
      ]}
    >
      <View
        style={[
          styles.menuIcon,
          {
            backgroundColor: colors.surfaceMuted,
          },
        ]}
      >

```

```

    ]}
    &gt;
    <Ionicons
      name={icon}
      size={20}
      color={colors.primaryDark}
    /&gt;
  </View>

  <View style={styles.menuTextArea}&gt;
    <Text
      style={[
        styles.menuTitle,
        {
          color: colors.text,
        },
      ]}
    &gt;
      {label}
    </Text>

    <Text
      style={[
        styles.menuSubtitle,
        {
          color: colors.textMuted,
        },
      ]}
    &gt;
      {subtitle}
    </Text>
  </View>

  <Ionicons
    name="chevron-forward"
    size={17}
    color={colors.textMuted}
  /&gt;
  </Pressable>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 17,
  },

  brandCard: {
    alignItems: "center",
    borderRadius: 16,
    flexDirection: "row",
    gap: 11,
    marginBottom: 15,
    padding: 12,
  },

  logo: {
    borderRadius: 10,
    height: 50,
    width: 50,
  },

```

```
brandTextArea: {
  flex: 1,
},

brandTitle: {
  fontFamily: "Manrope_800ExtraBold",
  fontSize: 15,
},

brandSubtitle: {
  fontFamily: "Manrope_400Regular",
  fontSize: 10,
  marginTop: 3,
},

newChatButton: {
  alignItems: "center",
  borderRadius: 14,
  flexDirection: "row",
  gap: 8,
  justifyContent: "center",
  minHeight: 50,
},

newChatText: {
  color: "#FFFFFF",
  fontFamily: "Manrope_700Bold",
  fontSize: 13,
},

menuContent: {
  paddingBottom: 15,
  paddingTop: 23,
},

menuLabel: {
  fontFamily: "Manrope_700Bold",
  fontSize: 10,
  letterSpacing: 1,
  marginBottom: 8,
},

sectionSpacing: {
  marginTop: 23,
},

menuItem: {
  alignItems: "center",
  borderRadius: 13,
  borderWidth: 1,
  flexDirection: "row",
  gap: 10,
  marginBottom: 5,
  minHeight: 66,
  paddingHorizontal: 9,
  paddingVertical: 9,
},

menuIcon: {
  alignItems: "center",
  borderRadius: 11,
  height: 39,
```

```

        justifyContent: "center",
        width: 39,
      },

      menuTextArea: {
        flex: 1,
      },

      menuTitle: {
        fontFamily: "Manrope_700Bold",
        fontSize: 12,
      },

      menuSubtitle: {
        fontFamily: "Manrope_400Regular",
        fontSize: 10,
        lineHeight: 15,
        marginTop: 3,
      },

      footer: {
        alignItems: "flex-start",
        borderRadius: 14,
        flexDirection: "row",
        gap: 8,
        marginTop: 8,
        padding: 12,
      },

      footerText: {
        flex: 1,
        fontFamily: "Manrope_600SemiBold",
        fontSize: 10,
        lineHeight: 15,
      },
    }
  });

```

File: src/components/ChatImagePreview.tsx

```

import React, { useState } from "react";
import { Image, Platform, View, Text, StyleSheet } from "react-native";
import { Ionicons } from "@expo/vector-icons";
import { useTheme } from "../contexts/ThemeContext";

export default function ChatImagePreview({ uri, width = 200, height = 250 }: { uri: string }) {
  const { colors } = useTheme();
  const [error, setError] = useState(false);

  if (!uri || error) {
    return (
      <View style={[styles.errorContainer, { backgroundColor: colors.surface, borderRadius: 14, padding: 12, gap: 8, alignItems: "center", justify-content: "center"}]}>
        <Ionicons name="image-outline" size={24} color={colors.textMuted} />
        <Text style={[styles.errorText, { color: colors.textMuted }]}>Preview unavailable</Text>
      </View>
    );
  }

  const isWebBlob = Platform.OS === "web" && uri.startsWith("blob:");

  return (
    <Image
      alt="Chat image preview"
      style={styles.image}
      source={isWebBlob ? { uri } : require(uri)}
      onError={() => setError(true)}
    />
  );
}

```

```

    <View style={styles.container}>
      {isWebBlob ? (
        <img
          src={uri}
          alt="Document preview"
          style={{
            maxWidth: '100%',
            maxHeight: height,
            objectFit: 'contain',
            borderRadius: 8,
            border: `1px solid ${colors.primarySoft}`,
          }}
          onError={() => setError(true)}
        />
      ) : (
        <Image
          source={{ uri }}
          style={[styles.image, { borderColor: colors.primarySoft, maxWidth: width, maxHeight: height, resizeMode: "contain" }]
          onError={() => setError(true)}
        />
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    marginVertical: 8,
    alignItems: "flex-start",
  },
  image: {
    width: "100%",
    borderRadius: 8,
    borderWidth: 1,
  },
  errorContainer: {
    width: 150,
    height: 100,
    justifyContent: "center",
    alignItems: "center",
    borderRadius: 8,
    borderWidth: 1,
    marginVertical: 8,
  },
  errorText: {
    fontSize: 10,
    marginTop: 4,
    fontFamily: "Manrope_400Regular",
  }
});

```

File: src/components/external-link.tsx

```

import { Href, Link } from 'expo-router';
import { openBrowserAsync, WebBrowserPresentationStyle } from 'expo-web-browser';
import { type ComponentProps } from 'react';

type Props = Omit<ComponentProps<Link>;, 'href'> & { href: Href &am

```

```

export function ExternalLink({ href, ...rest }: Props) {
  return (
    <Link
      target="_blank"
      {...rest}
      href={href}
      onPress={async (event) => {
        if (process.env.EXPO_OS !== 'web') {
          // Prevent the default behavior of linking to the default browser on native.
          event.preventDefault();
          // Open the link in an in-app browser.
          await openBrowserAsync(href, {
            presentationStyle: WebBrowserPresentationStyle.AUTOMATIC,
          });
        }
      }}
    />
  );
}

```

File: src/components/hint-row.tsx

```

import type { ReactNode } from 'react';
import { View, StyleSheet } from 'react-native';

import { ThemedText } from '../themed-text';
import { ThemedView } from '../themed-view';

import { Spacing } from '@constants/theme';

type HintRowProps = {
  title?: string;
  hint?: ReactNode;
};

export function HintRow({ title = 'Try editing', hint = 'app/index.tsx' }: HintRowProps) {
  return (
    <View style={styles.stepRow}>
      <ThemedText type="small">{title}</ThemedText>
      <ThemedView type="backgroundSelected" style={styles.codeSnippet}>
        <ThemedText themeColor="textSecondary">{hint}</ThemedText>
      </ThemedView>
    </View>
  );
}

const styles = StyleSheet.create({
  stepRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
  },
  codeSnippet: {
    borderRadius: Spacing.two,
    paddingVertical: Spacing.half,
    paddingHorizontal: Spacing.two,
  },
});

```

File: src\components\QueryComposer.tsx

```
import { Ionicons } from "@expo/vector-icons";
import * as ImagePicker from "expo-image-picker";
import { Audio } from "expo-av";
import { useState } from "react";
import {
  Alert,
  Image,
  Platform,
  Pressable,
  StyleSheet,
  Text,
  TextInput,
  View,
  ActivityIndicator,
} from "react-native";
import { API_BASE_URL } from "../config/api";
import { useTheme } from "../contexts/ThemeContext";

export type SelectedImage = {
  uri: string;
  mimeType: string;
  fileName: string;
  width: number;
  height: number;
};

type QueryComposerProps = {
  onSubmit: (query: string, image?: SelectedImage, inputType?: "text" | "voice") => void;
  disabled?: boolean;
  placeholder?: string;
};

const MAX_QUERY_LENGTH = 1000;

export default function QueryComposer({
  onSubmit,
  disabled = false,
  placeholder = "Describe your issue or question...",
}: QueryComposerProps) {
  const { colors } = useTheme();
  const [query, setQuery] = useState("");
  const [selectedImage, setSelectedImage] = useState<SelectedImage | null>(null);

  // Voice recording states
  const [recording, setRecording] = useState<Audio.Recording | null>(null);
  const [isRecording, setIsRecording] = useState(false);
  const [isTranscribing, setIsTranscribing] = useState(false);
  const [inputOrigin, setInputOrigin] = useState<"text" | "voice">("text");
  const [toastMessage, setToastMessage] = useState<string | null>(null);

  const cleanedQuery = query.trim();
  const canSubmit = (cleanedQuery.length > 0 || selectedImage !== null) && !di

  function showToast(msg: string) {
    setToastMessage(msg);
    setTimeout(() => {
      setToastMessage(null);
    }, 4000);
  }
}
```

```

async function startRecording() {
  try {
    if (Platform.OS === 'web') {
      showToast("Voice recording is not supported on Web. Please use the mobile app.");
      return;
    }
    const permission = await Audio.requestPermissionsAsync();
    if (permission.status !== 'granted') {
      Alert.alert("Permission Denied", "Microphone permission is required to use voice");
      return;
    }
    await Audio.setAudioModeAsync({
      allowsRecordingIOS: true,
      playsInSilentModeIOS: true,
    });

    const { recording } = await Audio.Recording.createAsync(
      Audio.RecordingOptionsPresets.HIGH_QUALITY
    );
    setRecording(recording);
    setIsRecording(true);
  } catch (err) {
    console.error("Failed to start recording", err);
    Alert.alert("Error", "Failed to start recording.");
  }
}

async function stopRecording() {
  if (!recording) return;
  setIsRecording(false);

  try {
    await recording.stopAndUnloadAsync();
    const uri = recording.getURI();
    setRecording(null);

    if (uri) {
      setIsTranscribing(true);
      // Upload audio to transcribe endpoint
      const formData = new FormData();
      const fileType = Platform.OS === 'ios' ? 'audio/m4a' : 'audio/m4a';
      const fileExtension = Platform.OS === 'ios' ? 'm4a' : 'm4a';

      formData.append("audio", {
        uri: Platform.OS === 'android' ? uri : uri.replace('file://', ''),
        name: `audio.${fileExtension}`,
        type: fileType,
      } as any);

      const response = await fetch(`${API_BASE_URL}/transcribe`, {
        method: "POST",
        body: formData,
      });

      if (!response.ok) {
        throw new Error(`Transcription failed: ${response.status}`);
      }

      const data = await response.json();
      if (data.text) {
        setQuery(prev => prev ? `${prev} ${data.text}` : data.text);
        setInputOrigin("voice");
      }
    }
  }
}

```

```

        } else {
            Alert.alert("No speech detected", "Could not transcribe any speech from the aud
        }
    }
} catch (err) {
    console.error("Transcription error:", err);
    Alert.alert("Error", "Transcription failed. Please try again.");
} finally {
    setIsTranscribing(false);
}
}
}

```

```

async function pickImage() {
    try {
        const permissionResult = await ImagePicker.requestMediaLibraryPermissionsAsync();
        if (permissionResult.granted === false) {
            setToastMessage("Permission to access camera roll is required!");
            return;
        }
        const result = await ImagePicker.launchImageLibraryAsync({
            mediaTypes: ImagePicker.MediaTypeOptions.Images,
            allowsEditing: false,
            quality: 0.85,
        });
        if (!result.canceled && result.assets?.length) {
            const asset = result.assets[0];
            setSelectedImage({
                uri: asset.uri,
                mimeType: asset.mimeType ?? "image/jpeg",
                fileName: asset.fileName ?? `document-${Date.now()}.jpg`,
                width: asset.width,
                height: asset.height,
            });
        }
    } catch (e) {
        console.error("[QueryComposer] Error in pickImage:", e);
    }
}
}

```

```

async function takePhoto() {
    try {
        const permissionResult = await ImagePicker.requestCameraPermissionsAsync();
        if (permissionResult.granted === false) {
            setToastMessage("Permission to access camera is required!");
            return;
        }
        const result = await ImagePicker.launchCameraAsync({
            mediaTypes: ImagePicker.MediaTypeOptions.Images,
            allowsEditing: false,
            quality: 0.85,
        });
        if (!result.canceled && result.assets?.length) {
            const asset = result.assets[0];
            setSelectedImage({
                uri: asset.uri,
                mimeType: asset.mimeType ?? "image/jpeg",
                fileName: asset.fileName ?? `document-${Date.now()}.jpg`,
                width: asset.width,
                height: asset.height,
            });
        }
    }
}

```

```

    } catch (e) {
      console.error("[QueryComposer] Error in takePhoto:", e);
    }
  }
}

```

```

function showAttachmentOptions() {
  try {
    if (Platform.OS === 'web') {
      pickImage();
    } else {
      Alert.alert("Attach Document", "Choose an option", [
        { text: "Take Photo", onPress: takePhoto },
        { text: "Choose from Gallery", onPress: pickImage },
        { text: "Cancel", style: "cancel" },
      ]);
    }
  } catch (e) {
    console.error("[QueryComposer] Error in showAttachmentOptions:", e);
  }
}

```

```

function handleSubmit() {
  if (!canSubmit) return;
  const currentImg = selectedImage;
  onSubmit(cleanedQuery, currentImg || undefined, inputOrigin);
  setQuery("");
  setSelectedImage(null);
  setInputOrigin("text");
}

```

```

return (
  <View style={styles.container}>
    <toastMessage &&& (
      <View style={{ marginHorizontal: 20, marginBottom: 10, padding: 10, background
        <Text style={{ color: colors.warning, fontSize: 12, textAlign: 'center', fon
          <toastMessage>
        </Text>
      </View>
    )}

    <selectedImage &&& (
      <View style={styles.previewContainer}>
        <Image source={{ uri: selectedImage.uri }} style={styles.previewImage} />
        <Pressable
          style={[styles.removeButton, { backgroundColor: colors.surface }]}
          onPress={() => setSelectedImage(null)}
        >
          <Ionicons name="close" size={14} color={colors.text} />
        </Pressable>
      </View>
    )}

    <Text style={[styles.privacyWarning, { color: colors.textMuted }]}>
      Avoid uploading photos showing your Aadhaar number, OTP, or other sensitive numbe
    </Text>

    <View style={styles.inputRow}>
      <Pressable
        onPress={showAttachmentOptions}
        disabled={disabled || isRecording || isTranscribing}
        style={styles.attachButton}
        accessibilityLabel="Attach Document"
      >

```

```

    &gt;
    <Ionicons
        name="attach-outline"
        size={28}
        color={colors.primary}
        style={{ transform: [{ rotate: "45deg" }] }}
    /&gt;
    <Text style={{ fontSize: 10, color: colors.primary, marginTop: 2 }}&gt;Attach
    </Pressable&gt;

    <Pressable
        onPress={isRecording ? stopRecording : startRecording}
        disabled={disabled || isTranscribing}
        style={styles.micButton}
        accessibilityLabel={isRecording ? "Stop Recording" : "Start Voice Input"}
    &gt;
        {isTranscribing ? (
            <ActivityIndicator size="small" color={colors.primary} /&gt;
        ) : (
            <&gt;
                <View style={[styles.micIconContainer, isRecording && styles.rec...
                    <Ionicons
                        name={isRecording ? "stop" : "mic"}
                        size={24}
                        color={isRecording ? "#FFF" : colors.primary}
                    /&gt;
                </View&gt;
                <Text style={{ fontSize: 10, color: isRecording ? '#e74c3c' : colors.pri...
                    {isRecording ? "Recording" : "Voice"}
                </Text&gt;
            </&gt;
        )}
    </Pressable&gt;

    <View
        style={[
            styles.pillInputContainer,
            {
                backgroundColor: colors.surface,
                borderColor: colors.border,
            },
        ]}
    &gt;
        <TextInput
            value={query}
            onChangeText={(text) => {
                if (text.length === 1 && query.length === 0) setInputOrigin("text")
                if (text.length <= MAX_QUERY_LENGTH) setQuery(text);
            }}
            onPress={(e: any) => {
                if (e.nativeEvent.key === 'Enter' && !e.nativeEvent.shiftKey) {
                    e.preventDefault();
                    handleSubmit();
                }
            }}
            placeholder={isRecording ? "Listening..." : isTranscribing ? "Transcribing..."
            placeholderTextColor={isRecording ? "#e74c3c" : colors.textMuted}
            multiline
            maxLength={MAX_QUERY_LENGTH}
            editable={!disabled && !isRecording && !isTranscribing}
            style={[styles.input, { color: colors.text }]}
        /&gt;

```

```

    </View>

    <Pressable
      onPress={handleSubmit}
      disabled={!canSubmit || isRecording || isTranscribing}
      accessibilityLabel="Send Message"
      style={({ pressed }) => [
        styles.sendButton,
        {
          backgroundColor: colors.primarySoft,
          opacity: pressed & canSubmit ? 0.7 : 1,
        },
      ]}
    >
      <Ionicons
        name="paper-plane-outline"
        size={20}
        color={canSubmit ? colors.primaryDark : colors.textMuted}
        style={{ marginLeft: 2, transform: [{ rotate: "45deg" }] }}
      />
      <Text style={{ fontSize: 10, color: canSubmit ? colors.primaryDark : colors.
    </Pressable>
  </View>
  </View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    width: "100%",
    paddingVertical: 10,
    backgroundColor: "#FFFFFF",
  },
  privacyWarning: {
    fontFamily: "Manrope_400Regular",
    fontSize: 11,
    marginBottom: 8,
    textAlign: "center",
    paddingHorizontal: 20,
  },
  previewContainer: {
    position: "relative",
    width: 60,
    height: 60,
    marginBottom: 10,
    marginLeft: 50,
  },
  previewImage: {
    width: "100%",
    height: "100%",
    borderRadius: 8,
  },
  removeButton: {
    position: "absolute",
    top: -8,
    right: -8,
    width: 24,
    height: 24,
    borderRadius: 12,
    alignItems: "center",
    justifyContent: "center",
    shadowColor: "#000",
  },
});

```

```

        shadowOffset: { width: 0, height: 1 },
        shadowOpacity: 0.2,
        shadowRadius: 1.5,
        elevation: 2,
    },
    inputRow: {
        flexDirection: "row",
        alignItems: "center",
        gap: 8,
        paddingHorizontal: 12,
    },
    attachButton: {
        padding: 4,
        justifyContent: "center",
        alignItems: "center",
        width: 48,
    },
    micButton: {
        padding: 4,
        justifyContent: "center",
        alignItems: "center",
        width: 48,
    },
    micIconContainer: {
        width: 32,
        height: 32,
        borderRadius: 16,
        justifyContent: "center",
        alignItems: "center",
        backgroundColor: "transparent",
    },
    recordingActive: {
        backgroundColor: "#e74c3c",
        shadowColor: "#e74c3c",
        shadowOffset: { width: 0, height: 0 },
        shadowOpacity: 0.8,
        shadowRadius: 5,
        elevation: 5,
    },
    pillInputContainer: {
        flex: 1,
        borderRadius: 20,
        borderWidth: 1.5,
        minHeight: 52,
        paddingHorizontal: 16,
        justifyContent: "center",
        paddingVertical: 8,
    },
    input: {
        fontFamily: "Manrope_400Regular",
        fontSize: 15,
        lineHeight: 20,
        maxHeight: 100,
        paddingTop: 0,
        paddingBottom: 0,
    },
    sendButton: {
        width: 48,
        height: 48,
        borderRadius: 24,
        justifyContent: "center",
        alignItems: "center",
    },

```

```

    },
  });

```

File: src/components\ServiceCategoryCard.tsx

```

import { Ionicons } from "@expo/vector-icons";
import { router } from "expo-router";
import {
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { useTheme } from "../../contexts/ThemeContext";

type ServiceCategoryCardProps = {
  category: string;
  title: string;
  description: string;
  icon: keyof typeof Ionicons.glyphMap;
};

export default function ServiceCategoryCard({
  category,
  title,
  description,
  icon,
}: ServiceCategoryCardProps) {
  const { colors } = useTheme();

  function openChat() {
    router.push({
      pathname: "/",
      params: {
        category,
      },
    });
  }

  return (
    <Pressable
      onPress={openChat}
      accessibilityRole="button"
      accessibilityLabel={`Ask about ${title}`}
      accessibilityHint="Opens the main chat with this topic as context"
      style={({ pressed }) => [
        styles.card,
        {
          backgroundColor: colors.surface,
          borderColor: colors.border,
          opacity: pressed ? 0.78 : 1,
        },
      ]}
    >
    <View
      style={[
        styles.iconBox,
        {
          backgroundColor: colors.primarySoft,

```

```

    },
  ]}
  &gt;
  <Ionicons name={icon} size={23} color={colors.primaryDark} /&gt;
</View>

<Text
  style={[
    styles.title,
    {
      color: colors.text,
    },
  ]}
  &gt;
  {title}
</Text>

<Text
  style={[
    styles.description,
    {
      color: colors.textMuted,
    },
  ]}
  &gt;
  {description}
</Text>

<View style={styles.footer}&gt;
  <Text
    style={[
      styles.footerText,
      {
        color: colors.primaryDark,
      },
    ]}
    &gt;
    Ask about this
  </Text>

  <Ionicons
    name="arrow-forward"
    size={16}
    color={colors.primaryDark}
  /&gt;
</View>
</Pressable>
);
}

const styles = StyleSheet.create({
  card: {
    borderRadius: 18,
    borderWidth: 1,
    minHeight: 205,
    padding: 15,
    width: 170,
  },
  iconBox: {
    alignItems: "center",
    borderRadius: 12,

```

```

    height: 42,
    justifyContent: "center",
    width: 42,
  },

  title: {
    fontSize: 14,
    fontWeight: "800",
    lineHeight: 19,
    marginTop: 14,
  },

  description: {
    fontSize: 11,
    lineHeight: 17,
    marginTop: 7,
  },

  footer: {
    alignItems: "center",
    bottom: 14,
    flexDirection: "row",
    gap: 5,
    position: "absolute",
    right: 14,
  },

  footerText: {
    fontSize: 10,
    fontWeight: "800",
  },
});

```

File: src\components\themed-text.tsx

```

import { Platform, StyleSheet, Text, type TextProps } from 'react-native';

import { Fonts, ThemeColor } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

export type ThemedTextProps = TextProps & {
  type?: 'default' | 'title' | 'small' | 'smallBold' | 'subtitle' | 'link' | 'linkPrimary'
  themeColor?: ThemeColor;
};

export function ThemedText({ style, type = 'default', themeColor, ...rest }: ThemedTextPr
const theme = useTheme();

return (
  <Text
    style={
      [
        { color: theme[themeColor ?? 'text'] },
        type === 'default' && styles.default,
        type === 'title' && styles.title,
        type === 'small' && styles.small,
        type === 'smallBold' && styles.smallBold,
        type === 'subtitle' && styles.subtitle,
        type === 'link' && styles.link,
        type === 'linkPrimary' && styles.linkPrimary,
        type === 'code' && styles.code,

```

```

        style,
      ]}
      {...rest}
    /&gt;
  );
}

const styles = StyleSheet.create({
  small: {
    fontSize: 14,
    lineHeight: 20,
    fontWeight: 500,
  },
  smallBold: {
    fontSize: 14,
    lineHeight: 20,
    fontWeight: 700,
  },
  default: {
    fontSize: 16,
    lineHeight: 24,
    fontWeight: 500,
  },
  title: {
    fontSize: 48,
    fontWeight: 600,
    lineHeight: 52,
  },
  subtitle: {
    fontSize: 32,
    lineHeight: 44,
    fontWeight: 600,
  },
  link: {
    lineHeight: 30,
    fontSize: 14,
  },
  linkPrimary: {
    lineHeight: 30,
    fontSize: 14,
    color: '#3c87f7',
  },
  code: {
    fontFamily: Fonts.mono,
    fontWeight: Platform.select({ android: 700 }) ?? 500,
    fontSize: 12,
  },
});

```

File: src/components/themed-view.tsx

```

import { View, type ViewProps } from 'react-native';

import { ThemeColor } from '@constants/theme';
import { useTheme } from '@hooks/use-theme';

export type ThemedViewProps = ViewProps & {
  lightColor?: string;
  darkColor?: string;
  type?: ThemeColor;
};

```

```

};

export function ThemedView({ style, lightColor, darkColor, type, ...otherProps }: ThemedV
  const theme = useTheme();

  return <View style={[{ backgroundColor: theme[type ?? 'background'] }, style]} {...o
}

```

File: src\components\web-badge.tsx

```

import { version } from 'expo/package.json';
import { Image } from 'expo-image';
import { useColorScheme, StyleSheet } from 'react-native';

import { ThemedText } from '../themed-text';
import { ThemedView } from '../themed-view';

import { Spacing } from '@constants/theme';

export function WebBadge() {
  const scheme = useColorScheme();

  return (
    <ThemedView style={styles.container}>
      <ThemedText type="code" themeColor="textSecondary" style={styles.versionText}>v{version}</ThemedText>
      <Image
        source={
          scheme === 'dark'
            ? require('@assets/images/expo-badge-white.png')
            : require('@assets/images/expo-badge.png')
        }
        style={styles.badgeImage}
      />
    </ThemedView>
  );
}

const styles = StyleSheet.create({
  container: {
    padding: Spacing.five,
    alignItems: 'center',
    gap: Spacing.two,
  },
  versionText: {
    textAlign: 'center',
  },
  badgeImage: {
    width: 123,
    aspectRatio: 123 / 24,
  },
});

```

File: src\components\uil\collapsible.tsx

```

import { SymbolView } from 'expo-symbols';

```

```

import { PropsWithChildren, useState } from 'react';
import { Pressable, StyleSheet } from 'react-native';
import Animated, { FadeIn } from 'react-native-reanimated';

import { ThemedText } from '@components/themed-text';
import { ThemedView } from '@components/themed-view';
import { Spacing } from '@constants/theme';
import { useTheme } from '@hooks/use-theme';

export function Collapsible({ children, title }: PropsWithChildren & { title: string }) {
  const [isOpen, setIsOpen] = useState(false);
  const theme = useTheme();

  return (
    <ThemedView>
      <Pressable
        style={({ pressed }) => [styles.heading, pressed && styles.pressedHeading,
          onPress={() => setIsOpen(value) => !value}]>
        <ThemedView type="backgroundElement" style={styles.button}>
          <SymbolView
            name={{ ios: 'chevron.right', android: 'chevron_right', web: 'chevron_right' }}
            size={14}
            weight="bold"
            tint={theme.text}
            style={{ transform: [{ rotate: isOpen ? '-90deg' : '90deg' }] }}
          />
        </ThemedView>

        <ThemedText type="small">{title}</ThemedText>
      </Pressable>
      {isOpen && (
        <Animated.View entering={FadeIn.duration(200)}>
          <ThemedView type="backgroundElement" style={styles.content}>
            {children}
          </ThemedView>
        </Animated.View>
      )}
    </ThemedView>
  );
}

const styles = StyleSheet.create({
  heading: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: Spacing.two,
  },
  pressedHeading: {
    opacity: 0.7,
  },
  button: {
    width: Spacing.four,
    height: Spacing.four,
    borderRadius: 12,
    justifyContent: 'center',
    alignItems: 'center',
  },
  content: {
    marginTop: Spacing.three,
    borderRadius: Spacing.three,
    marginLeft: Spacing.four,
    padding: Spacing.four,
  },
});

```

```
    },
  });
};
```

File: src\config\api.ts

```
// Centralized API config to prevent hardcoded localhost scattered across the app.
// Note on Platform-specific resolution:
// If device testing is planned, API_BASE_URL needs to be your machine's LAN IP
// (or a tunneled URL) when running on a physical device, not localhost, because
// 'localhost' on a device// For a downloadable APK tested on your local Wi-Fi, we use yo
// If you host the backend on the cloud later (e.g. Render/Heroku), change this to your c
import { Platform } from 'react-native';

// Add fallback for Android Emulator if physical IP doesn't work
const getLocalIP = () => {
  if (Platform.OS === 'web') return 'http://127.0.0.1:8001';
  if (Platform.OS === 'android' && !process.env.EXPO_PUBLIC_API_URL) {
    // If you're on emulator, 10.0.2.2 points to host's localhost
    return 'http://10.0.2.2:8001';
  }
  return 'http://192.168.16.241:8001';
};

export const API_BASE_URL = process.env.EXPO_PUBLIC_API_URL || getLocalIP();
```

File: src\constants\colors.ts

```
export const colors = {
  primary: "#0077B6",
  primaryDark: "#023E8A",
  primaryPressed: "#0096C7",
  primarySoft: "#E8F4F8",
  primarySoftStrong: "#CAF0F8",

  accent: "#00B4D8",
  accentSoft: "#E0FBFC",

  background: "#FFFFFF",
  surface: "#FFFFFF",
  surfaceMuted: "#F0F8FA",
  border: "#D0E6EE",
  borderStrong: "#90E0EF",

  text: "#03045E",
  textMuted: "#5C7C8A",
  textLight: "#8A9FA8",
  textOnPrimary: "#FFFFFF",

  warning: "#936400",
  warningBackground: "#FFF6DE",

  danger: "#C84040",
  dangerBackground: "#FDEBEC",

  overlay: "rgba(2, 62, 138, 0.42)",
};
```

```
export type AppColors = typeof colors;
```

File: src\constants\services.ts

```
export const SERVICE_CATEGORIES = [
  {
    id: "aadhaar_update",
    title: "Aadhaar updates",
    description:
      "Understand possible Aadhaar update guidance and supporting-document information.",
    icon: "card-outline",
  },
  {
    id: "identity_mismatch",
    title: "Identity mismatch",
    description:
      "Understand possible next steps when names or identity details differ.",
    icon: "git-compare-outline",
  },
  {
    id: "passport_support",
    title: "Passport support",
    description:
      "Prepare for passport-related guidance before visiting a service centre.",
    icon: "airplane-outline",
  },
  {
    id: "documents",
    title: "Document checklist",
    description:
      "Review possible document guidance where official evidence is available.",
    icon: "documents-outline",
  },
  {
    id: "eligibility",
    title: "Eligibility support",
    description:
      "Understand available eligibility information from selected official sources.",
    icon: "checkmark-circle-outline",
  },
  {
    id: "next_steps",
    title: "Next steps",
    description:
      "Receive practical advisory steps and official-verification guidance.",
    icon: "navigate-outline",
  },
] as const;
```

File: src\constants\theme.ts

```
/**
 * Below are the colors that are used in the app. The colors are defined in the light and
 * There are many other ways to style your app. For example, [Nativewind](https://www.nat
 */

import '@global.css';
```

```

import { Platform } from 'react-native';

export const Colors = {
  light: {
    text: '#000000',
    background: '#ffffff',
    backgroundElement: '#F0F0F3',
    backgroundSelected: '#E0E1E6',
    textSecondary: '#60646C',
  },
  dark: {
    text: '#ffffff',
    background: '#000000',
    backgroundElement: '#212225',
    backgroundSelected: '#2E3135',
    textSecondary: '#B0B4BA',
  },
} as const;

export type ThemeColor = keyof typeof Colors.light & keyof typeof Colors.dark;

export const Fonts = Platform.select({
  ios: {
    /** iOS `UIFontDescriptorSystemDesignDefault` */
    sans: 'system-ui',
    /** iOS `UIFontDescriptorSystemDesignSerif` */
    serif: 'ui-serif',
    /** iOS `UIFontDescriptorSystemDesignRounded` */
    rounded: 'ui-rounded',
    /** iOS `UIFontDescriptorSystemDesignMonospaced` */
    mono: 'ui-monospace',
  },
  default: {
    sans: 'normal',
    serif: 'serif',
    rounded: 'normal',
    mono: 'monospace',
  },
  web: {
    sans: 'var(--font-display)',
    serif: 'var(--font-serif)',
    rounded: 'var(--font-rounded)',
    mono: 'var(--font-mono)',
  },
});

export const Spacing = {
  half: 2,
  one: 4,
  two: 8,
  three: 16,
  four: 24,
  five: 32,
  six: 64,
} as const;

export const BottomTabInset = Platform.select({ ios: 50, android: 80 }) ?? 0;
export const MaxContentWidth = 800;

```

File: src\contexts\ThemeContext.tsx

```
import { createContext, PropsWithChildren, useContext, useMemo } from "react";
import { AppColors, colors as fixedColors } from "../constants/colors";

type ThemeContextValue = {
  colors: AppColors;
};

const ThemeContext = createContext<ThemeContextValue | null>(null);

export function ThemeProvider({ children }: PropsWithChildren) {
  const value = useMemo(
    () => ({
      colors: fixedColors,
    }),
    [],
  );

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);

  if (!context) {
    throw new Error("useTheme must be used inside ThemeProvider");
  }

  return context;
}
```

File: src\hooks\use-color-scheme.ts

```
export { useColorScheme } from 'react-native';
```

File: src\hooks\use-color-scheme.web.ts

```
import { useEffect, useState } from 'react';
import { useColorScheme as useRNColorScheme } from 'react-native';

/**
 * To support static rendering, this value needs to be re-calculated on the client side
 */
export function useColorScheme() {
  const [hasHydrated, setHasHydrated] = useState(false);

  useEffect(() => {
    setHasHydrated(true);
  }, []);

  const colorScheme = useRNColorScheme();
```

```

    if (hasHydrated) {
      return colorScheme;
    }

    return 'light';
  }

```

File: src\hooks\use-theme.ts

```

/**
 * Learn more about light and dark modes:
 * https://docs.expo.dev/guides/color-schemes/
 */

import { Colors } from '@constants/theme';
import { useColorScheme } from '@hooks/use-color-scheme';

export function useTheme() {
  const scheme = useColorScheme();
  const theme = scheme === 'unspecified' ? 'light' : scheme;

  return Colors[theme];
}

```

File: src\utils\cleanResponseText.ts

```

export function cleanText(value: string | undefined | null): string {
  if (!value) return '';
  let clean = value;

  // Replace breaks with newlines
  clean = clean.replace(/<br\s*\/?>/gi, '\n');
  clean = clean.replace(/<\/li>\s*<li>/gi, '\n');
  clean = clean.replace(/<li[^>]*>/gi, '');
  clean = clean.replace(/<\/li>/gi, '');

  // Remove other html tags
  clean = clean.replace(/<\/?(ul|ol|p|strong|b|em|i|div|span)[^>]*>/gi, '');
  clean = clean.replace(/<[>]+>/g, '');

  // Unescape common HTML entities
  clean = clean.replace(/&amp;/g, '&');
  clean = clean.replace(/&lt;/g, '<');
  clean = clean.replace(/&gt;/g, '>');
  clean = clean.replace(/&quot;/g, '"');
  clean = clean.replace(/&#39;/g, "'");

  // Cleanup spacing
  clean = clean.replace(/[ \t]+/g, ' ');
  clean = clean.replace(/\n[ \t]+/g, '\n');

  // Remove rogue markdown bold/italics
  clean = clean.replace(/\*\*/g, '');
  clean = clean.replace(/__/_/g, '');

  return clean.trim();
}

```

File: src\utils\device.ts

```
import { Platform } from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

const DEVICE_ID_KEY = 'akshaya_device_id';

function generateId(): string {
  return 'device_' + Math.random().toString(36).substring(2, 15) + Math.random().toString(36).substring(2, 15);
}

let memoryDeviceId: string | null = null;

export async function getDeviceId(): Promise<string> {
  if (memoryDeviceId) {
    return memoryDeviceId;
  }

  try {
    if (Platform.OS === 'web') {
      let id = localStorage.getItem(DEVICE_ID_KEY);
      if (!id) {
        id = generateId();
        localStorage.setItem(DEVICE_ID_KEY, id);
      }
      memoryDeviceId = id;
      return id;
    } else {
      let id = await AsyncStorage.getItem(DEVICE_ID_KEY);
      if (!id) {
        id = generateId();
        await AsyncStorage.setItem(DEVICE_ID_KEY, id);
      }
      memoryDeviceId = id;
      return id;
    }
  } catch (error) {
    console.error("Failed to get/set device ID:", error);
    return "fallback-device-id";
  }
}
```

File: src\utils\responseAdapter.ts

```
export function normalizeChatResponse(data: any): any {
  let summary = "";
  let answer = data.answer || {};

  if (answer.summary) {
    summary = answer.summary.trim();
  } else if (data.summary) {
    summary = data.summary.trim();
  }

  if (!summary) {
    summary = "I could not generate a clear summary for this response. Please review the response."
  }
}
```

```
if (data.success === false && data.status === "image_analysis_failed") {
  summary = answer.summary || "I could not evaluate this image right now. Please upload"
}

return {
  ...data,
  success: data.success !== false,
  status: data.status || "grounded_answer",
  answer: {
    ...answer,
    summary
  }
};
}
```